Statistical Machine Learning (BE4M33SSU)
Support Vector Machines

Czech Technical University in Prague
V.Franc

Linear classifier with minimal classification error

- $\mathcal{X}$ is a set of observations and $\mathcal{Y} = \{+1, -1\}$ a set of hidden labels
- $\phi\colon \mathcal{X} \to \mathbb{R}^n$ is fixed feature map embedding $\mathcal{X}$ to $\mathbb{R}^n$
- **Linear classifier** (classification strategy) $h\colon \mathcal{X} \to \mathcal{Y}$:

$$h(x; \boldsymbol{w}, b) = \operatorname{sign}(f(x; \boldsymbol{w}, b)) \qquad \text{where} \qquad f(x; \boldsymbol{w}, b) = \langle \boldsymbol{w}, \boldsymbol{\phi}(x) \rangle + b$$

  is a scoring function linear in parameters.

- **Task:** find linear classifier with the minimal probablity of misclassification

$$R^{0/1}(h) = \mathbb{E}_{(x,y)\sim p}\Big(\ell^{0/1}(y, h(x))\Big) \quad \text{where} \quad \ell^{0/1}(y, y') = [\![y \neq y']\!]$$

- We are given a set of training examples

$$\mathcal{T}^m = \{(x^i, y^i) \in (\mathcal{X} \times \mathcal{Y}) \mid i = 1, \ldots, m\}$$

  i.i.d. drawn from the distribution $p(x, y)$.

ERM applied to learning linear classifier

- The Empirical Risk Minimization principle leads to solving

$$(\boldsymbol{w}^*, b^*) \in \underset{(\boldsymbol{w},b)\in(\mathbb{R}^n\times\mathbb{R})}{\operatorname{Argmin}} R^{0/1}_{\mathcal{T}^m}(h(\cdot;\boldsymbol{w},b)) \tag{1}$$

where the empirical risk is

$$R^{0/1}_{\mathcal{T}^m}(h(\cdot;\boldsymbol{w},b)) = \frac{1}{m}\sum_{i=1}^{m}[\![y^i \neq h(x^i;\boldsymbol{w},b)]\!]$$

ERM applied to learning linear classifier

- The Empirical Risk Minimization principle leads to solving

$$(\boldsymbol{w}^*, b^*) \in \underset{(\boldsymbol{w},b)\in(\mathbb{R}^n\times\mathbb{R})}{\operatorname{Argmin}} R^{0/1}_{\mathcal{T}^m}(h(\cdot;\boldsymbol{w},b)) \tag{1}$$

  where the empirical risk is

$$R^{0/1}_{\mathcal{T}^m}(h(\cdot;\boldsymbol{w},b)) = \frac{1}{m}\sum_{i=1}^{m}[\![y^i \neq h(x^i;\boldsymbol{w},b)]\!]$$

- **Algorithmic issue:** there is no known algorithm solving the task (1) in time polynomial in feature dimension n and the number of examples m.

  Solution: Replace 0/1-loss by the Hinge loss.

ERM applied to learning linear classifier

- The Empirical Risk Minimization principle leads to solving

$$(\boldsymbol{w}^*, b^*) \in \underset{(\boldsymbol{w},b)\in(\mathbb{R}^n\times\mathbb{R})}{\operatorname{Argmin}} R^{0/1}_{\mathcal{T}^m}(h(\cdot;\boldsymbol{w},b)) \tag{1}$$

  where the empirical risk is

$$R^{0/1}_{\mathcal{T}^m}(h(\cdot;\boldsymbol{w},b)) = \frac{1}{m}\sum_{i=1}^{m}[\![y^i \neq h(x^i;\boldsymbol{w},b)]\!]$$

- **Algorithmic issue:** there is no known algorithm solving the task (1) in time polynomial in feature dimension n and the number of examples m.

  Solution: Replace 0/1-loss by the Hinge loss.

- **Statistical concern:** even with with a finite VC dimension, linear classifiers can still overfit when the feature dimension n is high and the number of examples m is small.

  Solution: Control the $L2$ norm of the parameter vector $\boldsymbol{w}$.

Replacing intractable 0/1-loss with the Hinge-loss

- The intractable ERM problem we wish to solve

$$(\boldsymbol{w}^*, b^*) \in \underset{(\boldsymbol{w},b)\in(\mathbb{R}^n\times\mathbb{R})}{\operatorname{Argmin}} \frac{1}{m}\sum_{i=1}^{m} \underbrace{[\![y^i \neq h(x^i;\boldsymbol{w},b))]\!]}_{\ell^{0/1}(y^i,h(x^i;\boldsymbol{w},b))}$$

where $h(x;\boldsymbol{w},b) = \operatorname{sign}(f(x;\boldsymbol{w},b))$ and $f(x;\boldsymbol{w},b) = \langle \boldsymbol{w}, \boldsymbol{\phi}(x)\rangle + b$.

Replacing intractable 0/1-loss with the Hinge-loss

- The intractable ERM problem we wish to solve

$$(\boldsymbol{w}^*, b^*) \in \operatorname*{Argmin}_{(\boldsymbol{w},b)\in(\mathbb{R}^n\times\mathbb{R})} \frac{1}{m}\sum_{i=1}^{m} \underbrace{[\![y^i \neq h(x^i;\boldsymbol{w},b))]\!]}_{\ell^{0/1}(y^i,h(x^i;\boldsymbol{w},b))}$$

where $h(x;\boldsymbol{w},b) = \operatorname{sign}(f(x;\boldsymbol{w},b))$ and $f(x;\boldsymbol{w},b) = \langle \boldsymbol{w}, \boldsymbol{\phi}(x)\rangle + b$.

- The ERM problem is approximated by a tractable **convex problem**

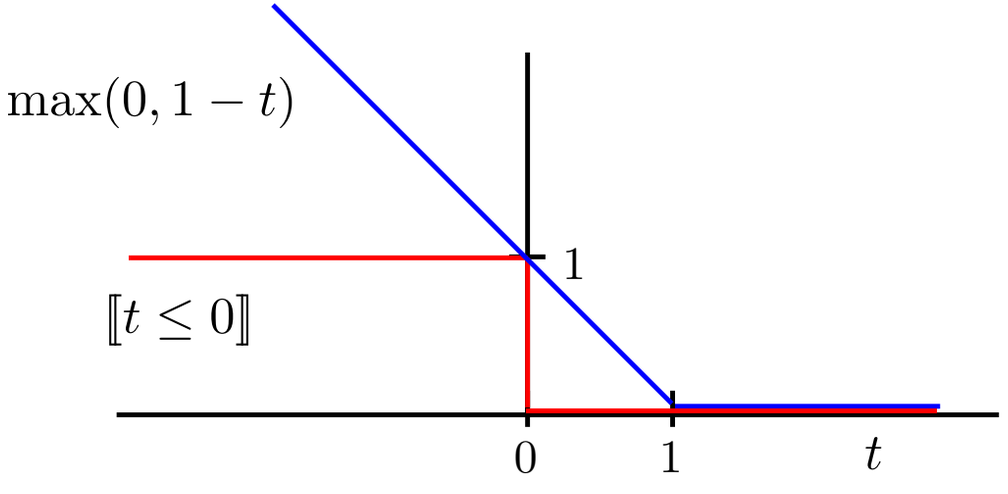
$$(\boldsymbol{w}^*, b^*) \in \operatorname*{Argmin}_{(\boldsymbol{w},b)\in(\mathbb{R}^n\times\mathbb{R})} \frac{1}{m}\sum_{i=1}^{m} \underbrace{\max\{0, 1 - y^i f(x^i;\boldsymbol{w},b)\}}_{\psi(y^i,f(x^i;\boldsymbol{w},b))}$$

where $\psi(y, f(x))$ is so called Hinge-loss.

The hinge-loss upper bounds the 0/1-loss

- The hinge-loss is an upper bound of the $0/1$-loss evaluated for the predictor $h(x) = \operatorname{sign}(f(x))$:

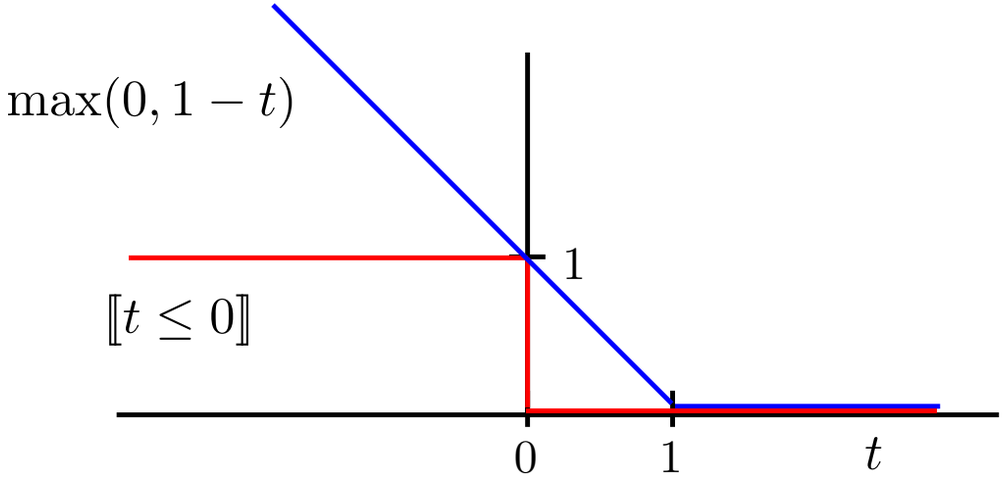
$$\underbrace{[\![\operatorname{sign}(f(x)) \neq y]\!]}_{\ell^{0/1}(y, f(x))} = [\![\, y\, f(x) \leq 0]\!] \leq \underbrace{\max\{0, 1 - y\ f(x)\}}_{\psi(y, f(x))}$$



The hinge-loss upper bounds the 0/1-loss

- The hinge-loss is an upper bound of the $0/1$-loss evaluated for the predictor $h(x) = \text{sign}(f(x))$:

$$\underbrace{[\![\text{sign}(f(x)) \neq y]\!]}_{\ell^{0/1}(y, f(x))} = [\![\, y\, f(x) \leq 0]\!] \leq \underbrace{\max\{0, 1 - y\ f(x)\}}_{\psi(y, f(x))}$$



- Consequently, the average hinge-loss upper upper bounds the average $0/1$-loss:

$$\frac{1}{m}\sum_{i=1}^{m} \underbrace{[\![y^i \neq h(x^i; \boldsymbol{w}, b))]\!]}_{\ell^{0/1}(y^i, h(x^i; \boldsymbol{w}, b))} \leq \frac{1}{m}\sum_{i=1}^{m} \underbrace{\max\{0, 1 - y^i f(x^i; \boldsymbol{w}, b)\}}_{\psi(y^i, f(x^i; \boldsymbol{w}, b))}$$

Support Vector Machines = ERM with the hinge loss and a constraint on the parameter norm

- Recall that ERM applied to $\mathcal{H} = \{h(x) = \operatorname{sign}(\langle \boldsymbol{\phi}(x), \boldsymbol{w} \rangle + b) \mid \boldsymbol{w} \in \mathbb{R}^n, b \in \mathbb{R}\}$ with $0/1$-loss is PAC successful with the sample complexity $m_{\mathcal{H}}^{\text{pac}}(\varepsilon, \delta) = \mathcal{O}\big(\frac{n+1-\log \delta}{\varepsilon^2}\big)$.

Support Vector Machines = ERM with the hinge loss and a constraint on the parameter norm

- Recall that ERM applied to $\mathcal{H} = \{h(x) = \operatorname{sign}(\langle \boldsymbol{\phi}(x), \boldsymbol{w} \rangle + b) \mid \boldsymbol{w} \in \mathbb{R}^n, b \in \mathbb{R}\}$ with $0/1$-loss is PAC successful with the sample complexity $m^{\text{pac}}_{\mathcal{H}}(\varepsilon, \delta) = \mathcal{O}\big(\frac{n+1-\log \delta}{\varepsilon^2}\big)$.
- A hypothesis space of linear scores with parameter vector inside a ball of radius r:

$$\mathcal{F}_r = \big\{ f(x) = \langle \boldsymbol{\phi}(x), \boldsymbol{w} \rangle + b \mid (\boldsymbol{w}, b) \in \mathbb{R}^{n+1}, \|\boldsymbol{w}\| \leq r \big\}$$

- ERM applied to learning scores from $\mathcal{F}_r$ with the hinge loss $\psi(y, t) = \max\{0, 1 - t\}$ is PAC successful with the sample complexity $m^{\text{pac}}_{\mathcal{F}_r}(\varepsilon, \delta) = \mathcal{O}\big(\frac{r^2}{\varepsilon^2\, \delta^2}\big)$.

Support Vector Machines = ERM with the hinge loss and a constraint on the parameter norm

- Recall that ERM applied to $\mathcal{H} = \{h(x) = \operatorname{sign}(\langle \boldsymbol{\phi}(x), \boldsymbol{w} \rangle + b) \mid \boldsymbol{w} \in \mathbb{R}^n, b \in \mathbb{R}\}$ with $0/1$-loss is PAC successful with the sample complexity $m^{\text{pac}}_{\mathcal{H}}(\varepsilon, \delta) = \mathcal{O}\big(\frac{n+1-\log \delta}{\varepsilon^2}\big)$.
- A hypothesis space of linear scores with parameter vector inside a ball of radius r:

$$\mathcal{F}_r = \big\{ f(x) = \langle \boldsymbol{\phi}(x), \boldsymbol{w} \rangle + b \mid (\boldsymbol{w}, b) \in \mathbb{R}^{n+1}, \|\boldsymbol{w}\| \leq r \big\}$$

- ERM applied to learning scores from $\mathcal{F}_r$ with the hinge loss $\psi(y, t) = \max\{0, 1 - t\}$ is PAC successful with the sample complexity $m^{\text{pac}}_{\mathcal{F}_r}(\varepsilon, \delta) = \mathcal{O}\big(\frac{r^2}{\varepsilon^2\, \delta^2}\big)$.
- ERM instantiated for $\mathcal{F}_r$ and the hinge loss $\psi(y, t)$ leads to:

$$f^* = \operatorname*{Argmin}_{f \in \mathcal{F}_r} R^{\psi}_{\mathcal{T}^m}(f) \quad \text{where} \quad R^{\psi}_{\mathcal{T}^m}(f) = \frac{1}{m} \sum_{i=1}^{m} \psi(y^i, f(x^i))$$

which is equivalent to

$$(\boldsymbol{w}^*, b^*) = \operatorname*{argmin}_{\|\boldsymbol{w}\| \leq r, b \in \mathbb{R}} \left(\frac{1}{m} \sum_{i=1}^{m} \max\{0, 1 - y^i(\langle \boldsymbol{w}, \boldsymbol{\phi}(x^i) \rangle + b)\} \right)$$

SVM as a convex Quadratic Program

- The minimization of the hinge-loss with a constraint on the parameter norm leads to:

$$(\boldsymbol{w}_1^*, b_1^*) = \underset{\|\boldsymbol{w}\| \leq r, b \in \mathbb{R}}{\operatorname{argmin}} \left(\frac{1}{m} \sum_{i=1}^{m} \max\{0, 1 - y^i(\langle \boldsymbol{w}, \boldsymbol{\phi}(x^i) \rangle + b)\} \right) \tag{2}$$

  where $r > 0$ is a hyperparameter.

- Problem (2) is equivalen to a convex *quadratic program*, a.k.a the linear Support Vector Machines algorithm:

$$(\boldsymbol{w}_2^*, b_2^*, \boldsymbol{\xi}^*) = \underset{\substack{(\boldsymbol{w}, b) \in \mathbb{R}^{n+1} \\ \boldsymbol{\xi} \in \mathbb{R}^m}}{\operatorname{argmin}} \left(\frac{1}{2} \|\boldsymbol{w}\|^2 + \frac{C}{m} \sum_{i=1}^{m} \xi_i \right)$$

  subject to

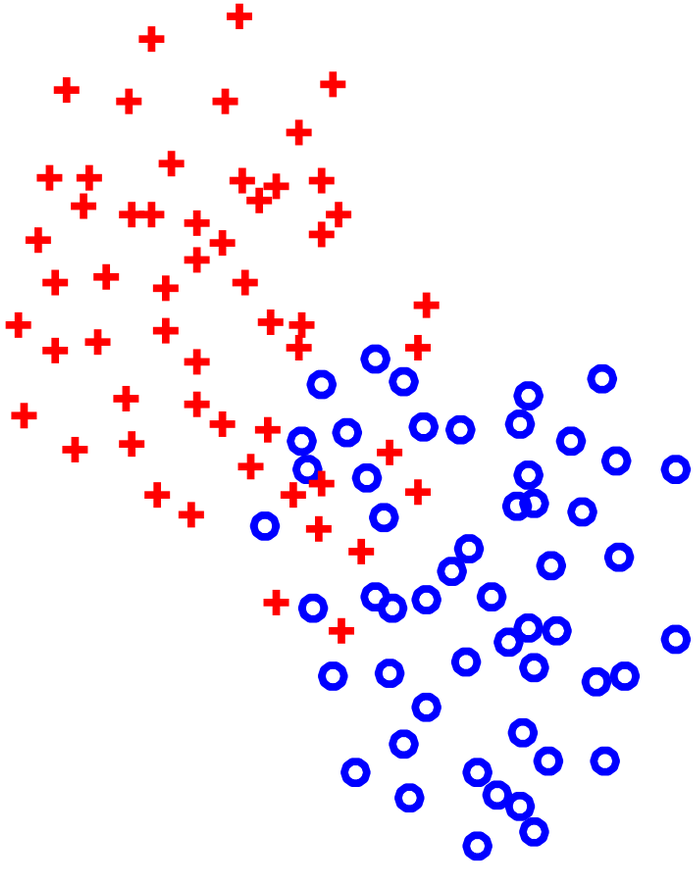
$$\begin{aligned} y^i(\langle \boldsymbol{w}, \boldsymbol{\phi}(x^i) \rangle + b) &\geq 1 - \xi_i\,, \quad i \in \{1, \ldots, m\} \\ \xi_i &\geq 0\,, \quad i \in \{1, \ldots, m\} \end{aligned}$$

- For any $C > 0$, there exists $r > 0$ such that $\boldsymbol{w}_1^* = \boldsymbol{w}_2^*$ and $b_1^* = b_2^*$.

- The optimal value of the hyperparameter C is tuned on validation data.

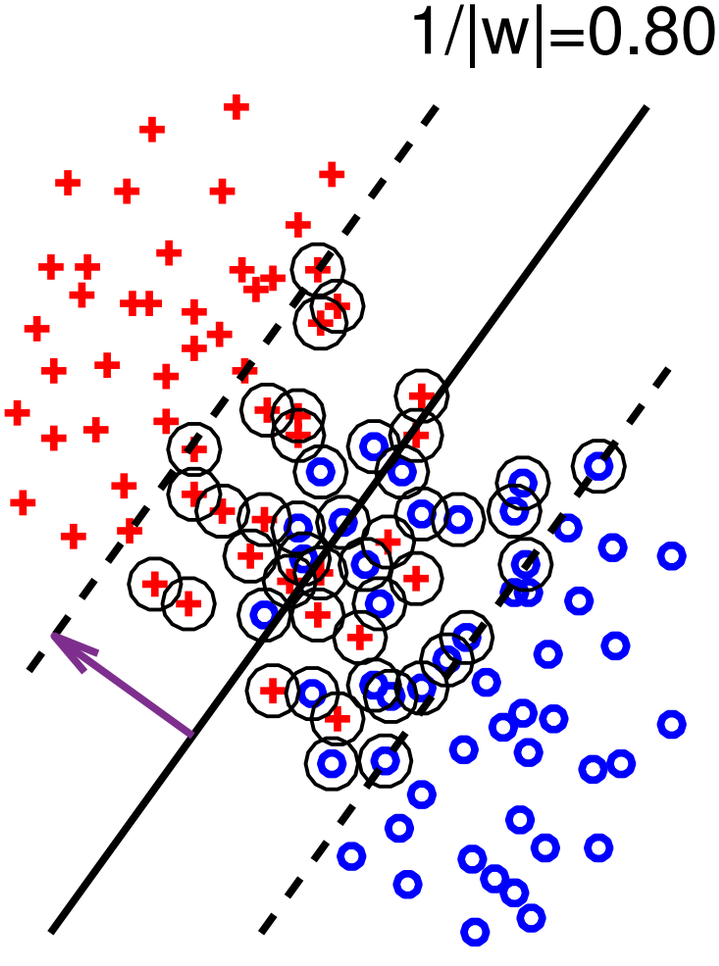
How the regularization constant C controls $\|w^*\|$ and $R^{\psi}(w^*, b^*)$

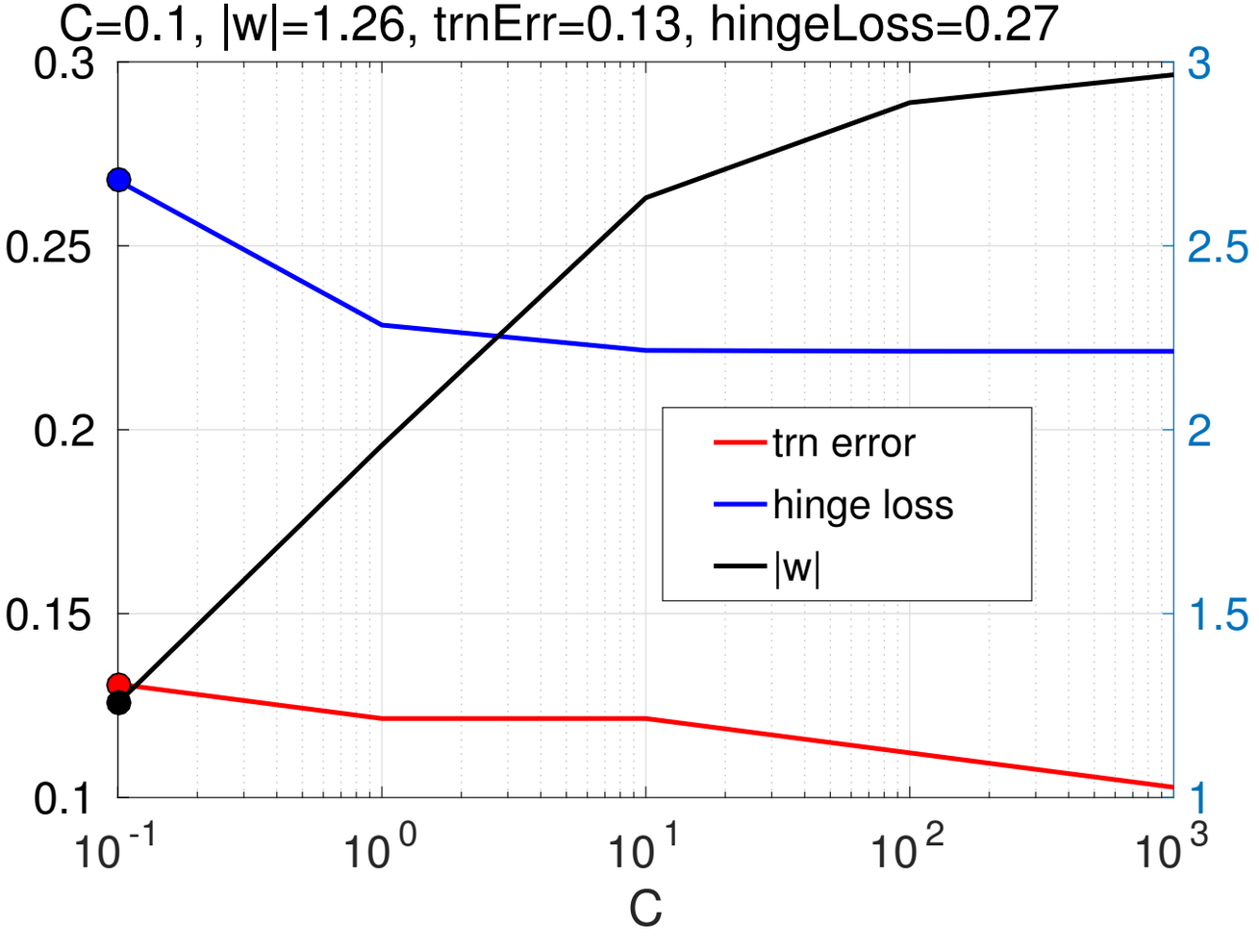
$$\left(\boldsymbol{w}^*, b^*, \boldsymbol{\xi}^*\right) = \underset{\substack{(\boldsymbol{w},b)\in\mathbb{R}^{n+1}\\ \boldsymbol{\xi}\in\mathbb{R}^m}}{\operatorname{argmin}} \left(\tfrac{1}{2}\|\boldsymbol{w}\|^2 + \tfrac{C}{m}\sum_{i=1}^{m}\xi_i\right) \quad \text{s.t.} \quad \begin{array}{rcl} y^i(\langle \boldsymbol{w}, \boldsymbol{\phi}(x^i)\rangle + b) & \geq & 1-\xi_i \\ \xi_i & \geq & 0 \end{array}$$



How the regularization constant C controls $\|w^*\|$ and $R^{\psi}(w^*, b^*)$

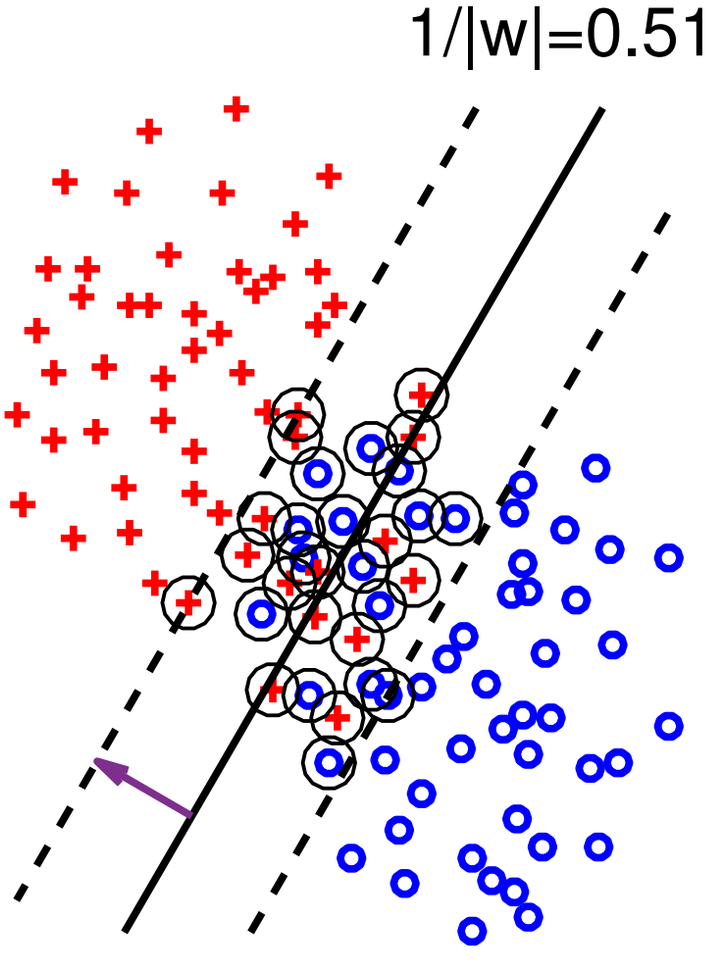
$$(\boldsymbol{w}^*, b^*, \boldsymbol{\xi}^*) = \underset{\substack{(\boldsymbol{w},b)\in\mathbb{R}^{n+1}\\ \boldsymbol{\xi}\in\mathbb{R}^m}}{\operatorname{argmin}} \left(\tfrac{1}{2}\|\boldsymbol{w}\|^2 + \tfrac{C}{m}\sum_{i=1}^{m}\xi_i \right) \quad \text{s.t.} \quad \begin{aligned} y^i(\langle \boldsymbol{w}, \boldsymbol{\phi}(x^i)\rangle + b) &\geq 1-\xi_i \\ \xi_i &\geq 0 \end{aligned}$$

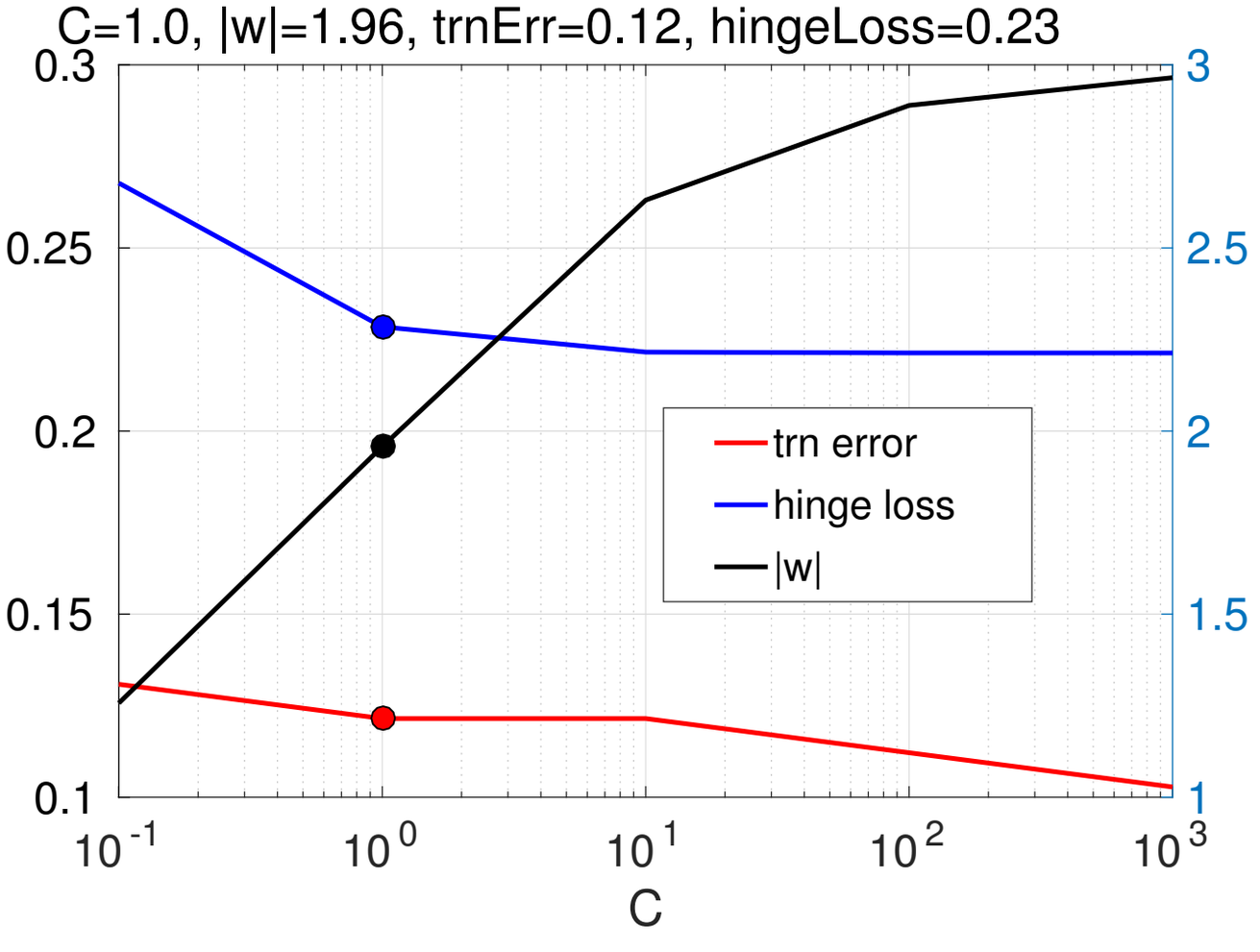




How the regularization constant C controls $\|w^*\|$ and $R^{\psi}(w^*, b^*)$

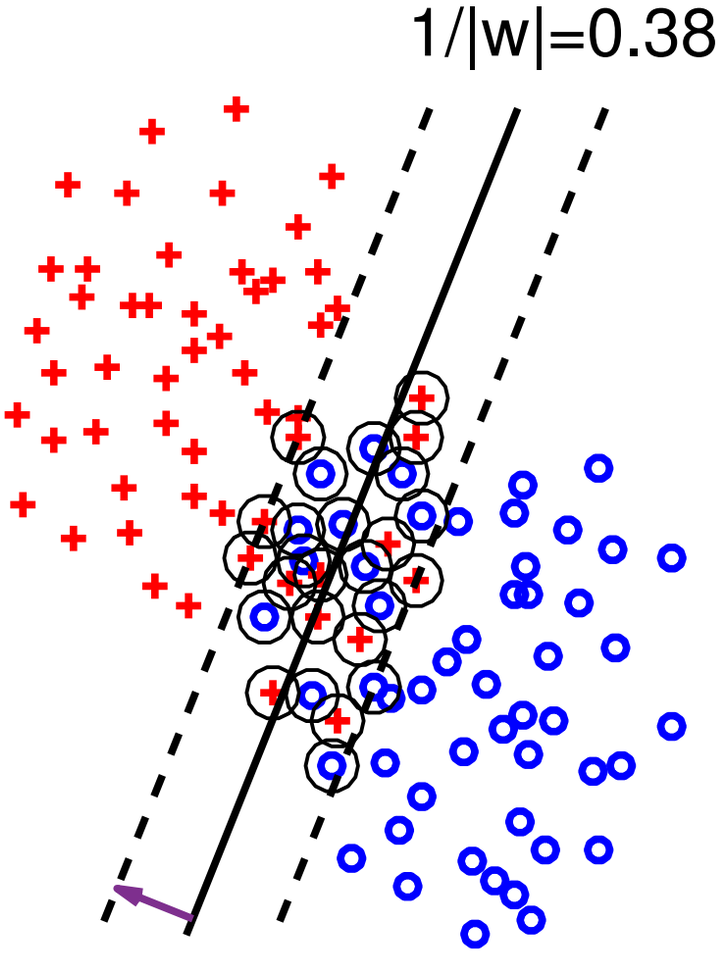
$$(\boldsymbol{w}^*, b^*, \boldsymbol{\xi}^*) = \underset{\substack{(\boldsymbol{w},b)\in\mathbb{R}^{n+1}\\ \boldsymbol{\xi}\in\mathbb{R}^m}}{\operatorname{argmin}} \left(\tfrac{1}{2}\|\boldsymbol{w}\|^2 + \tfrac{C}{m}\sum_{i=1}^{m}\xi_i \right) \quad \text{s.t.} \quad \begin{aligned} y^i(\langle \boldsymbol{w}, \boldsymbol{\phi}(x^i)\rangle + b) &\geq 1-\xi_i \\ \xi_i &\geq 0 \end{aligned}$$

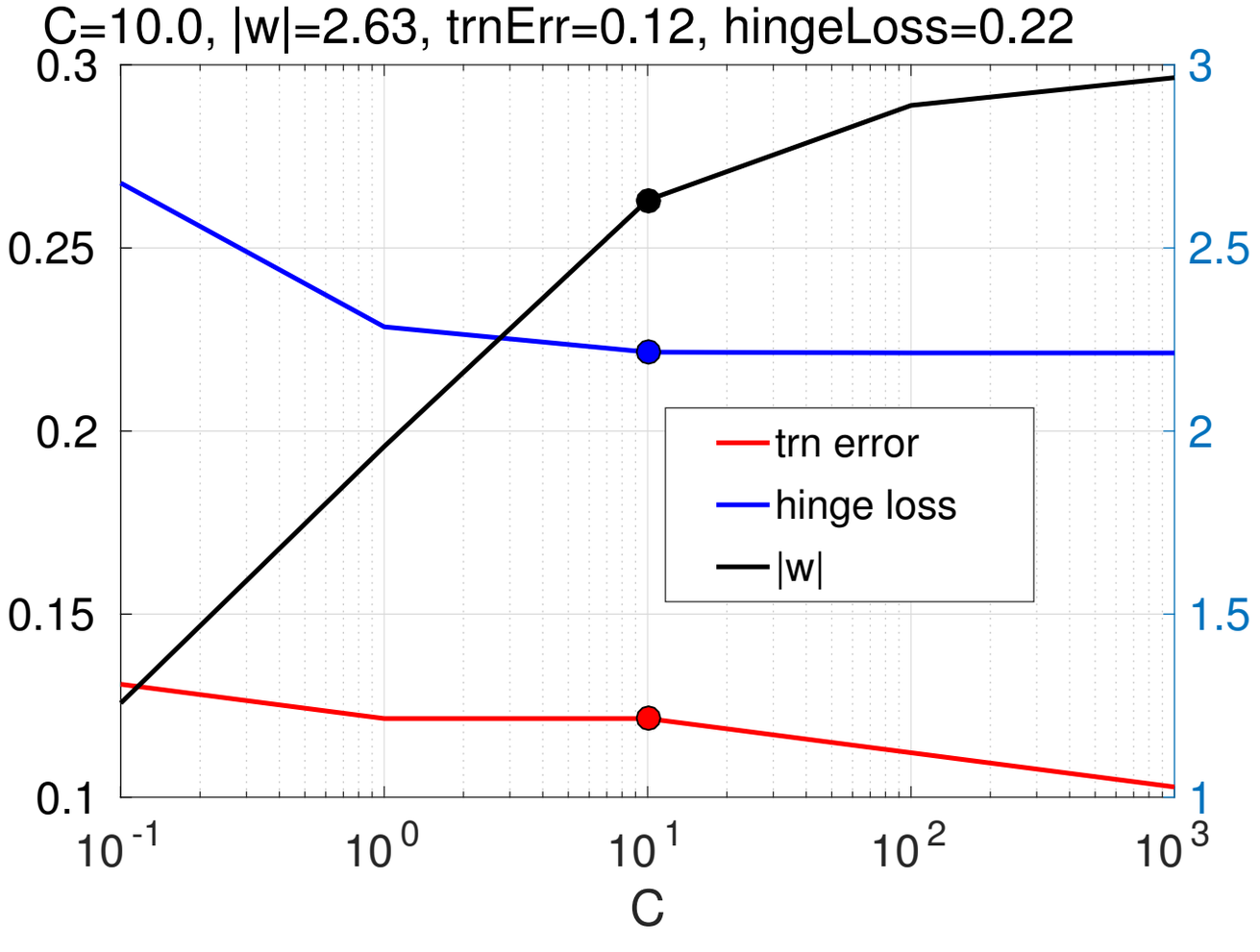




How the regularization constant C controls $\|w^*\|$ and $R^{\psi}(w^*, b^*)$

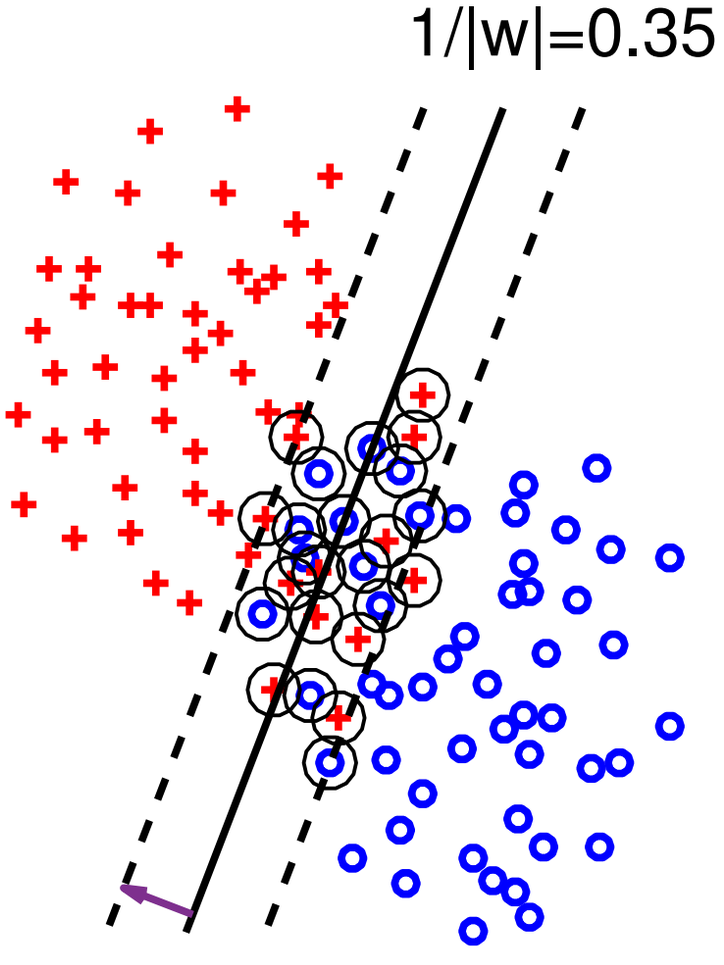
$$(\boldsymbol{w}^*, b^*, \boldsymbol{\xi}^*) = \underset{\substack{(\boldsymbol{w},b)\in\mathbb{R}^{n+1}\\ \boldsymbol{\xi}\in\mathbb{R}^m}}{\operatorname{argmin}} \left(\tfrac{1}{2}\|\boldsymbol{w}\|^2 + \tfrac{C}{m}\sum_{i=1}^{m}\xi_i \right) \quad \text{s.t.} \quad \begin{aligned} y^i(\langle \boldsymbol{w}, \boldsymbol{\phi}(x^i)\rangle + b) &\geq 1-\xi_i \\ \xi_i &\geq 0 \end{aligned}$$

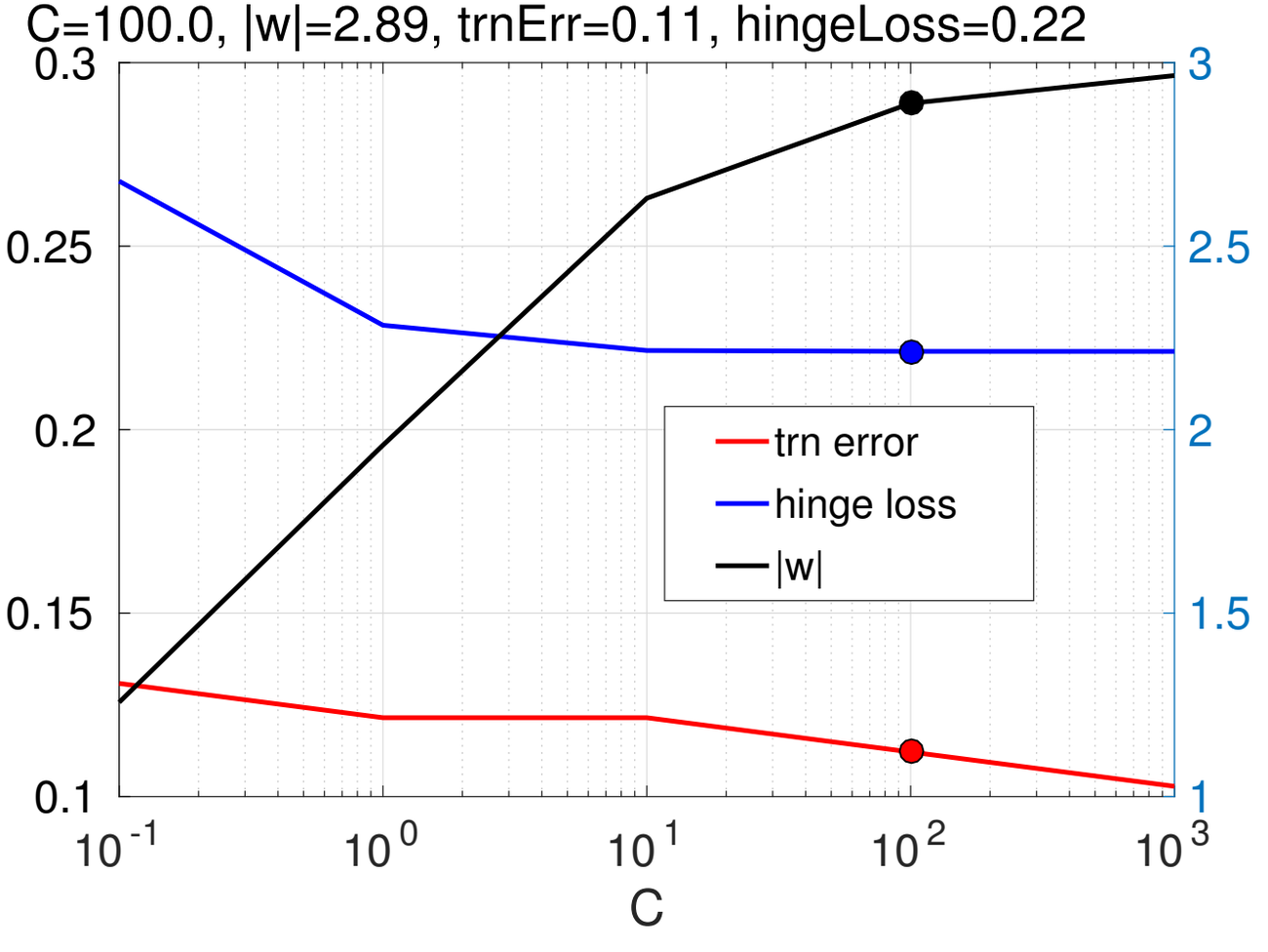




How the regularization constant C controls $\|w^*\|$ and $R^{\psi}(w^*, b^*)$

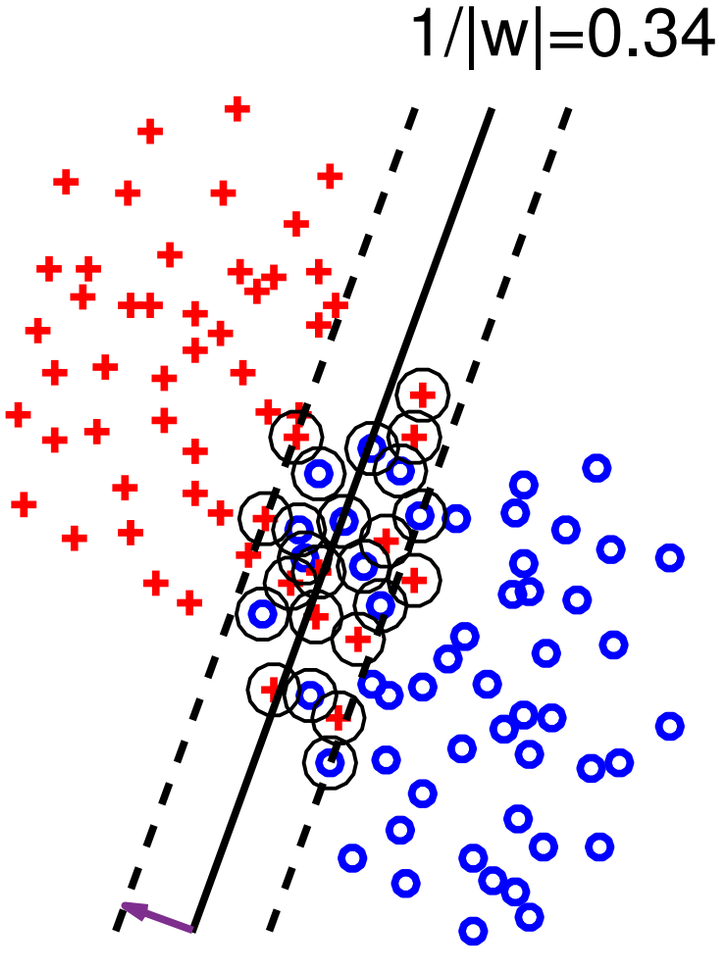
$$(\boldsymbol{w}^*, b^*, \boldsymbol{\xi}^*) = \underset{\substack{(\boldsymbol{w},b)\in\mathbb{R}^{n+1}\\ \boldsymbol{\xi}\in\mathbb{R}^m}}{\operatorname{argmin}} \left(\tfrac{1}{2}\|\boldsymbol{w}\|^2 + \tfrac{C}{m}\sum_{i=1}^{m}\xi_i \right) \quad \text{s.t.} \quad \begin{aligned} y^i(\langle \boldsymbol{w}, \boldsymbol{\phi}(x^i)\rangle + b) &\geq 1-\xi_i \\ \xi_i &\geq 0 \end{aligned}$$

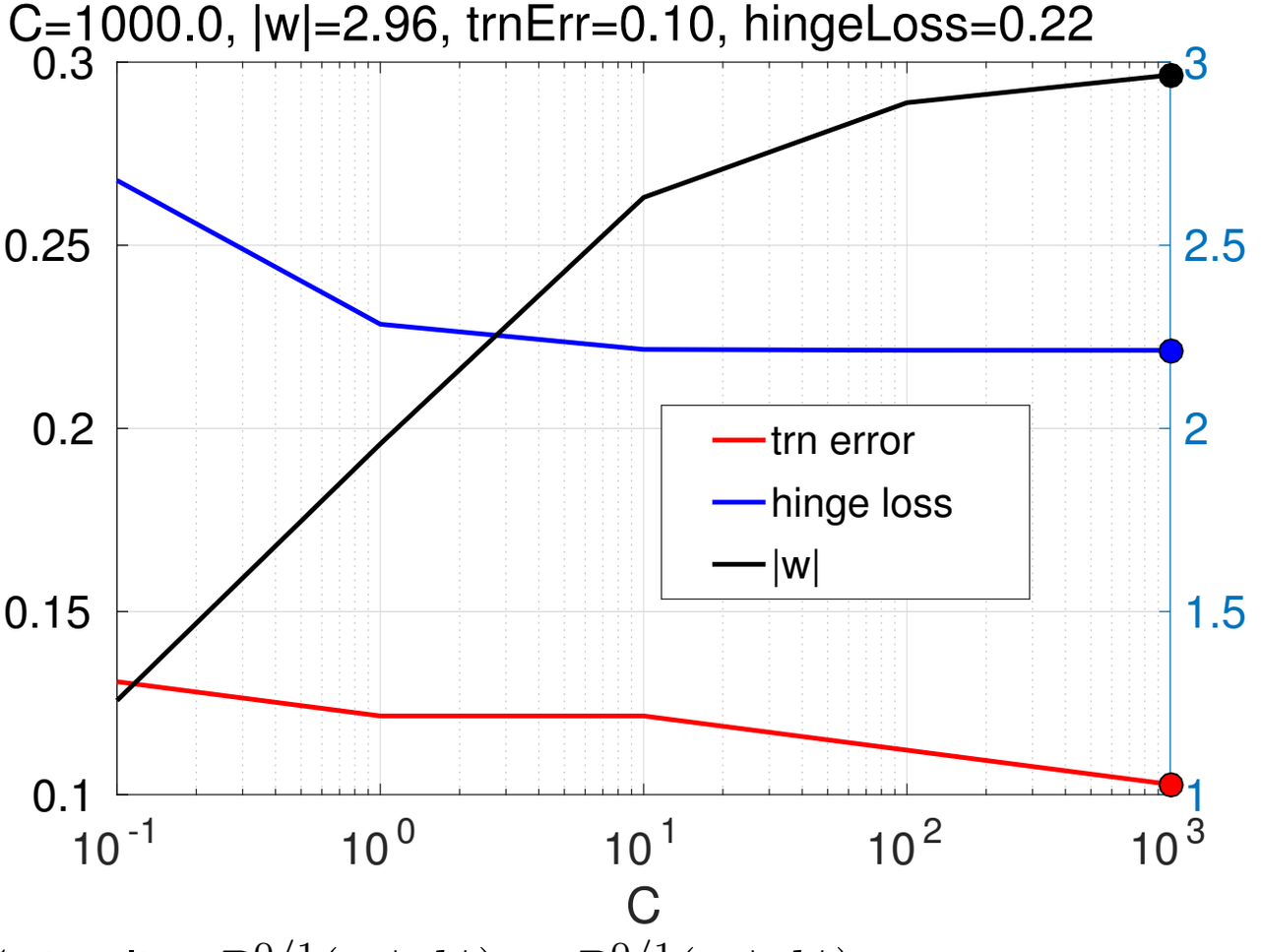




How the regularization constant C controls $\|w^*\|$ and $R^{\psi}(w^*, b^*)$

$$(\boldsymbol{w}^*, b^*, \boldsymbol{\xi}^*) = \underset{\substack{(\boldsymbol{w},b)\in\mathbb{R}^{n+1}\\ \boldsymbol{\xi}\in\mathbb{R}^m}}{\operatorname{argmin}} \left(\tfrac{1}{2}\|\boldsymbol{w}\|^2 + \tfrac{C}{m}\sum_{i=1}^{m}\xi_i \right) \quad \text{s.t.} \quad \begin{array}{rcl} y^i(\langle \boldsymbol{w}, \boldsymbol{\phi}(x^i)\rangle + b) & \geq & 1-\xi_i \\ \xi_i & \geq & 0 \end{array}$$





Remark: No guarantee that $C_1 < C_2$ implies $R^{0/1}(\boldsymbol{w}_1^*, b_1^*) \geq R^{0/1}(\boldsymbol{w}_2^*, b_2^*)$.

From Primal SVM to Dual SVM problem

- Lagrangian of the primal SVM problem:

$$L(\boldsymbol{w}, b, \boldsymbol{\xi}, \boldsymbol{\alpha}, \boldsymbol{\mu}) = \underbrace{\frac{1}{2}\|\boldsymbol{w}\|^2 + \frac{C}{m}\sum_{i=1}^{m}\xi_i}_{\text{original objective}} \underbrace{- \sum_{i=1}^{m}\alpha_i(y^i(\langle \boldsymbol{w}, \boldsymbol{\phi}(x^i)\rangle + b) - 1 + \xi_i) - \sum_{i=1}^{m}\mu_i\xi_i}_{\text{constraint violation penalty}}$$

- Strong duality:

$$\underbrace{\min_{\substack{(\boldsymbol{w},b)\in\mathbb{R}^{n+1}\,,\,\xi_i\geq 0\,,\,i\in\{1,\ldots,m\} \\ y^i(\langle \boldsymbol{w},\boldsymbol{\phi}(x^i)\rangle + b)\geq 1-\xi_i\,,i\in\{1,\ldots,m\}}} \left(\frac{1}{2}\|\boldsymbol{w}\|^2 + \frac{C}{m}\sum_{i=1}^{m}\xi_i\right)}_{\text{primal problem}} =$$

$$= \min_{\substack{\boldsymbol{w}\in\mathbb{R}^n \\ b\in\mathbb{R} \\ \boldsymbol{\xi}\in\mathbb{R}^m}} \max_{\substack{\boldsymbol{\alpha}\in\mathbb{R}^m_+ \\ \boldsymbol{\mu}\in\mathbb{R}^m_+}} L(\boldsymbol{w}, b, \boldsymbol{\xi}, \boldsymbol{\alpha}, \boldsymbol{\mu}) = \max_{\substack{\boldsymbol{\alpha}\in\mathbb{R}^m_+ \\ \boldsymbol{\mu}\in\mathbb{R}^m_+}} \min_{\substack{\boldsymbol{w}\in\mathbb{R}^n \\ b\in\mathbb{R} \\ \boldsymbol{\xi}\in\mathbb{R}^m}} L(\boldsymbol{w}, b, \boldsymbol{\xi}, \boldsymbol{\alpha}, \boldsymbol{\mu}) =$$

$$= \underbrace{\max_{\substack{\boldsymbol{\alpha}\in[0,C/m]^m \\ \sum_{i=1}^{m}\alpha_i\, y^i = 0}} \left(\sum_{i=1}^{m}\alpha_i - \frac{1}{2}\sum_{i=1}^{m}\sum_{j=1}^{m}\alpha_i\alpha_j y^i y^j \langle \boldsymbol{\phi}(x^i), \boldsymbol{\phi}(x^j)\rangle\right)}_{\text{dual problem}}$$

Dual SVM problem

- The dual SVM formulation is a convex quadratic program

$$\boldsymbol{\alpha}^* = \underset{\boldsymbol{\alpha}\in\mathbb{R}^m}{\operatorname{argmax}} \left(\sum_{i=1}^{m} \alpha_i - \tfrac{1}{2} \sum_{i=1}^{m} \sum_{j=1}^{m} \alpha_i \alpha_j y^i y^j \langle \boldsymbol{\phi}(x^i), \boldsymbol{\phi}(x^j) \rangle \right)$$

$$\text{s.t.} \qquad \sum_{i=1}^{m} \alpha_i \, y^i = 0 \,, \quad 0 \leq \alpha_i \leq \frac{C}{m} \,, \quad i \in \{1, \ldots, m\}$$

Dual SVM problem

- The dual SVM formulation is a convex quadratic program

$$\boldsymbol{\alpha}^* = \operatorname*{argmax}_{\boldsymbol{\alpha}\in\mathbb{R}^m} \left(\sum_{i=1}^m \alpha_i - \tfrac{1}{2} \sum_{i=1}^m \sum_{j=1}^m \alpha_i \alpha_j y^i y^j \langle \boldsymbol{\phi}(x^i), \boldsymbol{\phi}(x^j) \rangle \right)$$
$$\text{s.t.} \qquad \sum_{i=1}^m \alpha_i \, y^i = 0 \,, \quad 0 \le \alpha_i \le \frac{C}{m} \,, \quad i \in \{1, \ldots, m\}$$

- The primal optimal variables $(\boldsymbol{w}^*, b^*)$ can be computed from the dual variables $\boldsymbol{\alpha}^*$ by

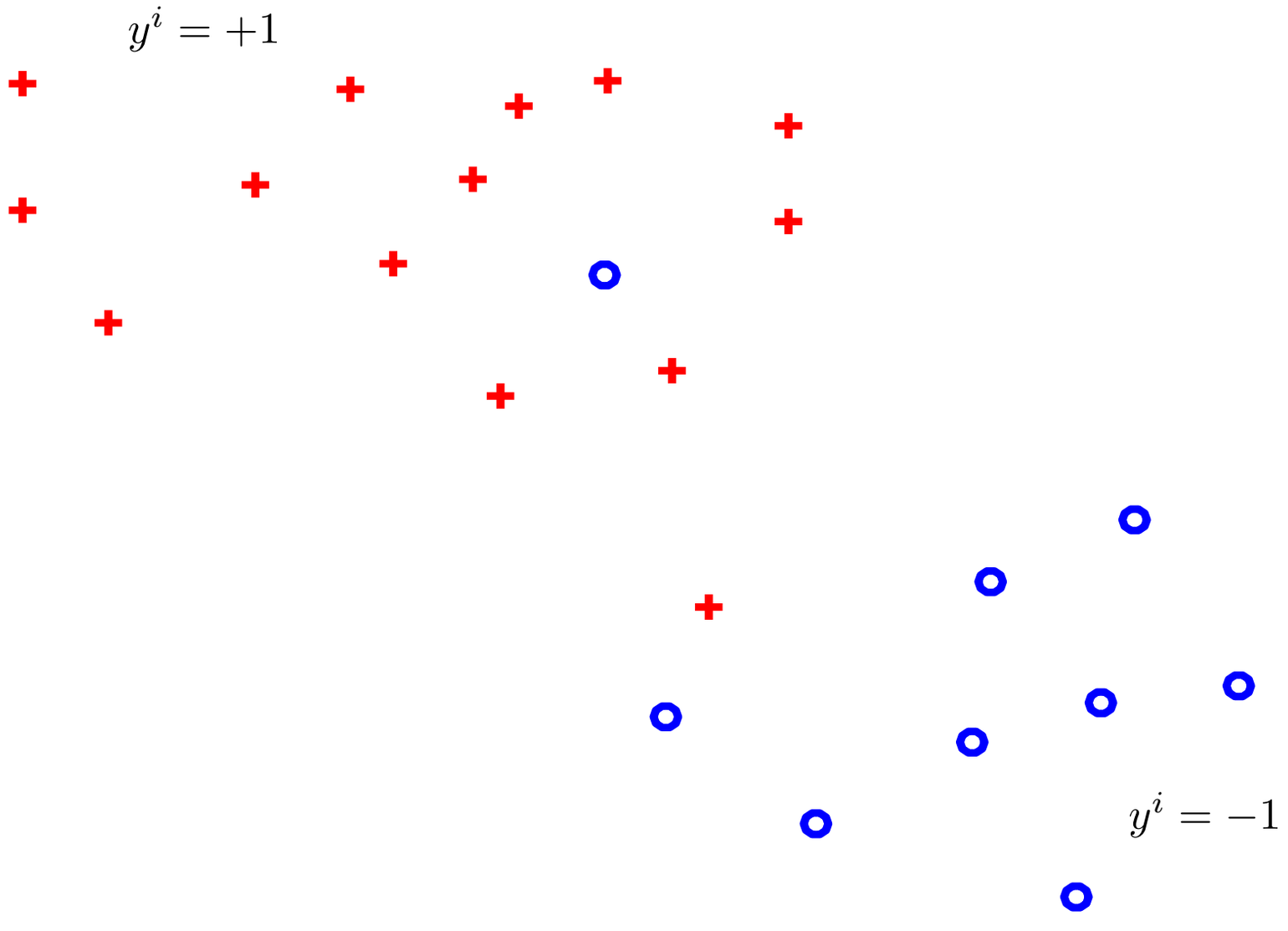
$$\boldsymbol{w}^* = \sum_{i=1}^m y^i \, \boldsymbol{\phi}(x^i) \, \alpha_i^* = \sum_{i \in \mathcal{I}_{\text{SV}}} y^i \, \boldsymbol{\phi}(x^i) \, \alpha_i^*$$

$$b^* = y^i - \langle \boldsymbol{w}^*, \boldsymbol{\phi}(x^i) \rangle \,, \; \forall i \in \mathcal{I}_{\text{sv}}^{\text{b}} = \{ j \mid 0 < \alpha_j^* < \frac{C}{m} \}$$

- $\boldsymbol{\alpha}^*$ is sparse; $\boldsymbol{w}^*$ is lin. combination of Support Vectors $\mathcal{I}_{\text{sv}} = \{ j \mid \alpha_j^* > 0 \}$

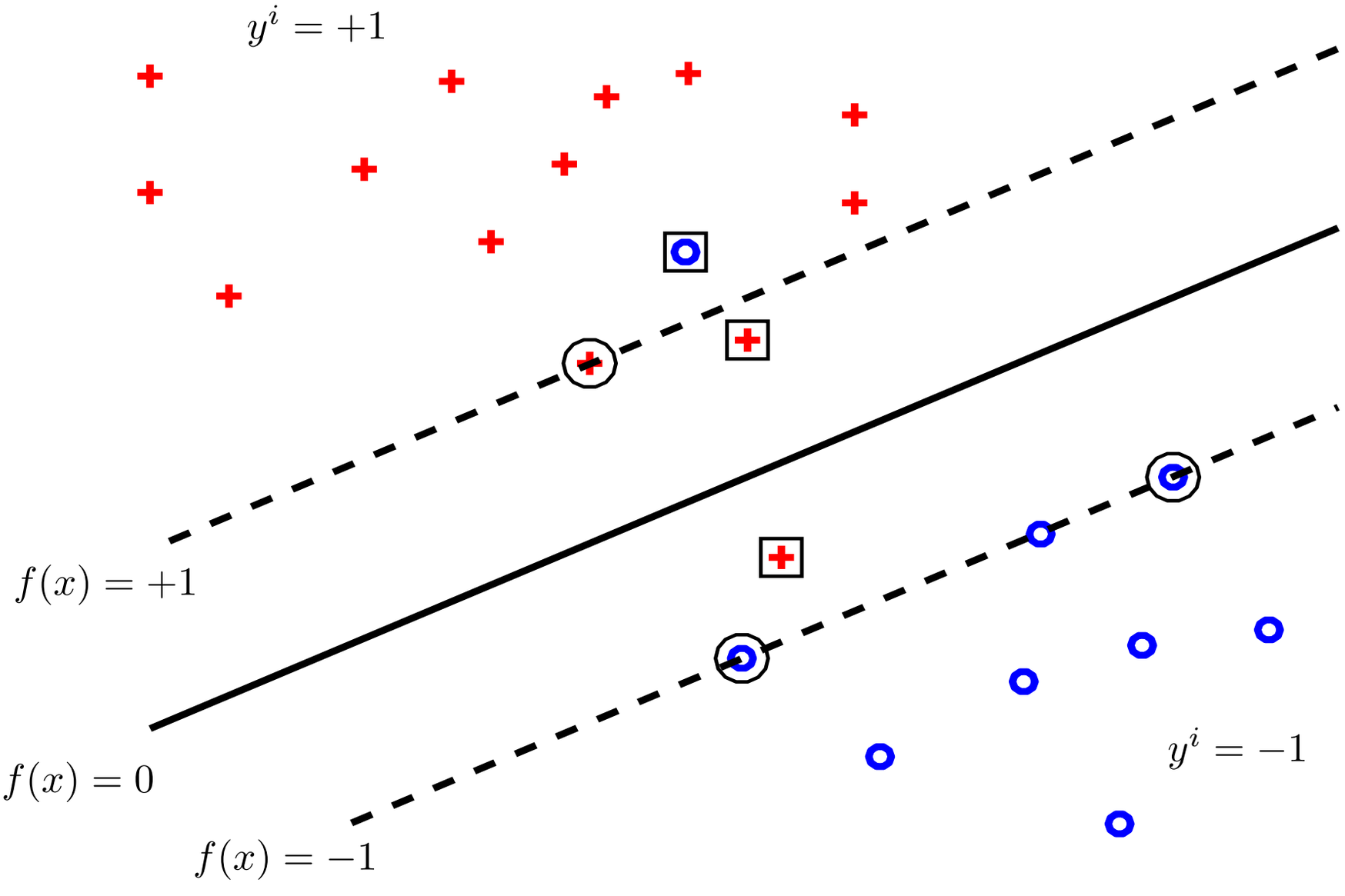
Dual SVM classifier

$$f(x) = \langle \boldsymbol{w}, \boldsymbol{\phi}(x) \rangle + b = \langle \underbrace{\sum_{i=1}^{m} y^i \, \alpha_i \, \boldsymbol{\phi}(x^i)}_{\boldsymbol{w}}, \boldsymbol{\phi}(x) \rangle + b$$



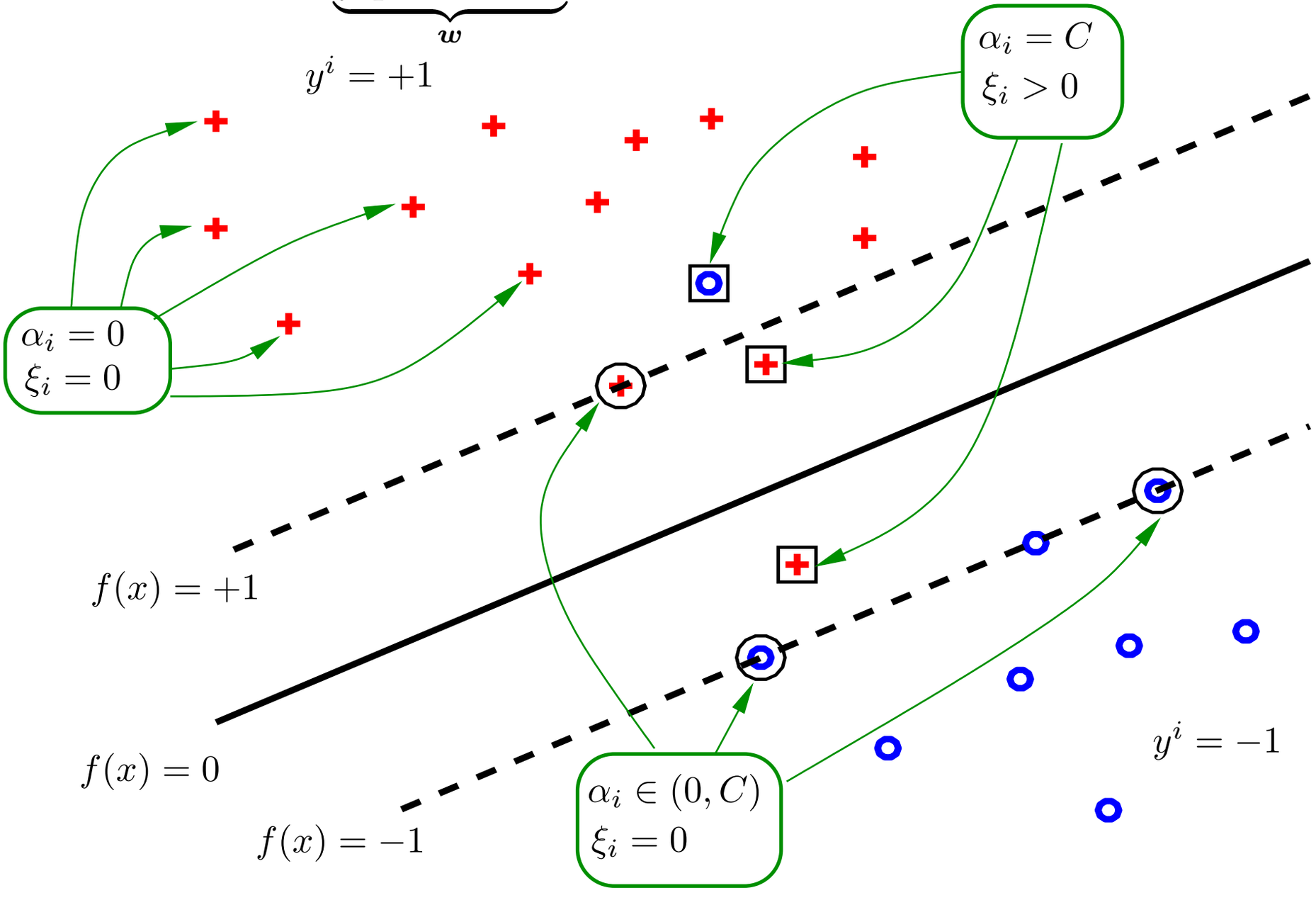
Dual SVM classifier

$$f(x) = \langle \boldsymbol{w}, \boldsymbol{\phi}(x) \rangle + b = \langle \underbrace{\sum_{i=1}^{m} y^i \, \alpha_i \, \boldsymbol{\phi}(x^i)}_{\boldsymbol{w}}, \boldsymbol{\phi}(x) \rangle + b$$



Dual SVM classifier

$$f(x) = \langle \boldsymbol{w}, \boldsymbol{\phi}(x) \rangle + b = \langle \underbrace{\sum_{i=1}^{m} y^i \, \alpha_i \, \boldsymbol{\phi}(x^i)}_{\boldsymbol{w}}, \boldsymbol{\phi}(x) \rangle + b$$



Kernel SVM

- The SVM algorithm requires observations in terms of **dot products** only:

  Learning: Given $\mathcal{T}^m = \{(x^i, y^i) \in \mathcal{X} \times \{-1, +1\} \mid i = 1, \ldots, m\}$, solve

$$\boldsymbol{\alpha}^* = \underset{\boldsymbol{\alpha} \in \mathbb{R}^m}{\operatorname{argmax}} \left(\sum_{i=1}^{m} \alpha_i - \tfrac{1}{2} \sum_{i=1}^{m} \sum_{j=1}^{m} \alpha_i \, \alpha_j \, y^i \, y^j \, \langle \boldsymbol{\phi}(x^i), \boldsymbol{\phi}(x^j) \rangle \right)$$
$$\text{s.t.} \qquad \sum_{i=1}^{m} \alpha_i \, y^i = 0 \, , \quad 0 \leq \alpha_i \leq \frac{C}{m} \, , i \in \{1, \ldots, m\}$$

  Prediction:

$$h(x; \boldsymbol{\alpha}, b) = \operatorname{sign}(\langle \boldsymbol{w}, \boldsymbol{\phi}(x) \rangle + b) = \operatorname{sign} \left(\sum_{i \in \mathcal{I}_{\text{sv}}} y^i \alpha_i \, \langle \boldsymbol{\phi}(x^i), \boldsymbol{\phi}(x) \rangle + b \right)$$

- Given a feature map $\boldsymbol{\phi} \colon \mathcal{X} \to \mathbb{R}^n$, define kernel function $k \colon \mathcal{X} \times \mathcal{X} \to \mathbb{R}$

$$k(x, x') = \langle \boldsymbol{\phi}(x), \boldsymbol{\phi}(x') \rangle$$

Kernel SVM

- The SVM algorithm requires observations in terms of **dot products** only:

  Learning: Given $\mathcal{T}^m = \{(x^i, y^i) \in \mathcal{X} \times \{-1, +1\} \mid i = 1, \ldots, m\}$, solve

$$\boldsymbol{\alpha}^* = \underset{\boldsymbol{\alpha} \in \mathbb{R}^m}{\operatorname{argmax}} \left(\sum_{i=1}^{m} \alpha_i - \tfrac{1}{2} \sum_{i=1}^{m} \sum_{j=1}^{m} \alpha_i \, \alpha_j \, y^i \, y^j \;\; k(x^i, x^j) \right)$$
$$\text{s.t.} \qquad \sum_{i=1}^{m} \alpha_i \, y^i = 0 \,, \quad 0 \leq \alpha_i \leq C \,, i \in \{1, \ldots, m\}$$

  Prediction:

$$h(x; \boldsymbol{\alpha}, b) = \operatorname{sign}(\langle \boldsymbol{w}, \boldsymbol{\phi}(x) \rangle + b) = \operatorname{sign} \left(\sum_{i \in \mathcal{I}_{\text{sv}}} y^i \alpha_i \;\; k(x^i, x) + b \right)$$

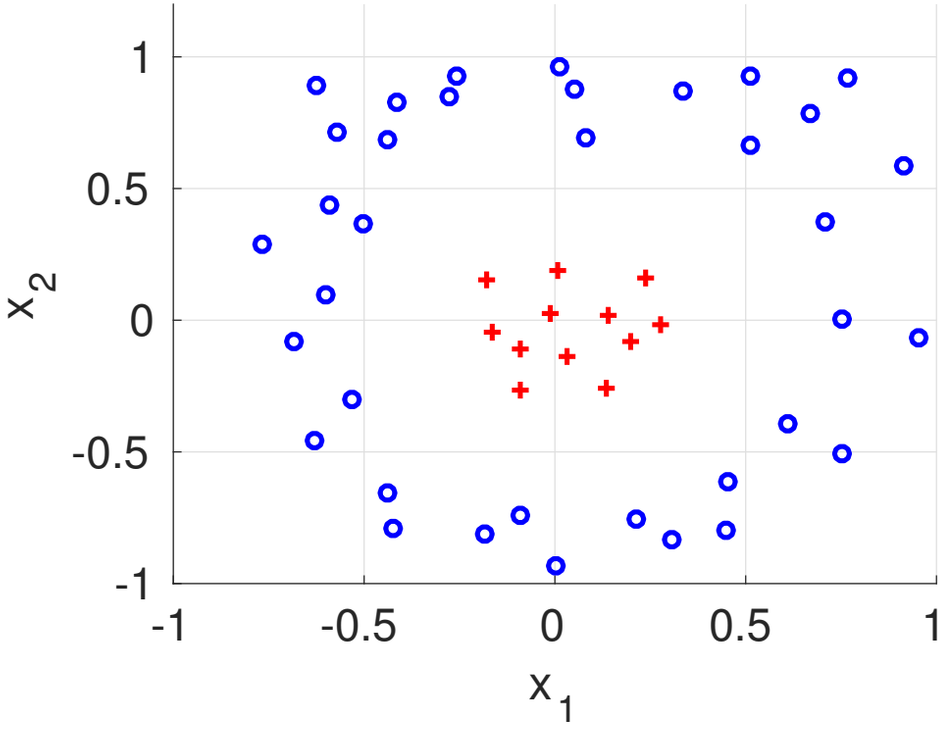
- Given a feature map $\boldsymbol{\phi} \colon \mathcal{X} \to \mathbb{R}^n$, define kernel function $k \colon \mathcal{X} \times \mathcal{X} \to \mathbb{R}$

$$k(x, x') = \langle \boldsymbol{\phi}(x), \boldsymbol{\phi}(x') \rangle$$

Making linear classifier quadratic via mapping data to higher dimensional space

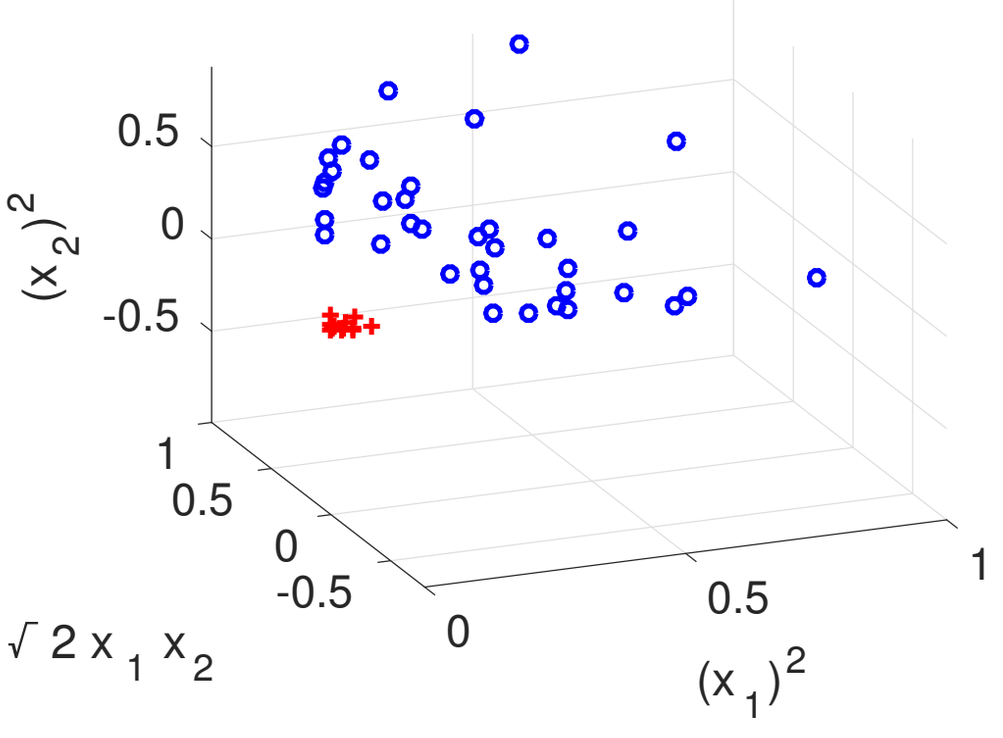
observations

$$\boldsymbol{x} = (x_1, x_2) \in \mathbb{R}^2$$



features

$$\phi(\boldsymbol{x}) = (x_1^2, \sqrt{2}x_1x_2, x_2^2) \in \mathbb{R}^3$$



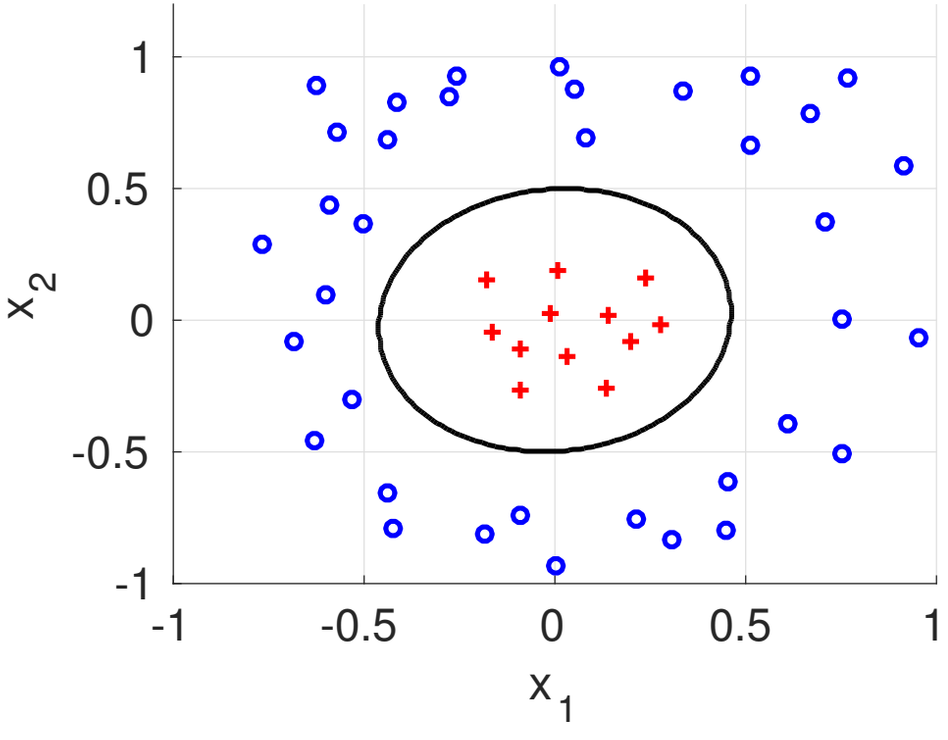
Making linear classifier quadratic via mapping data to higher dimensional space

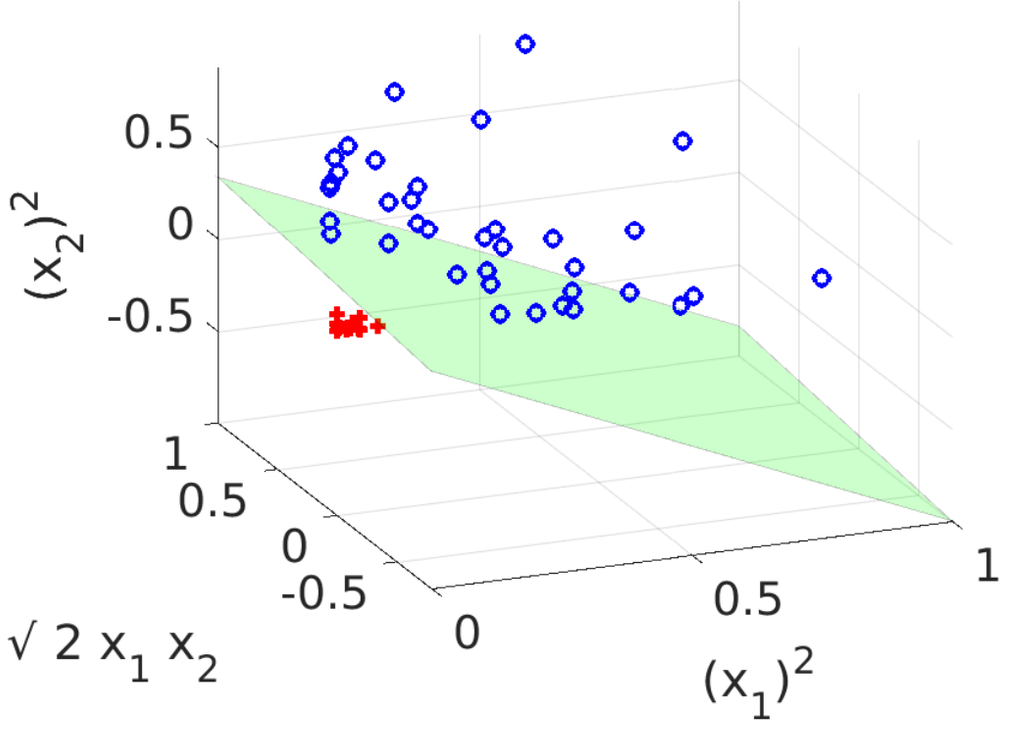
observations

$$\boldsymbol{x} = (x_1, x_2) \in \mathbb{R}^2$$

features

$$\phi(\boldsymbol{x}) = (x_1^2, \sqrt{2}x_1x_2, x_2^2) \in \mathbb{R}^3$$





The classifier $h(x; \boldsymbol{w}, b) = \text{sign}(f(\boldsymbol{x}; \boldsymbol{w}, b))$ uses the following score function

$$f(\boldsymbol{x}) = w_1\phi_1(\boldsymbol{x}) + w_2\phi_2(\boldsymbol{x}) + w_3\phi_3(\boldsymbol{x}) + b = w_1x_1^2 + w_2\sqrt{2}x_1\,x_2 + w_3\,x_2^2 + b$$

which is both linear in parameters $(\boldsymbol{w}, b)$ and non-linear in the inputs $\boldsymbol{x}$.

Homogeneous polynomial kernel of degree 2

- Consider quadratic function of d-dimensional inputs $\boldsymbol{x} \in \mathcal{X} = \mathbb{R}^d$

$$\begin{aligned} f(\boldsymbol{x}) &= x_1^2\, w_1 + \cdots + x_d^2\, w_d + \sqrt{2}\, x_1\, x_2\, w_{d+1} + \cdots + \sqrt{2}\, x_{d-1}\, x_d\, w_n + b \\ &= \langle \boldsymbol{w}, \boldsymbol{\phi}(\boldsymbol{x}) \rangle + b \end{aligned}$$

where $\boldsymbol{w} \in \mathbb{R}^n$, $n = \frac{(d+1)d}{2}$, and

$$\boldsymbol{\phi}(\boldsymbol{x}) = (x_1^2, \ldots, x_d^2, \sqrt{2}x_1x_2, \ldots, \sqrt{2}x_{d-1}x_d)$$

- In case of $d = 2$, we have

$$\begin{aligned} k(\boldsymbol{x}, \boldsymbol{x}') &= (x_1\, x_1' + x_2\, x_2')^2 = x_1^2\, x_1'^2 + 2\, x_1\, x_2\, x_1'\, x_2' + x_2^2\, x_2'^2 \\ &= \left\langle (x_1^2,\ \sqrt{2}\, x_1\, x_2\, ,\ x_2^2), (x_1'^2,\ \sqrt{2}\, x_1'\, x_2'\, ,\ x_2'^2) \right\rangle \end{aligned}$$

- For any $d \geq 2$ and $\boldsymbol{x}, \boldsymbol{x}' \in \mathbb{R}^d$, the dot product $\langle \boldsymbol{\phi}(\boldsymbol{x}), \boldsymbol{\phi}(\boldsymbol{x}') \rangle$ can be computed via the homogeneous polynomial kernel of degree 2:

$$k(\boldsymbol{x}, \boldsymbol{x}') = \langle \boldsymbol{\phi}(\boldsymbol{x}), \boldsymbol{\phi}(\boldsymbol{x}') \rangle = \langle \boldsymbol{x}, \boldsymbol{x}' \rangle^2$$

Radial basis function kernel

- Assume the observations are real-valued features: $\mathcal{X} = \mathbb{R}^d$
- The RBF kernel

$$k(\boldsymbol{x}, \boldsymbol{x}') = \exp(-\gamma \|\boldsymbol{x} - \boldsymbol{x}'\|^2)$$

corresponds to dot product $\langle \boldsymbol{\phi}(\boldsymbol{x}), \boldsymbol{\phi}(\boldsymbol{x}') \rangle$ in infinite dimensional space.

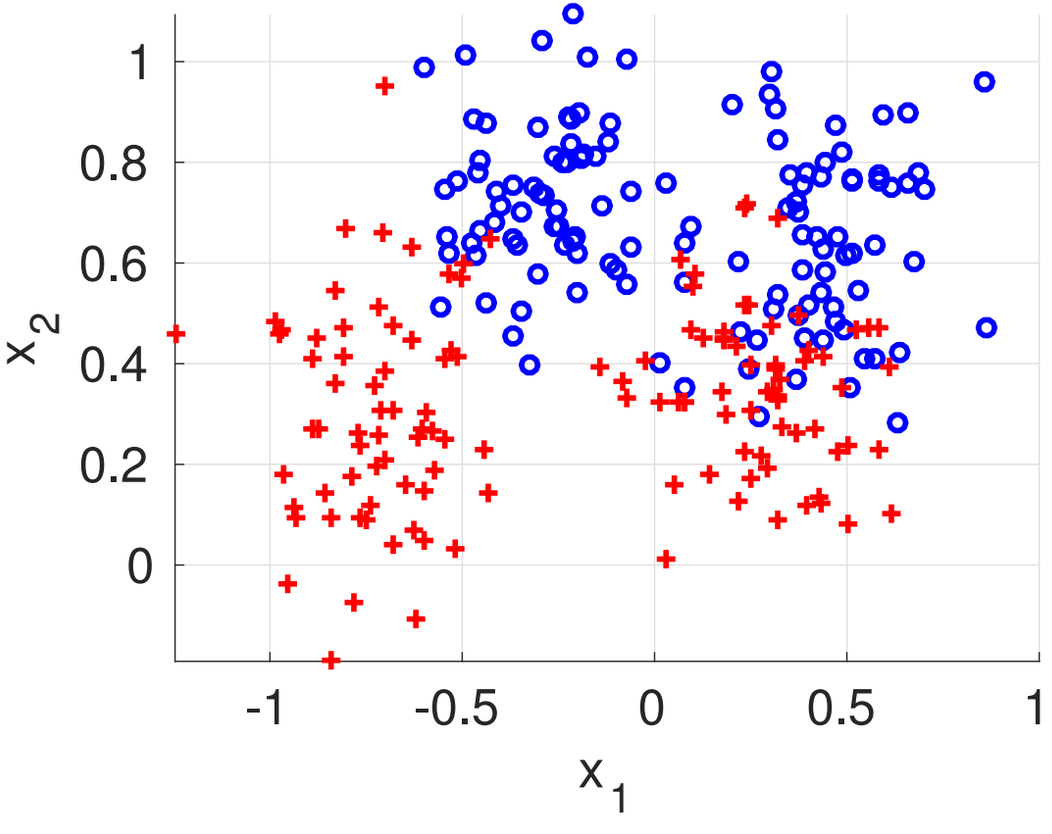
- In one dimensional case, $\mathcal{X} = \mathbb{R}$ the RBF kernel reads

$$\begin{aligned}
k(x, x') &= \exp(-\gamma (x - x')^2) \\
&= \exp(-\gamma\, x^2) \exp(-\gamma\, x'^2) \exp(2\, \gamma\, x\, x') \\
&= \exp(-\gamma\, x^2) \exp(-\gamma\, x'^2) \sum_{n=0}^{\infty} \frac{(2\, x\, x'\, \gamma)^n}{n!} \\
&= \exp(-\gamma\, x^2)\, \exp(-\gamma\, x'^2) \sum_{n=0}^{\infty} \frac{(\sqrt{2\, \gamma}\, x)^n}{\sqrt{n!}} \frac{(\sqrt{2\, \gamma}\, x')^n}{\sqrt{n!}}
\end{aligned}$$

where we used the Taylor expansion $\exp(a) = \sum_{n=0}^{\infty} \frac{a^n}{n!}$

Example: SVM with RBF kernel
training error vs. γ

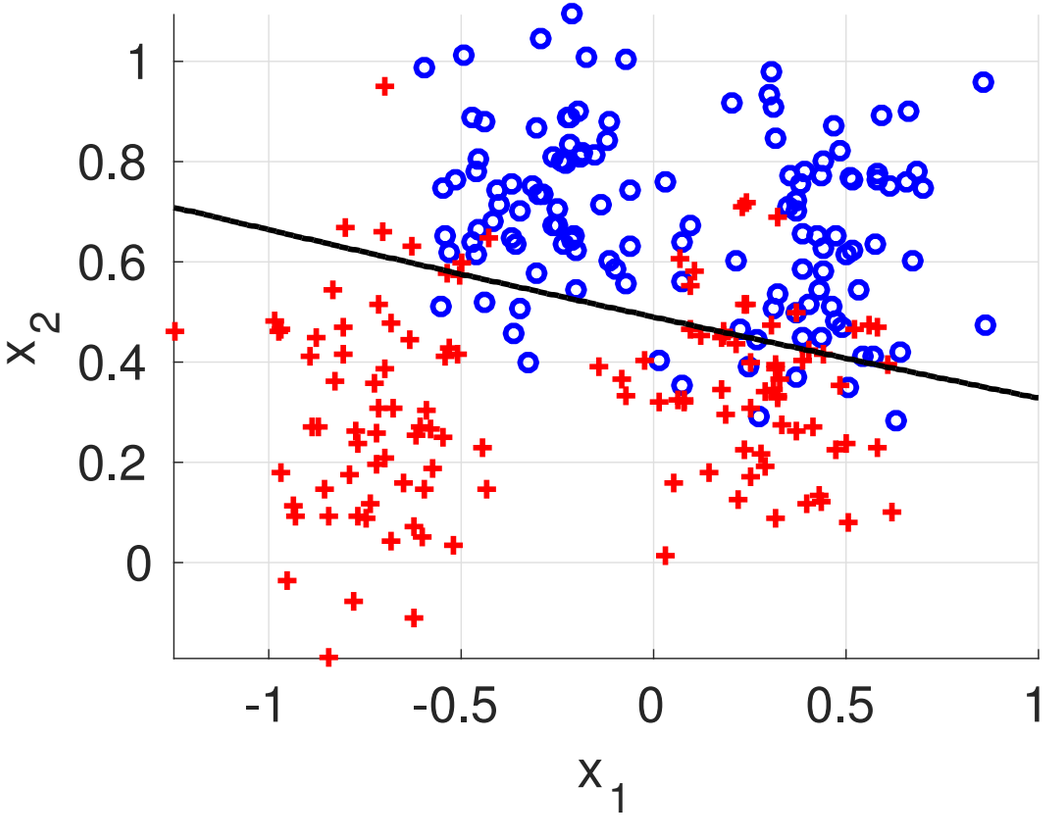
$$f(\boldsymbol{x}) = \sum_{i \in \mathcal{I}_{\mathrm{SV}}} y^i \, \alpha_i \, \exp(-\gamma \|\boldsymbol{x} - \boldsymbol{x}^i\|^2) + b$$

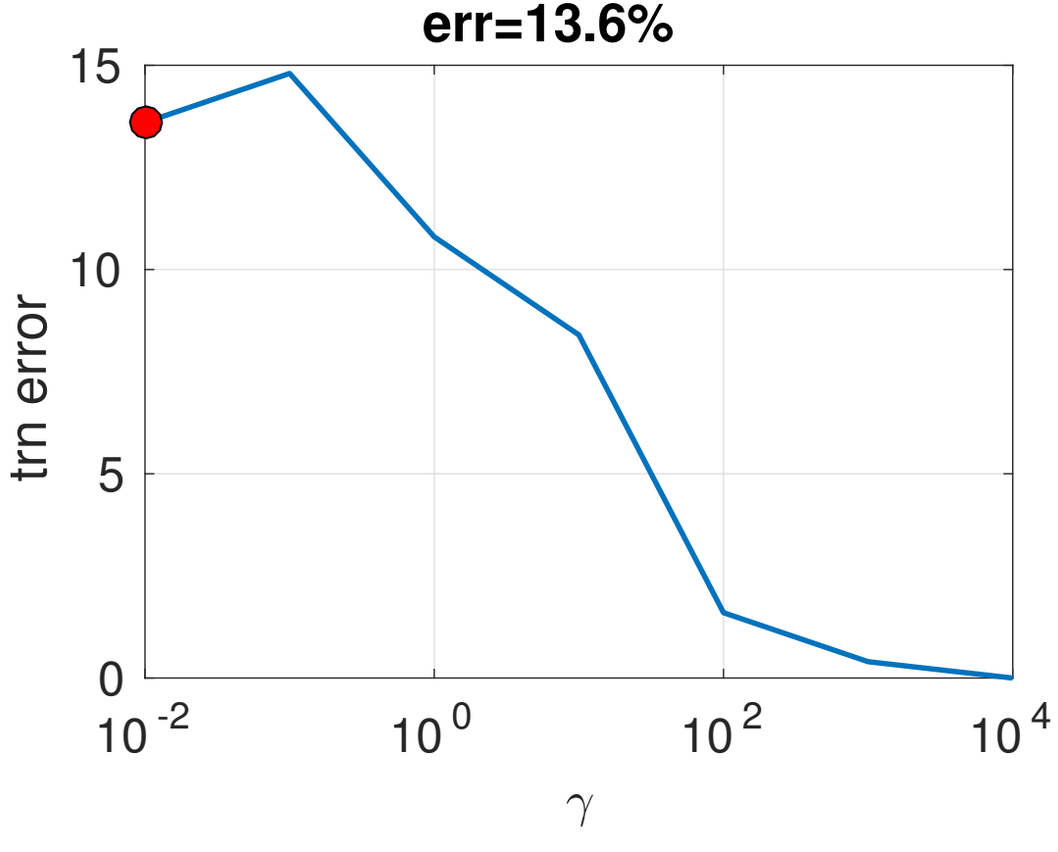


Example: SVM with RBF kernel
training error vs. γ

$$f(\boldsymbol{x}) = \sum_{i \in \mathcal{I}_{\mathrm{SV}}} y^i \, \alpha_i \, \exp(-\gamma \|\boldsymbol{x} - \boldsymbol{x}^i\|^2) + b$$

$$\gamma = 0.01, \, C = 100$$

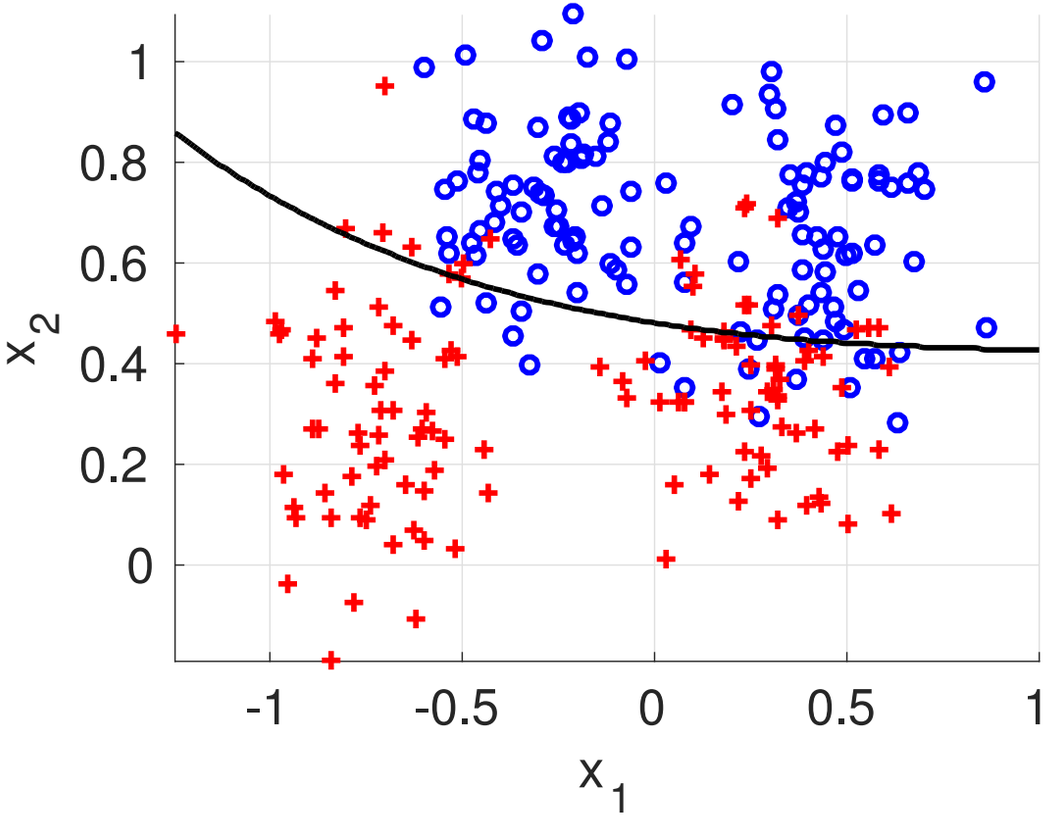


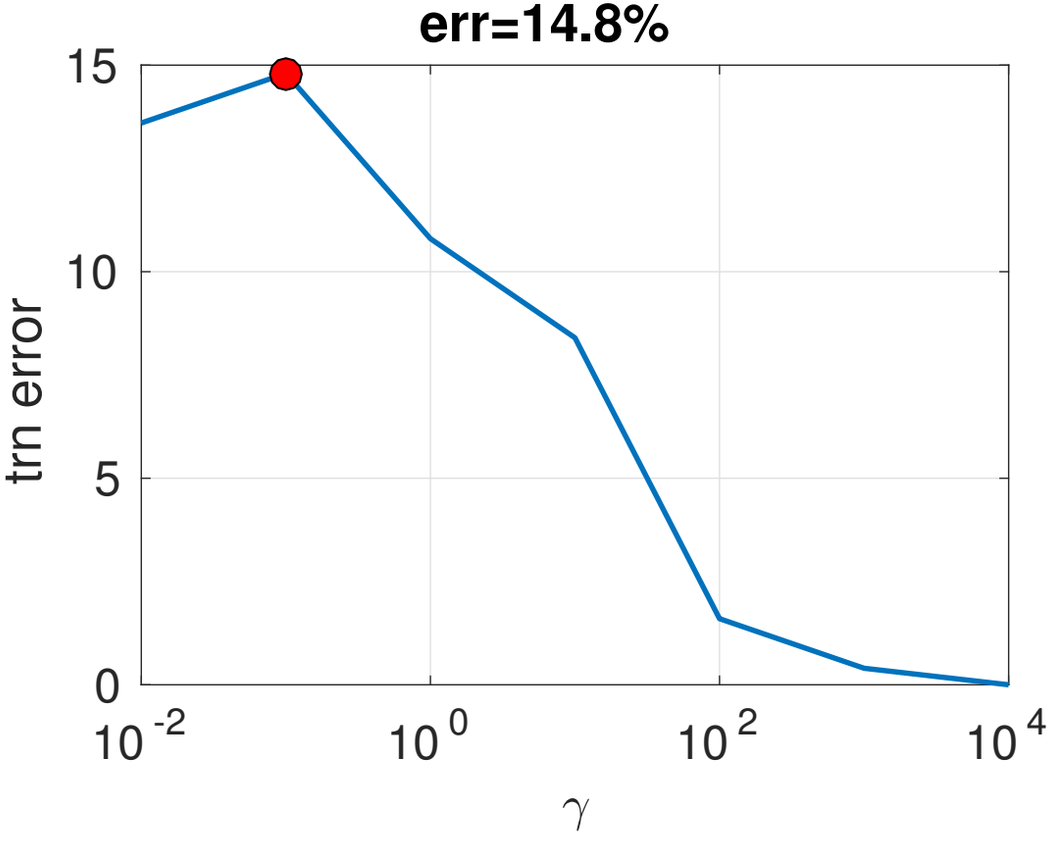


Example: SVM with RBF kernel
training error vs. γ

$$f(\boldsymbol{x}) = \sum_{i \in \mathcal{I}_{\mathrm{SV}}} y^i \, \alpha_i \, \exp(-\gamma \|\boldsymbol{x} - \boldsymbol{x}^i\|^2) + b$$

$$\gamma = 0.1, \, C = 100$$

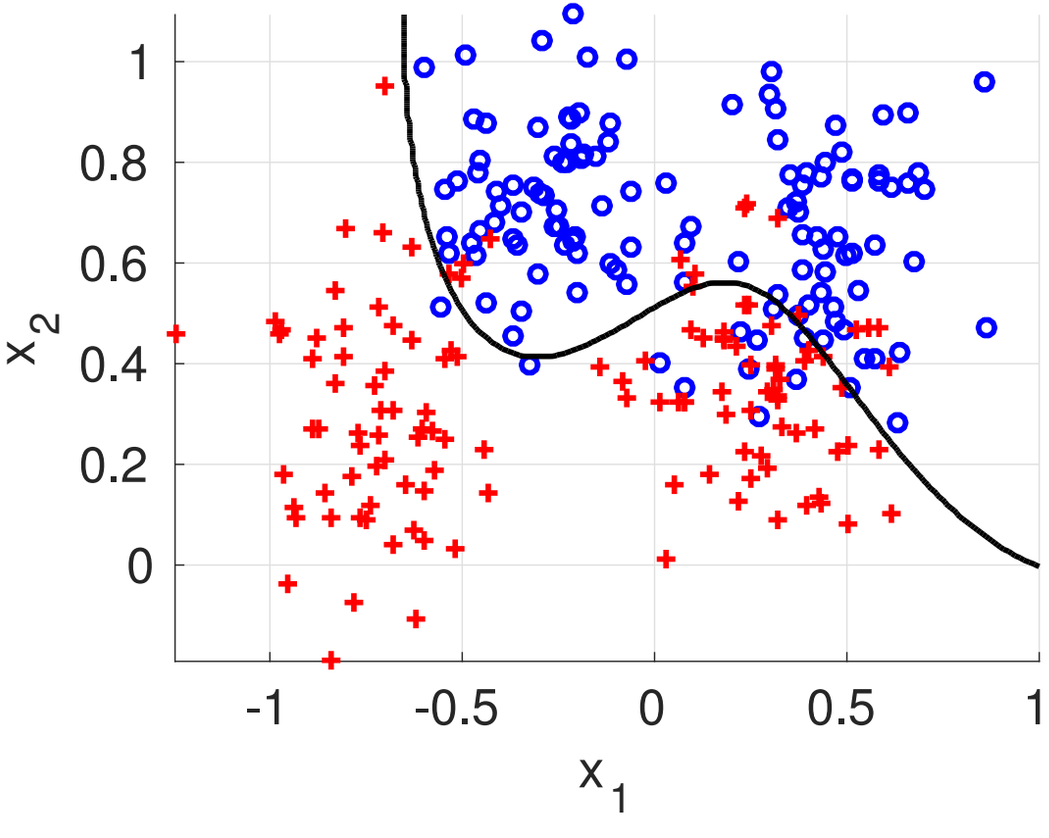


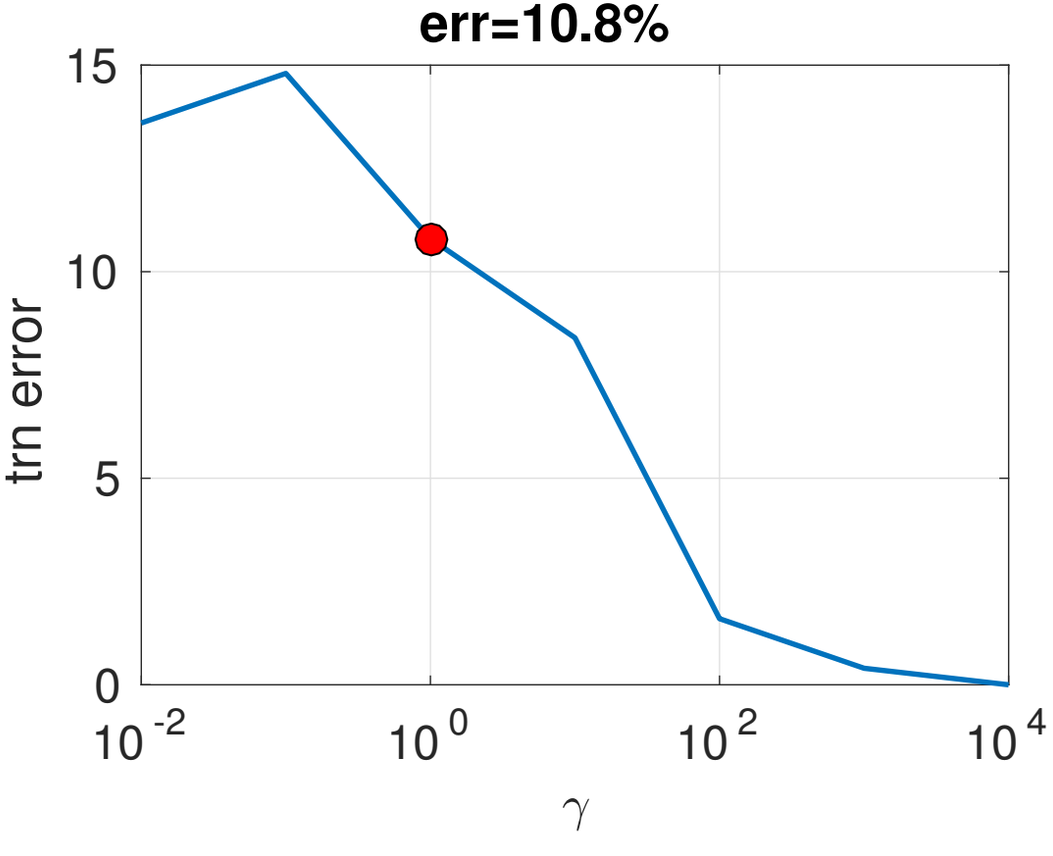


Example: SVM with RBF kernel
training error vs. γ

$$f(\boldsymbol{x}) = \sum_{i \in \mathcal{I}_{\mathrm{SV}}} y^i \, \alpha_i \, \exp(-\gamma \|\boldsymbol{x} - \boldsymbol{x}^i\|^2) + b$$

$$\gamma = 1, \, C = 100$$

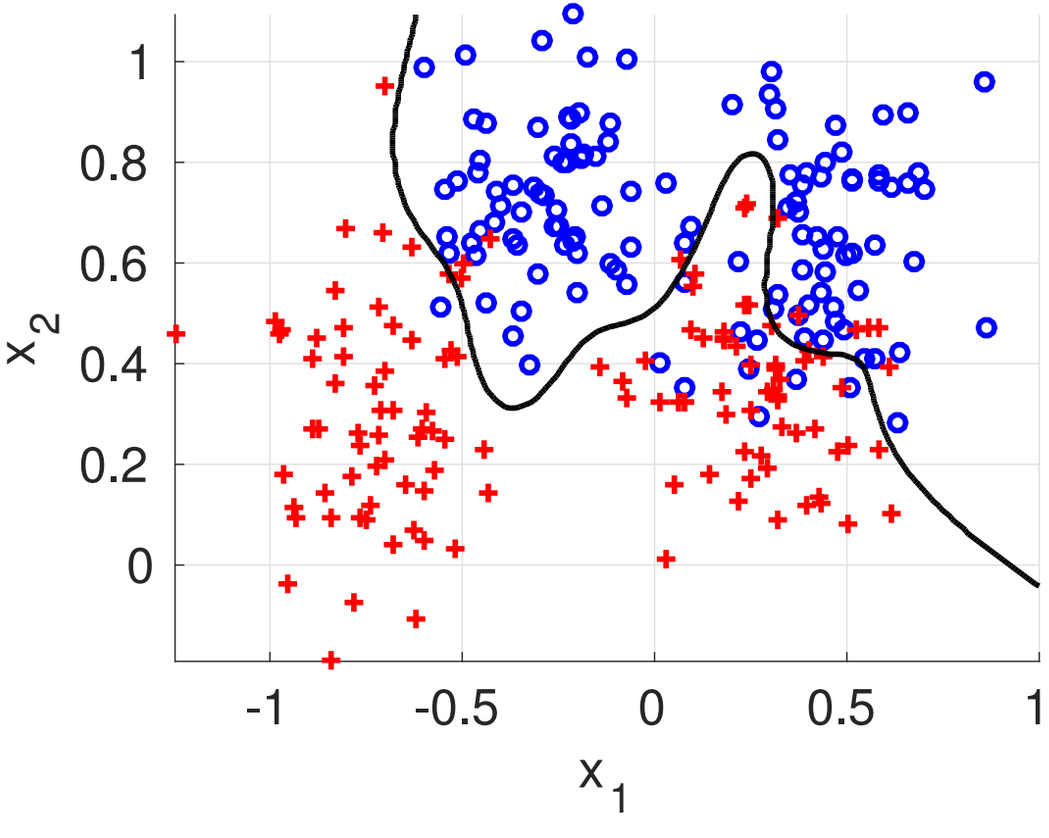


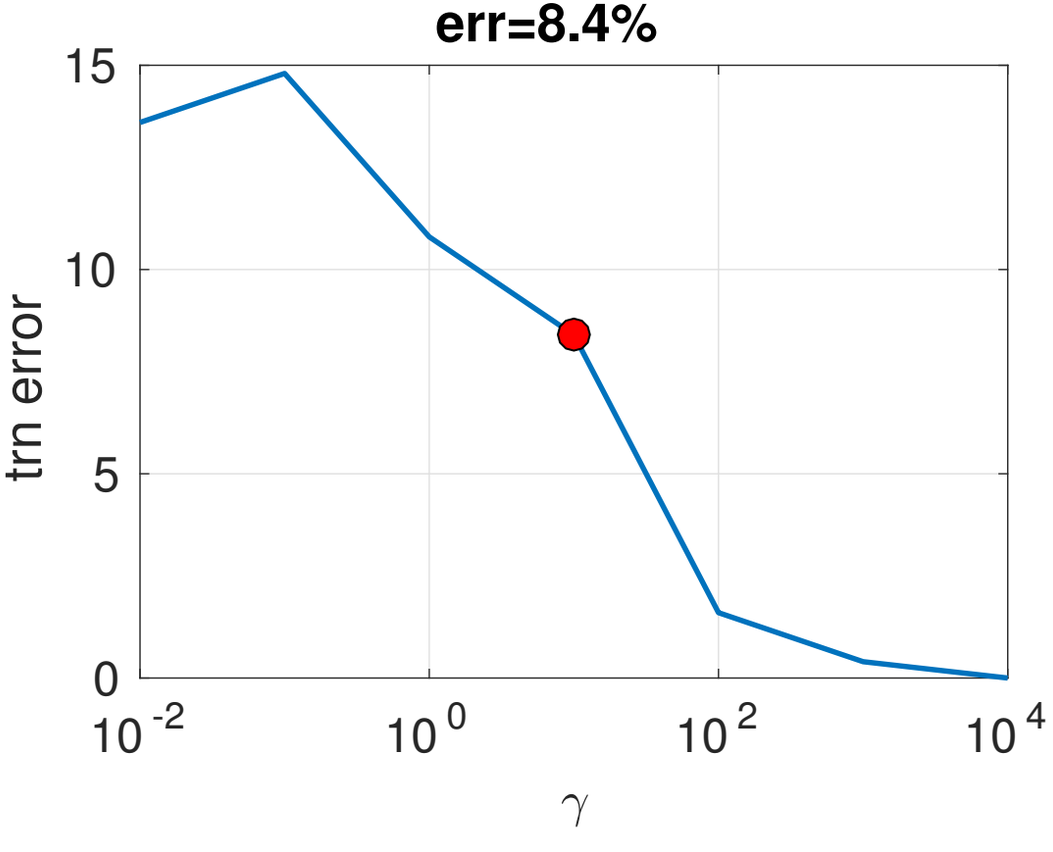


Example: SVM with RBF kernel
training error vs. γ

$$f(\boldsymbol{x}) = \sum_{i \in \mathcal{I}_{\mathrm{SV}}} y^i \, \alpha_i \, \exp(-\gamma \|\boldsymbol{x} - \boldsymbol{x}^i\|^2) + b$$

$$\gamma = 10, \; C = 100$$

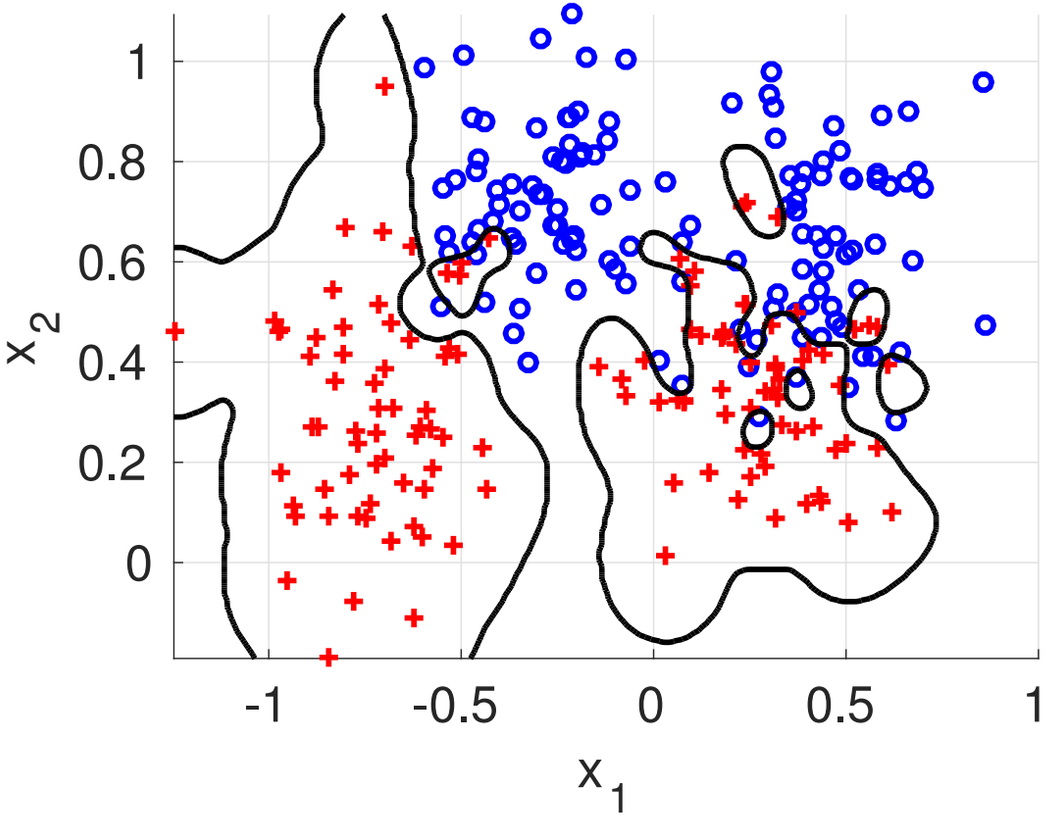


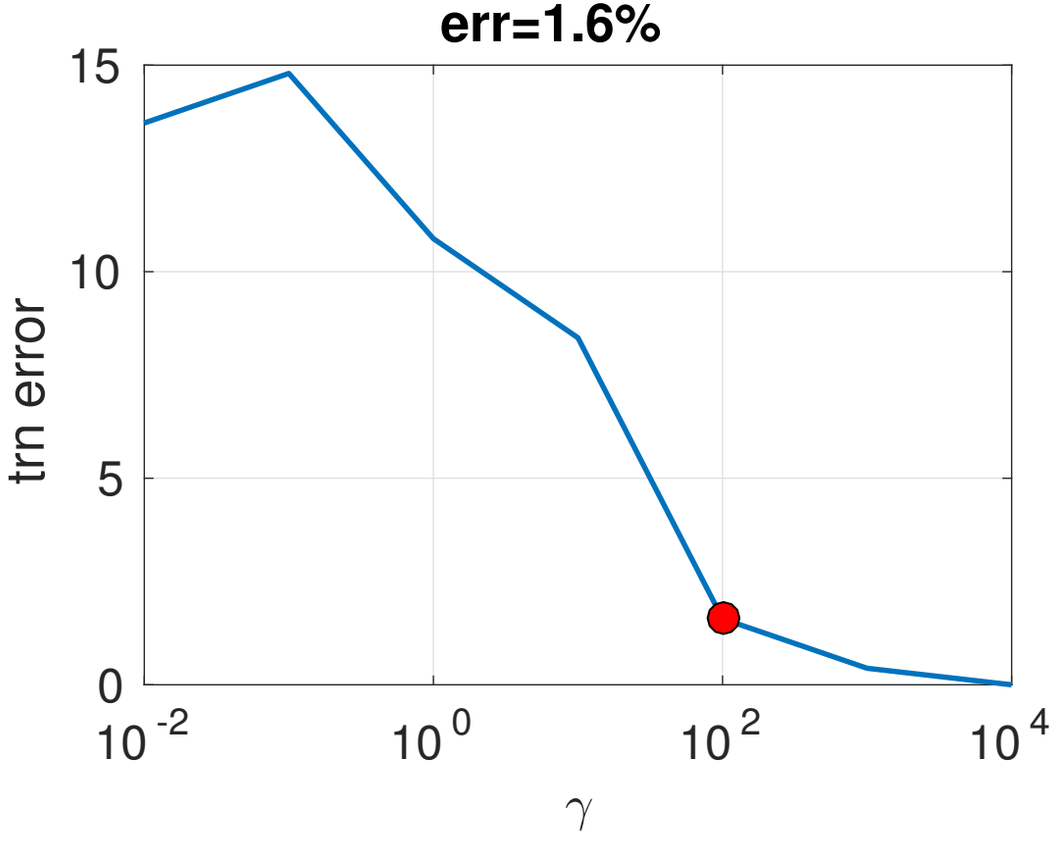


Example: SVM with RBF kernel
training error vs. γ

$$f(\boldsymbol{x}) = \sum_{i \in \mathcal{I}_{\mathrm{SV}}} y^i \, \alpha_i \, \exp(-\gamma \|\boldsymbol{x} - \boldsymbol{x}^i\|^2) + b$$

$$\gamma = 100, \; C = 100$$

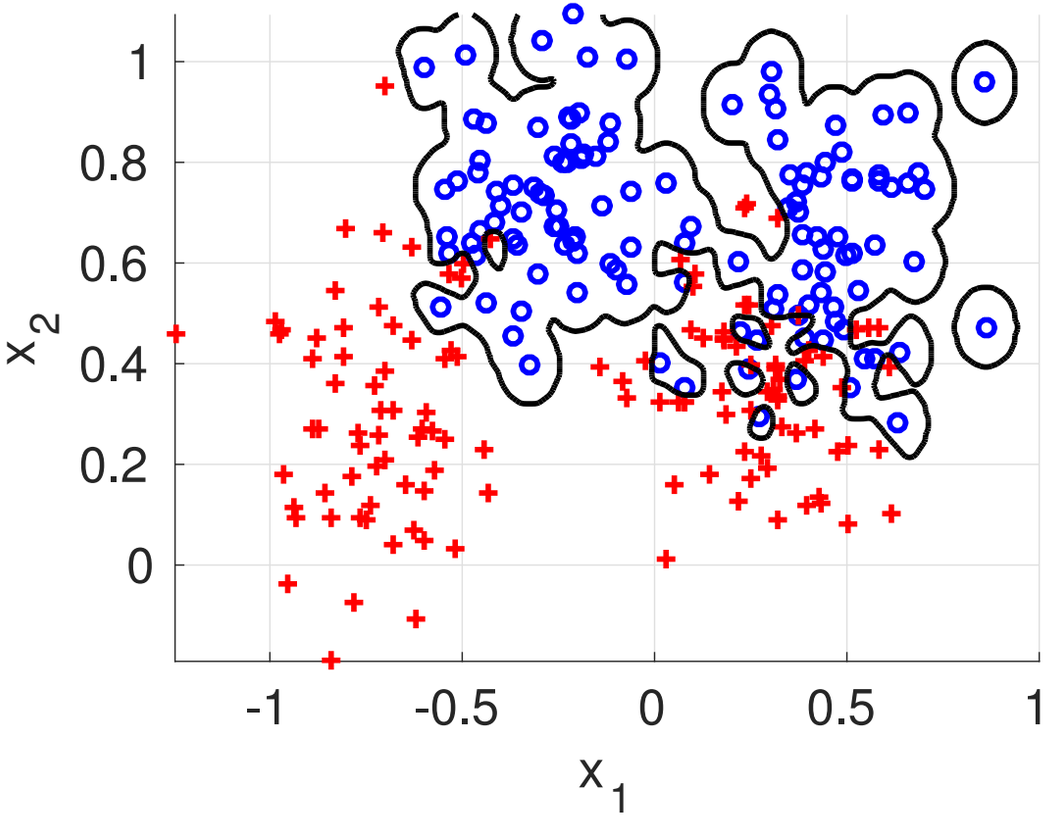


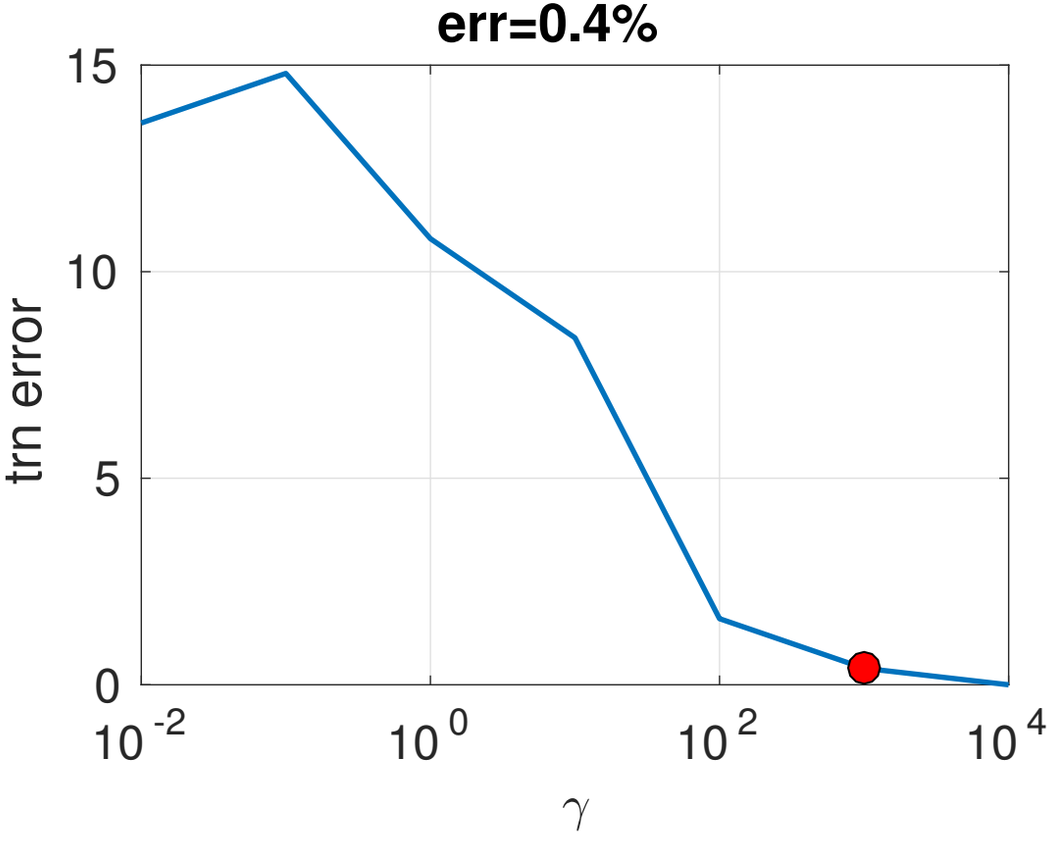


Example: SVM with RBF kernel
training error vs. γ

$$f(\boldsymbol{x}) = \sum_{i \in \mathcal{I}_{\mathrm{SV}}} y^i \, \alpha_i \, \exp(-\gamma \|\boldsymbol{x} - \boldsymbol{x}^i\|^2) + b$$

$$\gamma = 1000, \; C = 100$$

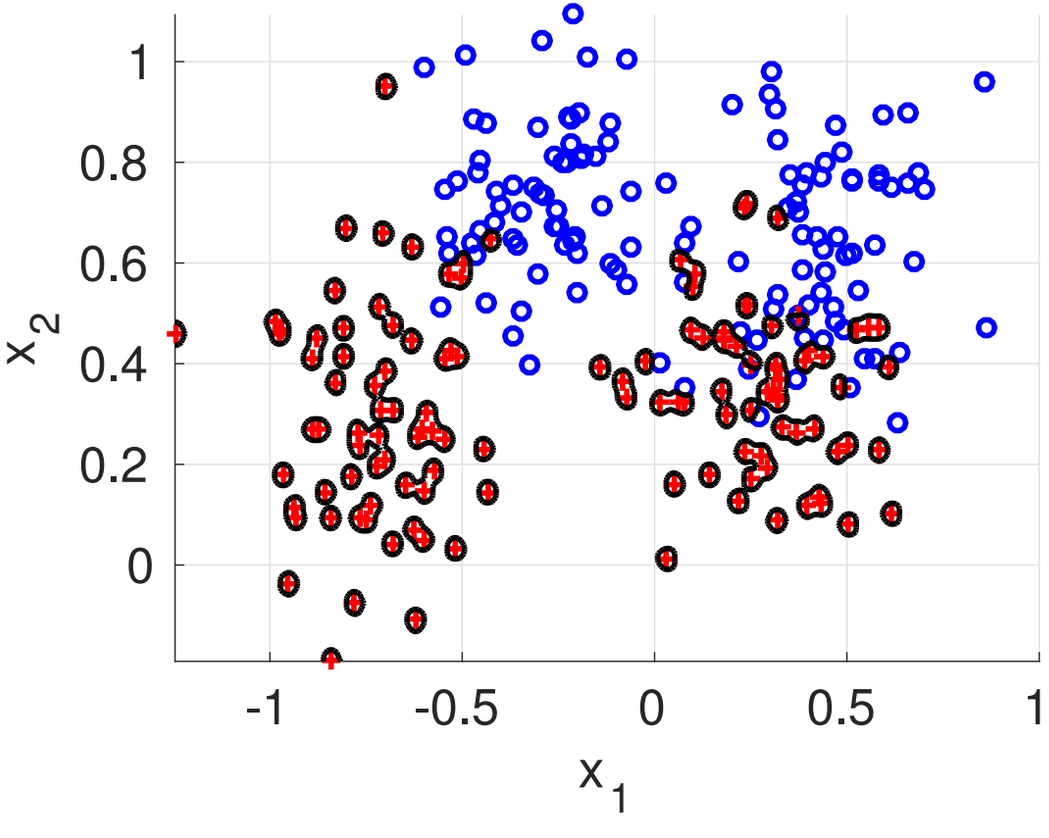


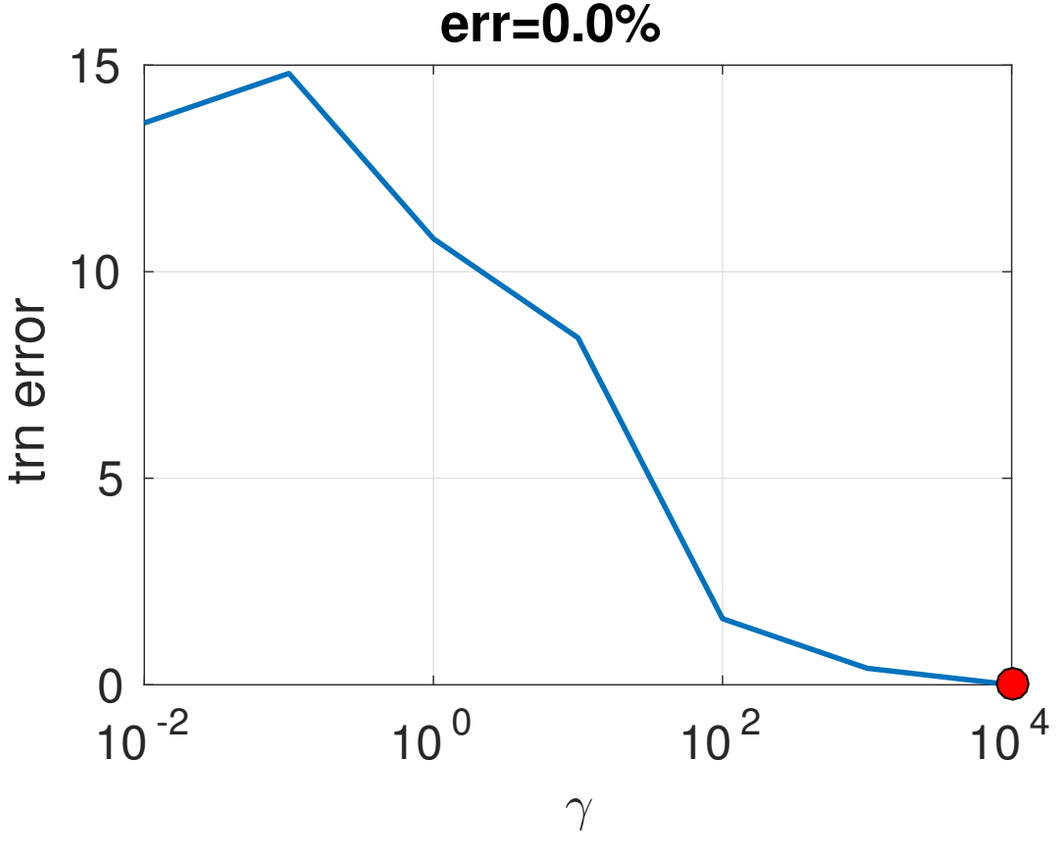


Example: SVM with RBF kernel
training error vs. γ

$$f(\boldsymbol{x}) = \sum_{i \in \mathcal{I}_{\mathrm{SV}}} y^i \, \alpha_i \, \exp(-\gamma \|\boldsymbol{x} - \boldsymbol{x}^i\|^2) + b$$

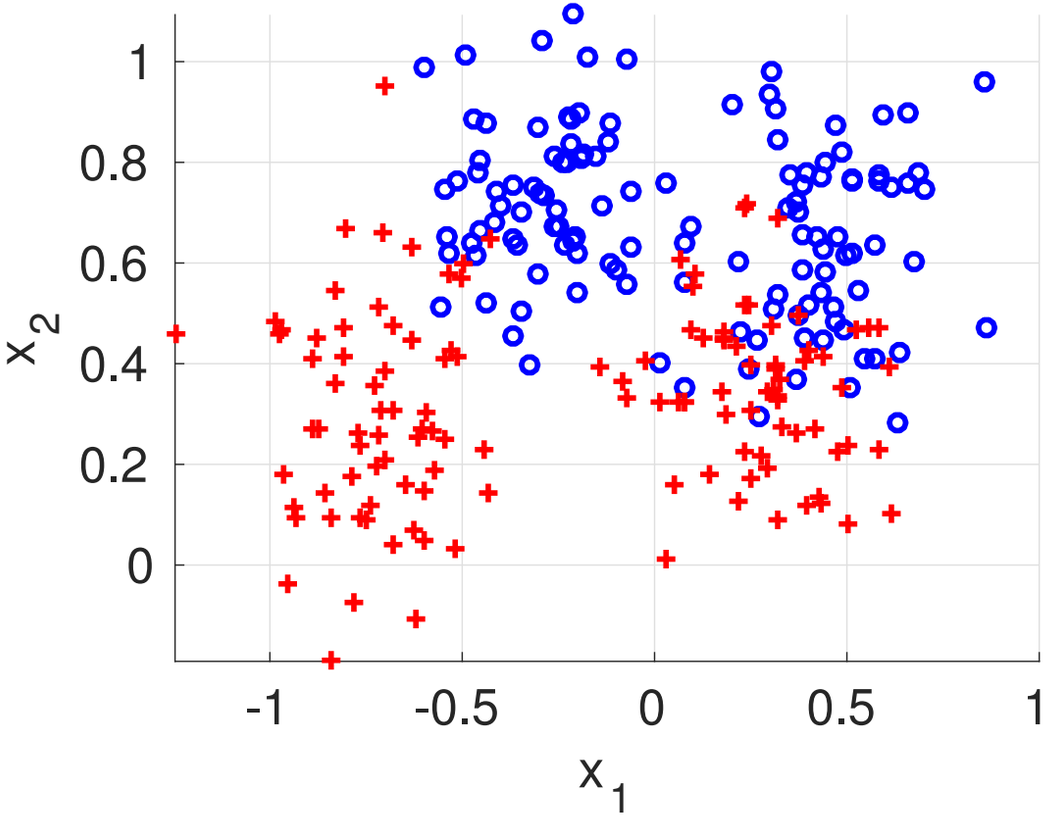
$$\gamma = 10000, \; C = 100$$





Example: SVM with RBF kernel
training error vs. C

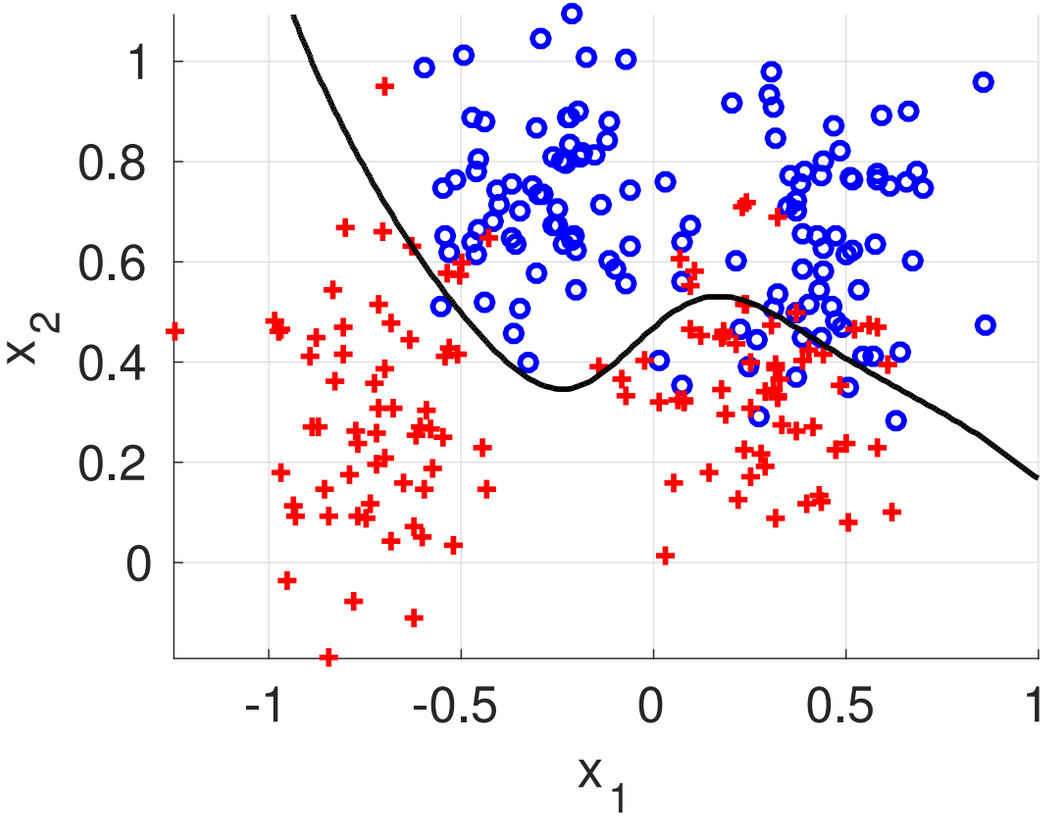
$$f(\boldsymbol{x}) = \sum_{i \in \mathcal{I}_{\mathrm{SV}}} y^i \, \alpha_i \, \exp(-\gamma \|\boldsymbol{x} - \boldsymbol{x}^i\|^2) + b$$

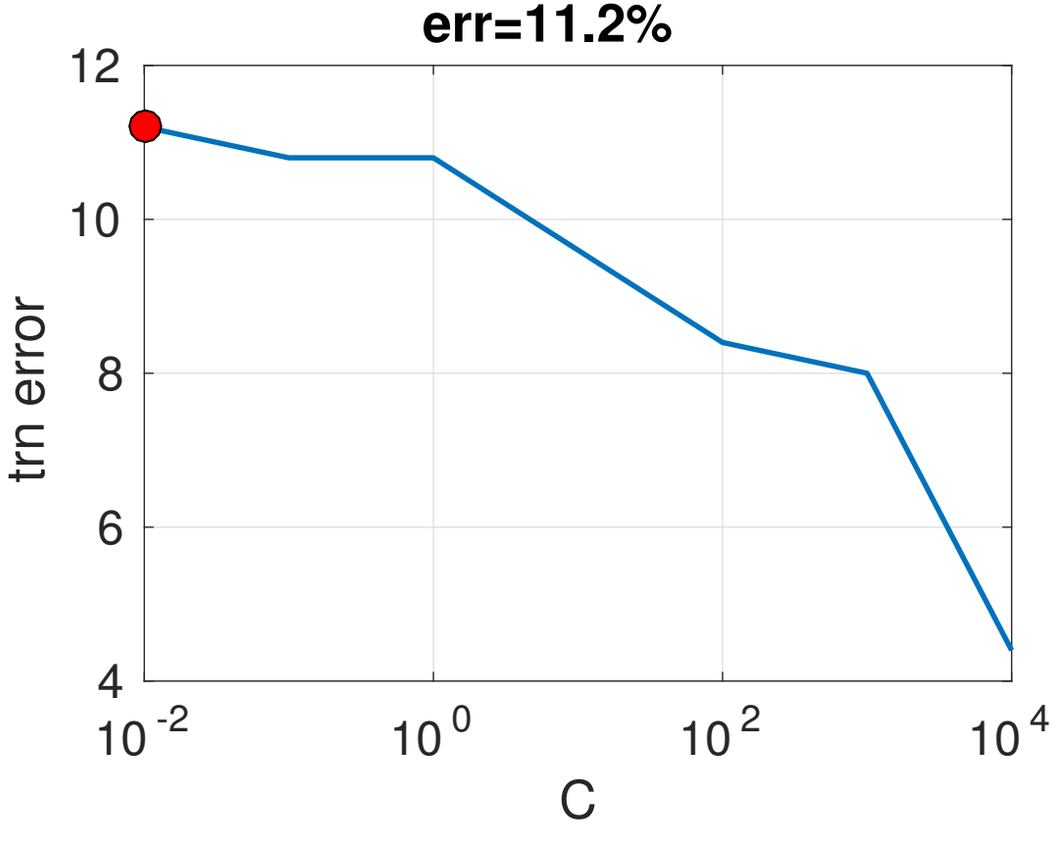


Example: SVM with RBF kernel
training error vs. C

$$f(\boldsymbol{x}) = \sum_{i \in \mathcal{I}_{\mathrm{SV}}} y^i \, \alpha_i \, \exp(-\gamma \|\boldsymbol{x} - \boldsymbol{x}^i\|^2) + b$$

$$\gamma = 10, \; C = 0.01$$

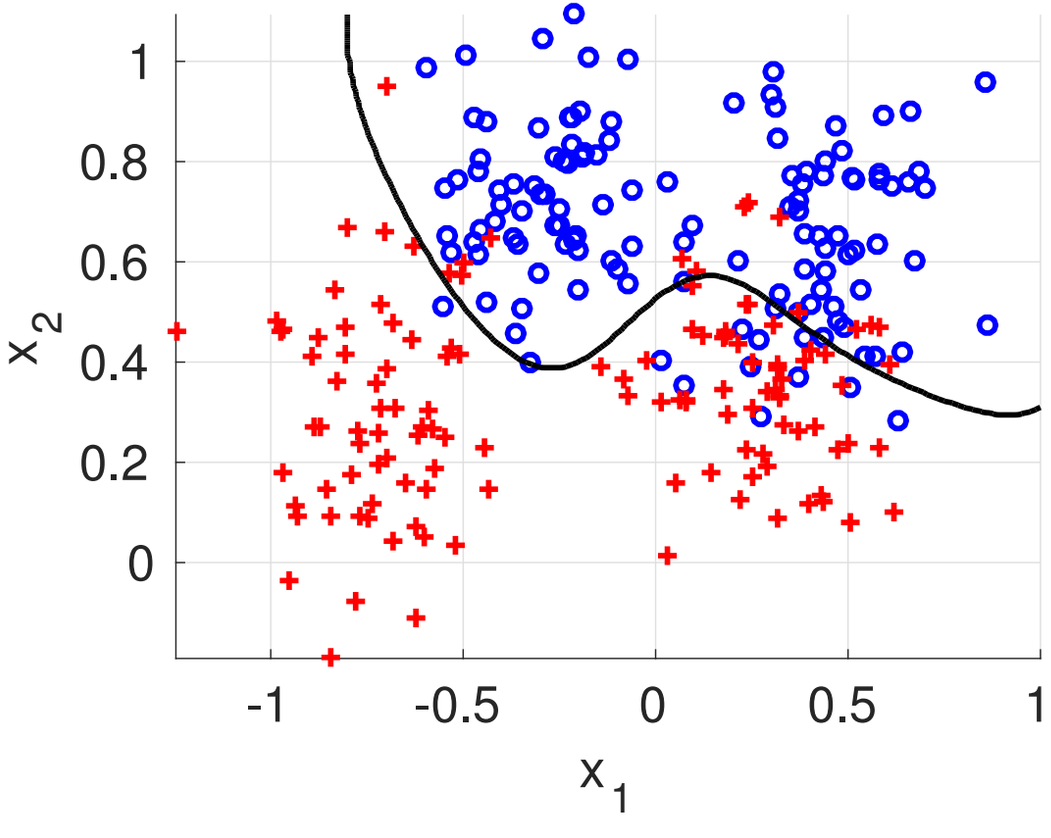


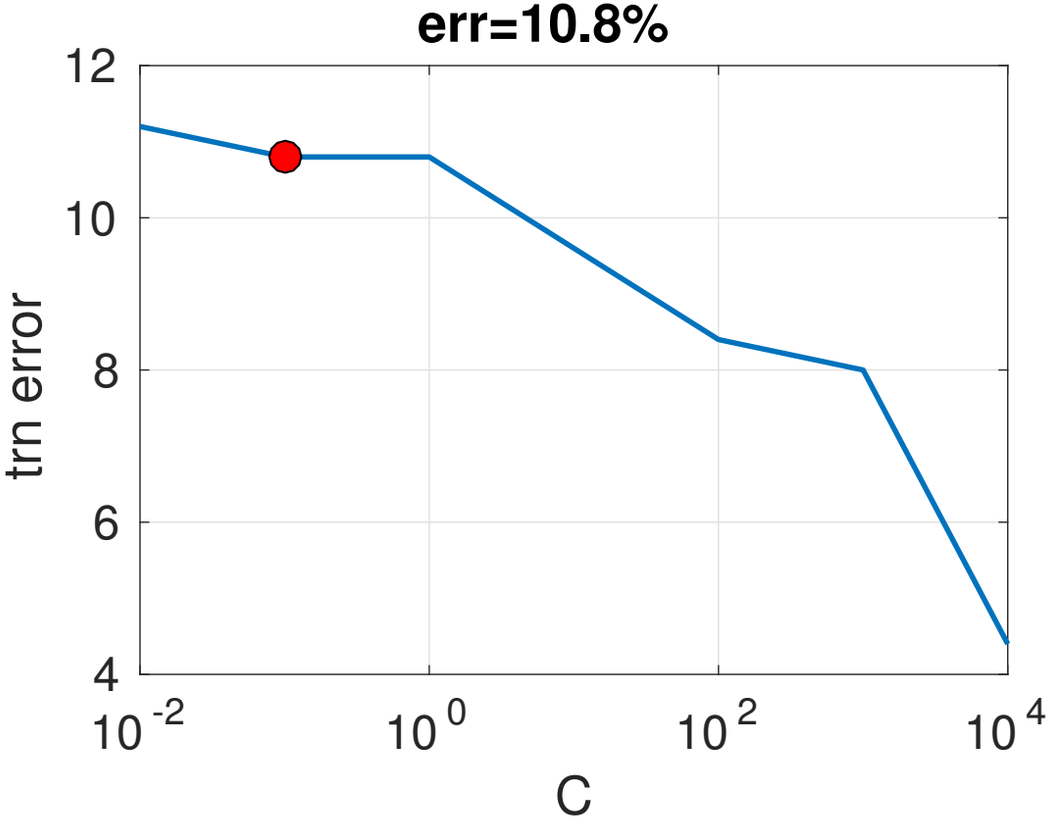


Example: SVM with RBF kernel
training error vs. C

$$f(\boldsymbol{x}) = \sum_{i \in \mathcal{I}_{\mathrm{SV}}} y^i \, \alpha_i \, \exp(-\gamma \|\boldsymbol{x} - \boldsymbol{x}^i\|^2) + b$$

$$\gamma = 10, \; C = 0.1$$

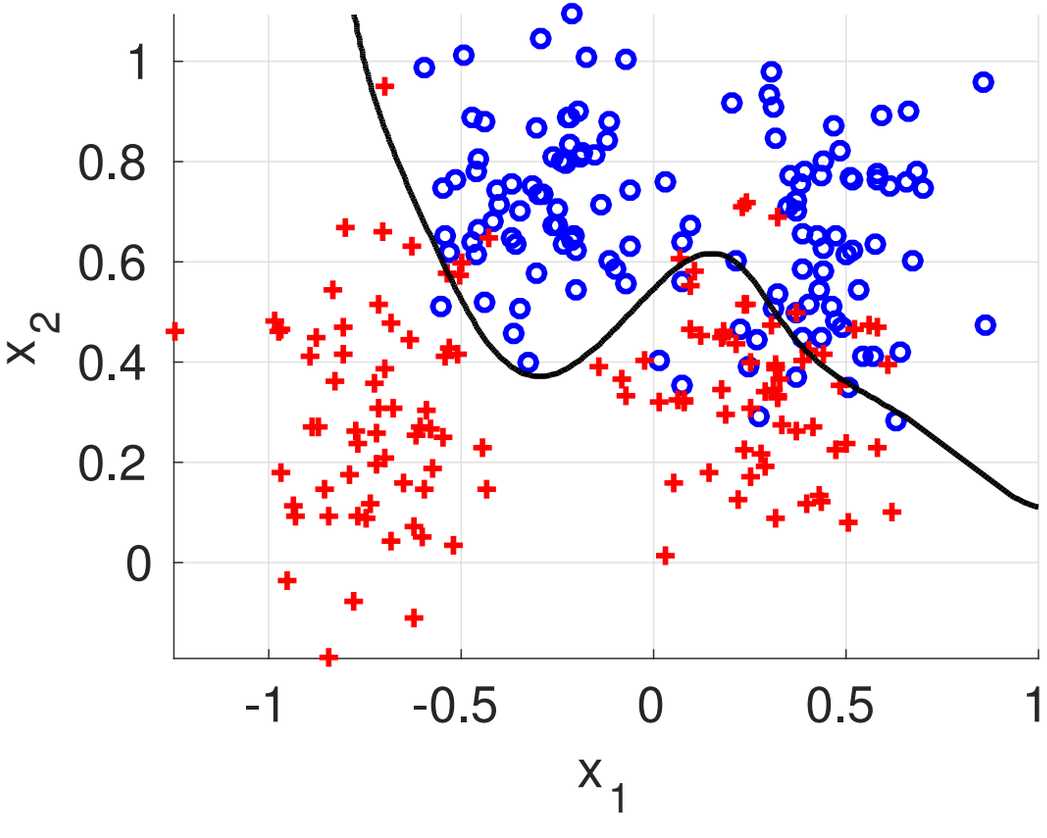


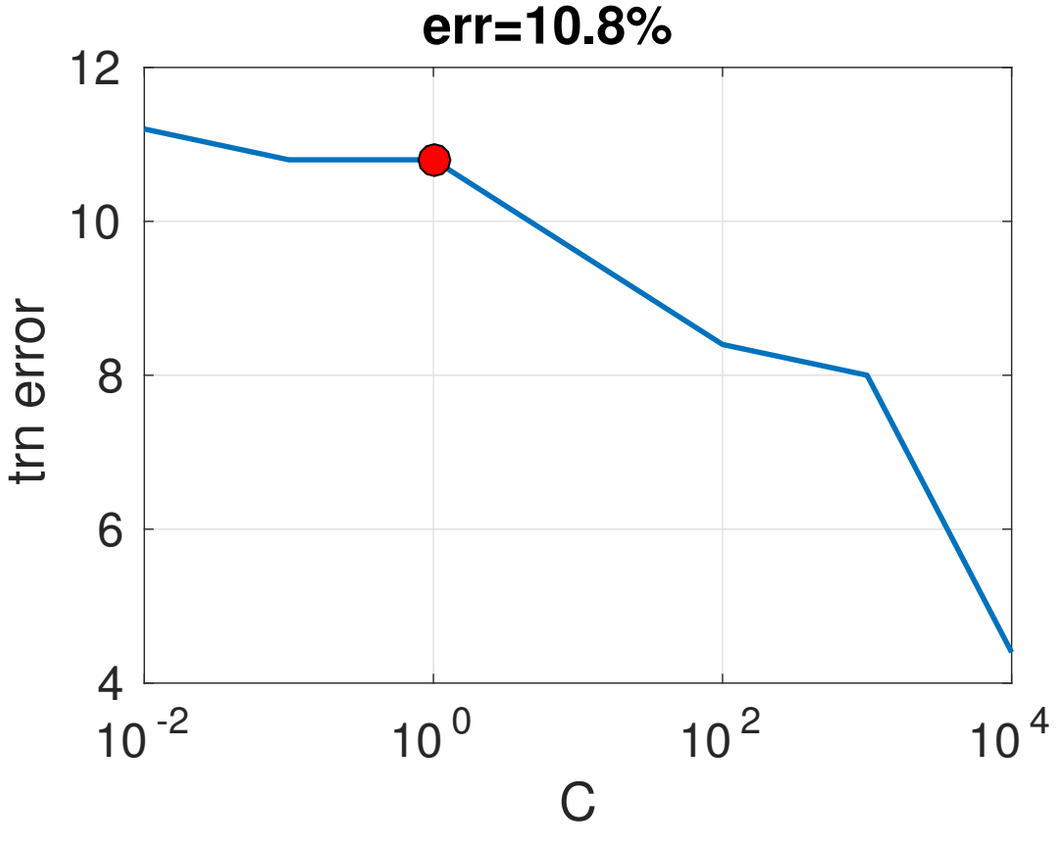


Example: SVM with RBF kernel
training error vs. C

$$f(\boldsymbol{x}) = \sum_{i \in \mathcal{I}_{\mathrm{SV}}} y^i \, \alpha_i \, \exp(-\gamma \|\boldsymbol{x} - \boldsymbol{x}^i\|^2) + b$$

$$\gamma = 10, \; C = 1$$

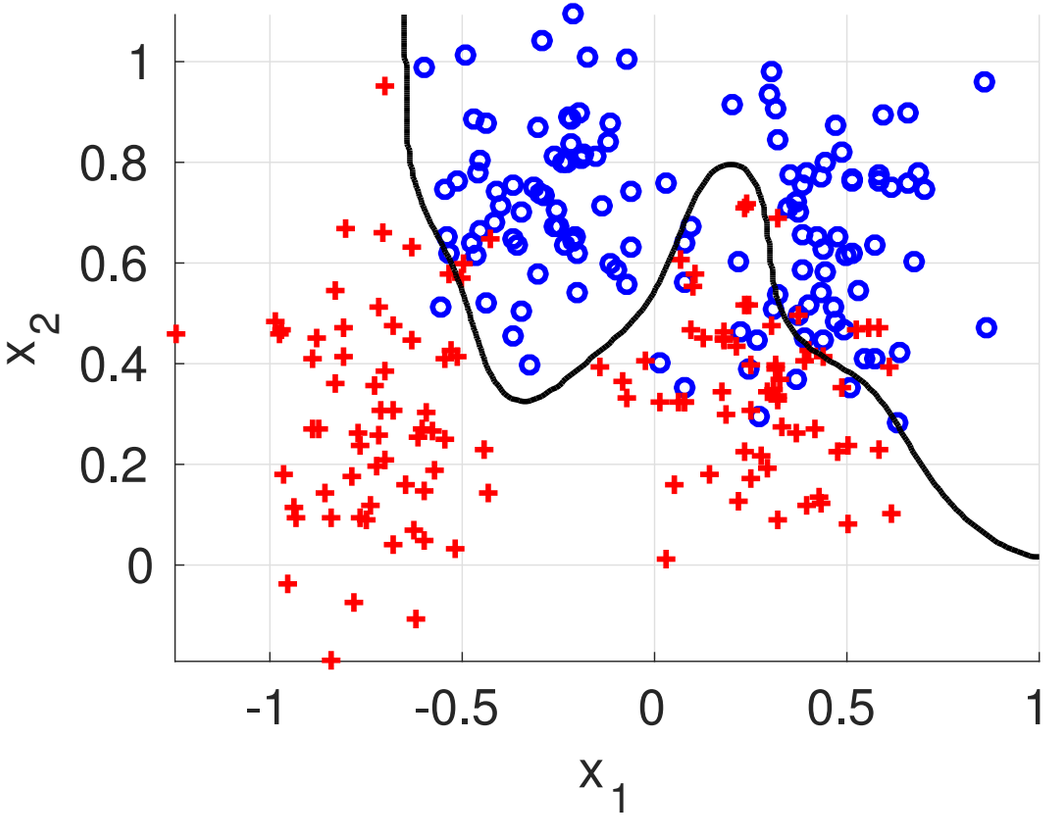


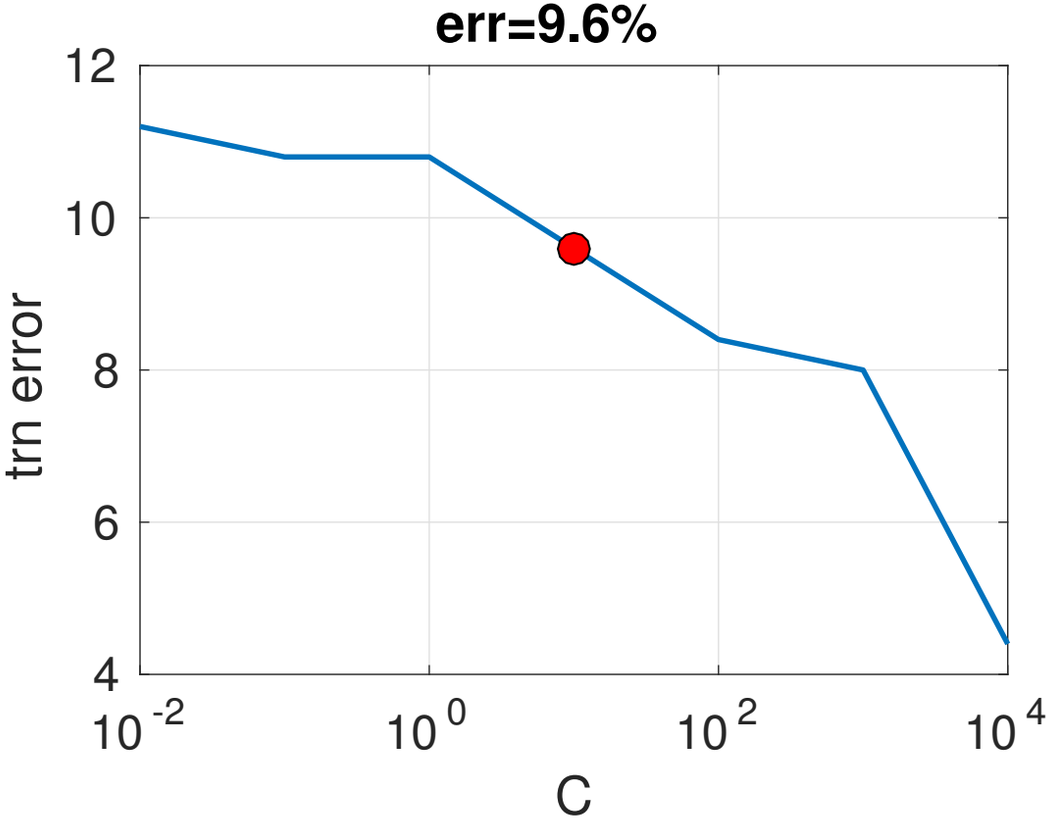


Example: SVM with RBF kernel
training error vs. C

$$f(\boldsymbol{x}) = \sum_{i \in \mathcal{I}_{\mathrm{SV}}} y^i \, \alpha_i \, \exp(-\gamma \|\boldsymbol{x} - \boldsymbol{x}^i\|^2) + b$$

$$\gamma = 10, \, C = 10$$

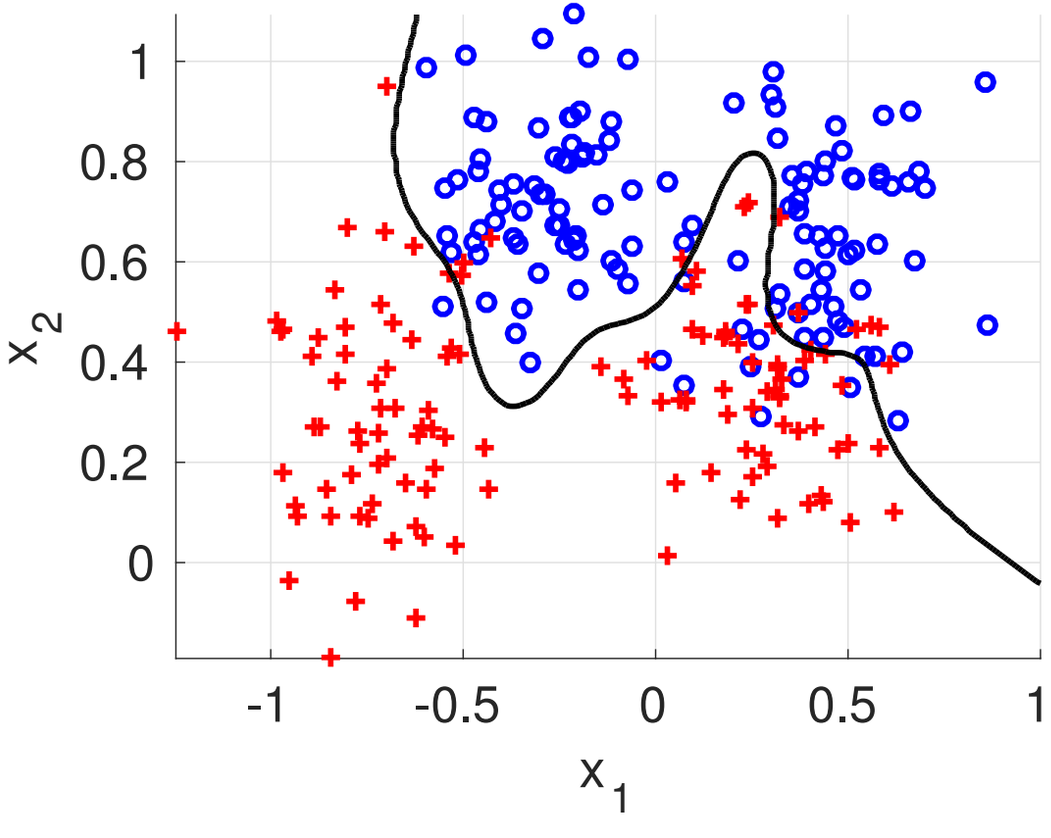


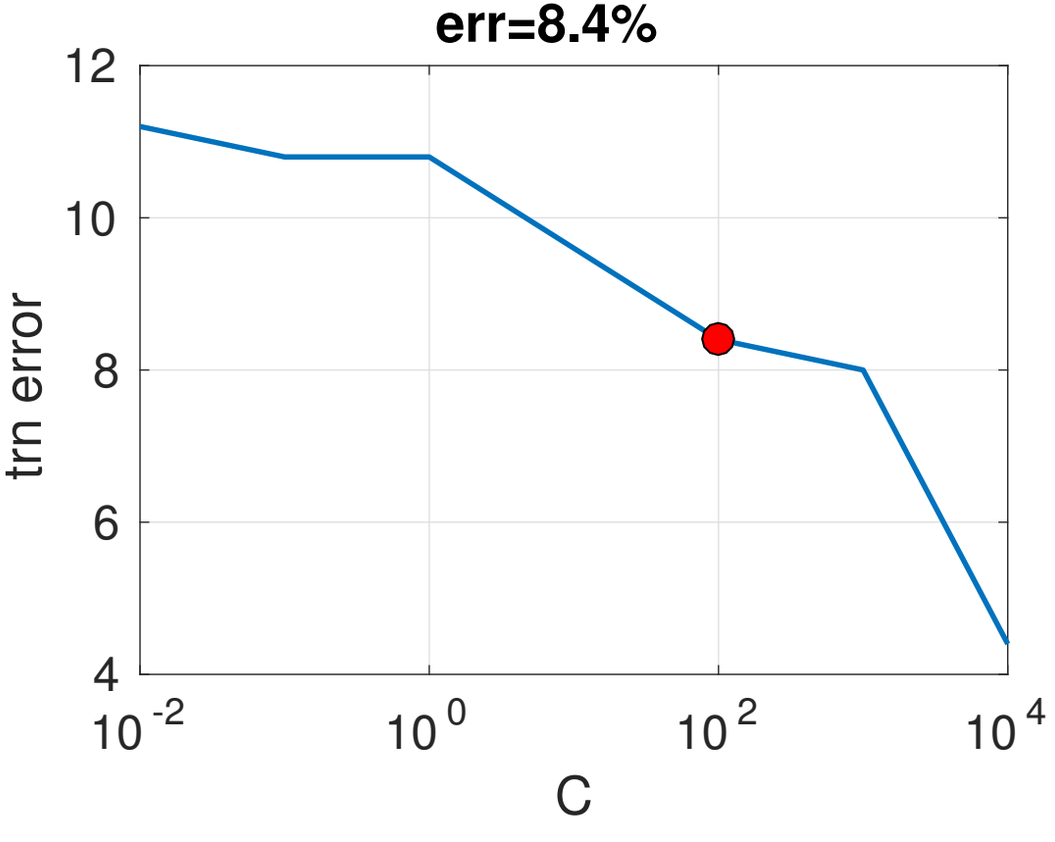


Example: SVM with RBF kernel
training error vs. C

$$f(\boldsymbol{x}) = \sum_{i \in \mathcal{I}_{\mathrm{SV}}} y^i \, \alpha_i \, \exp(-\gamma \|\boldsymbol{x} - \boldsymbol{x}^i\|^2) + b$$

$$\gamma = 10, \; C = 100$$

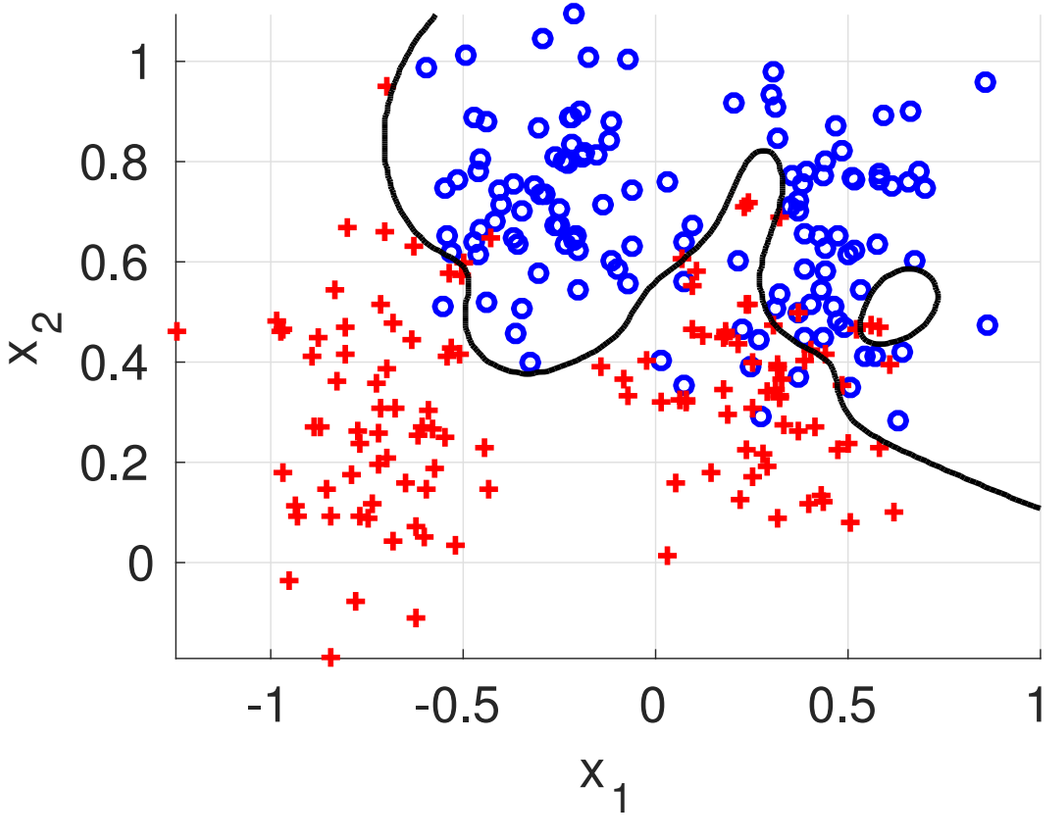


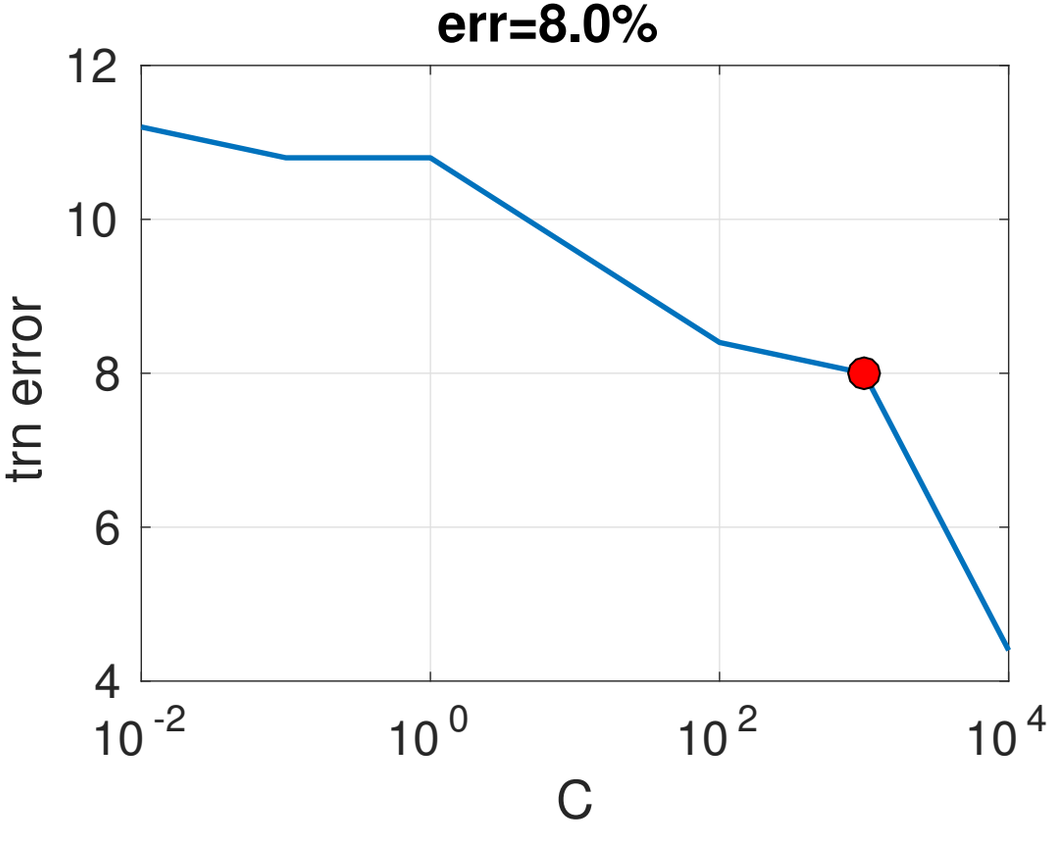


Example: SVM with RBF kernel
training error vs. C

$$f(\boldsymbol{x}) = \sum_{i \in \mathcal{I}_{\mathrm{SV}}} y^i \, \alpha_i \, \exp(-\gamma \|\boldsymbol{x} - \boldsymbol{x}^i\|^2) + b$$

$$\gamma = 10, \; C = 1000$$

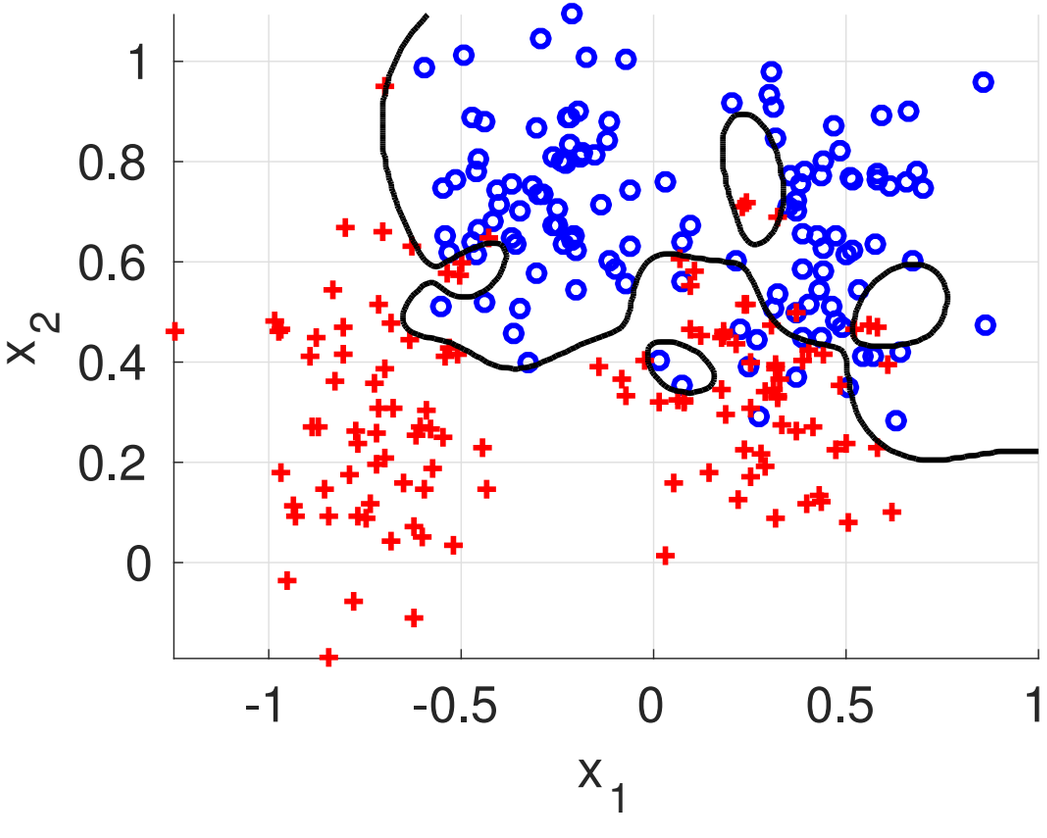


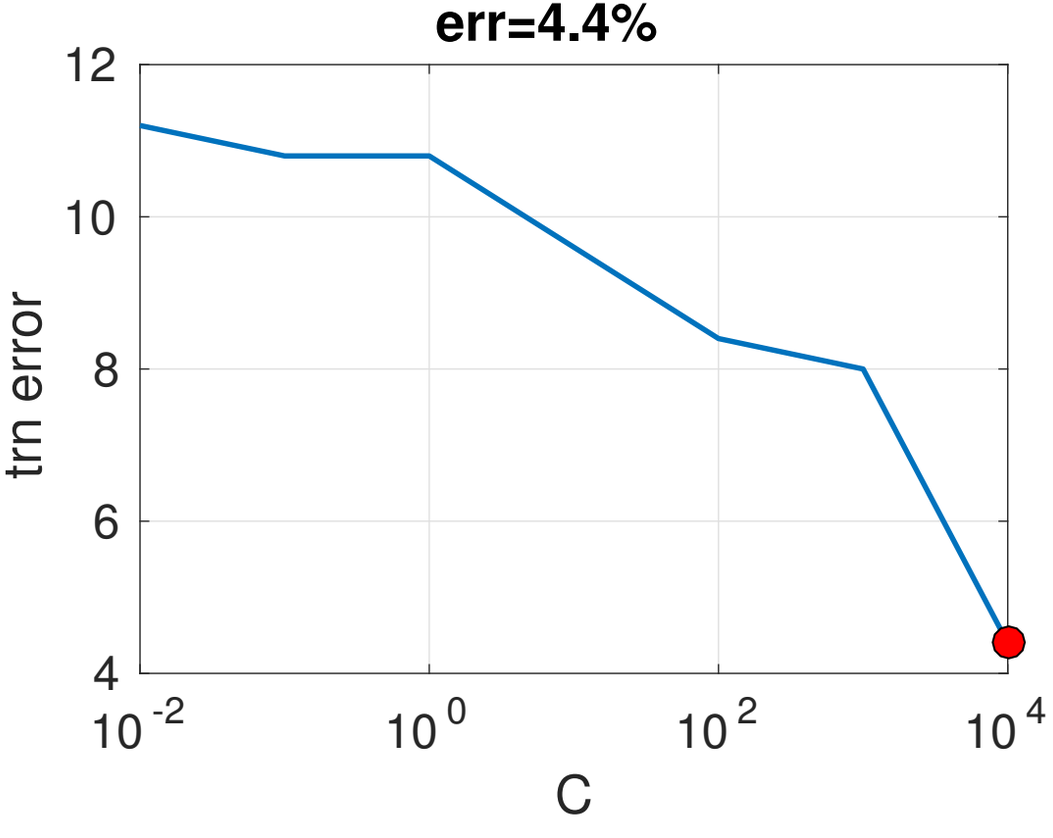


Example: SVM with RBF kernel
training error vs. C

$$f(\boldsymbol{x}) = \sum_{i \in \mathcal{I}_{\mathrm{SV}}} y^i \, \alpha_i \, \exp(-\gamma \|\boldsymbol{x} - \boldsymbol{x}^i\|^2) + b$$

$$\gamma = 10, \; C = 10000$$





Linear vs. kernel SVM in terms of memory

m .. number of training examples

d .. dimensionality of the input features $\boldsymbol{x} \in \mathbb{R}^d$

n .. dimensionality of the inputs represented via feature map $\boldsymbol{\phi}$: $\mathbb{R}^d \to \mathbb{R}^n$

Memory requirements: (the number of real numbers, e.g. floats)

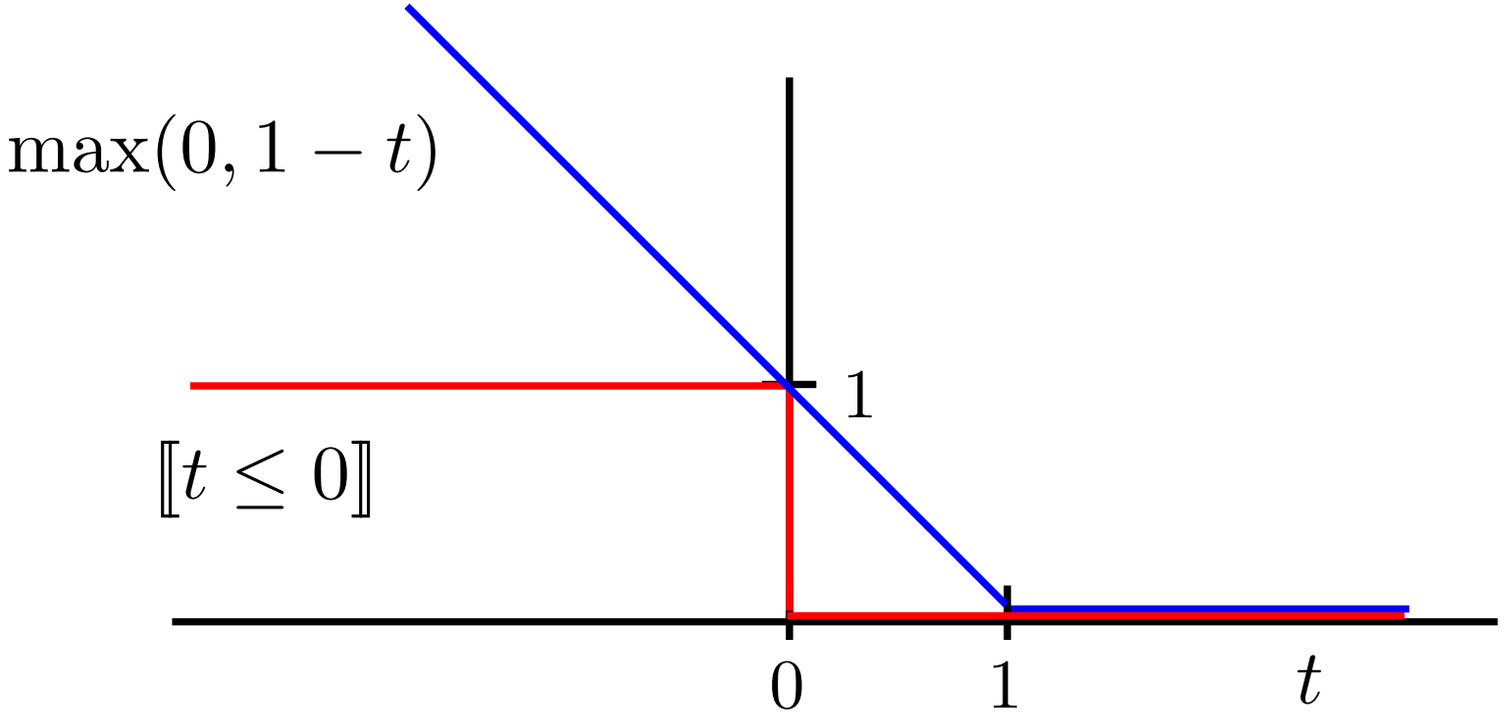
	training set $\mathcal{T}^m$	prediction rule
primal formulation	$\mathcal{O}(m \cdot n)$	$\mathcal{O}(n)$
kernel formulation	$\mathcal{O}(m^2)$	$\mathcal{O}(d \cdot \lvert\mathcal{I}_{\mathrm{SV}}\rvert)$

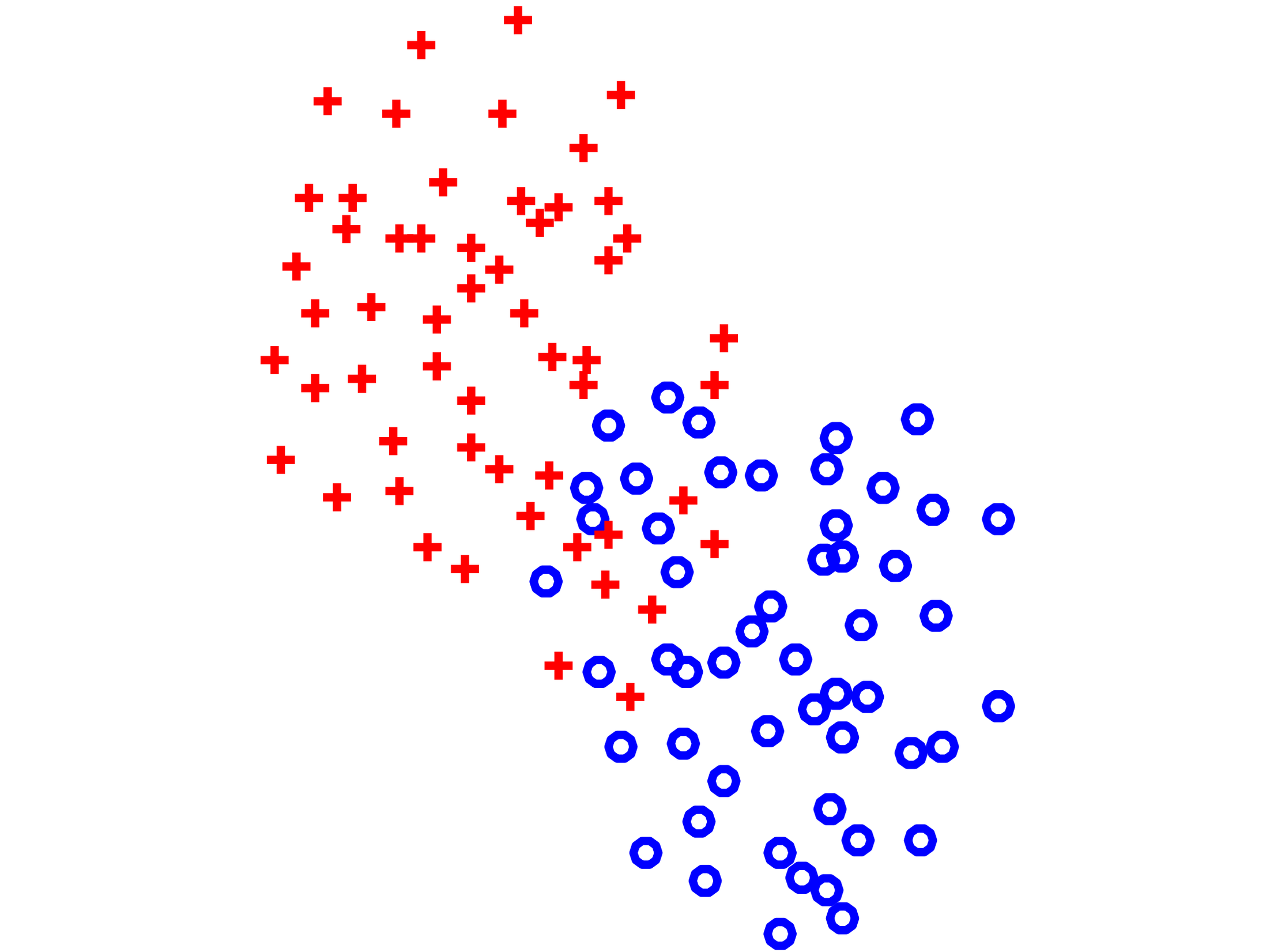
Examples of kernel functions:

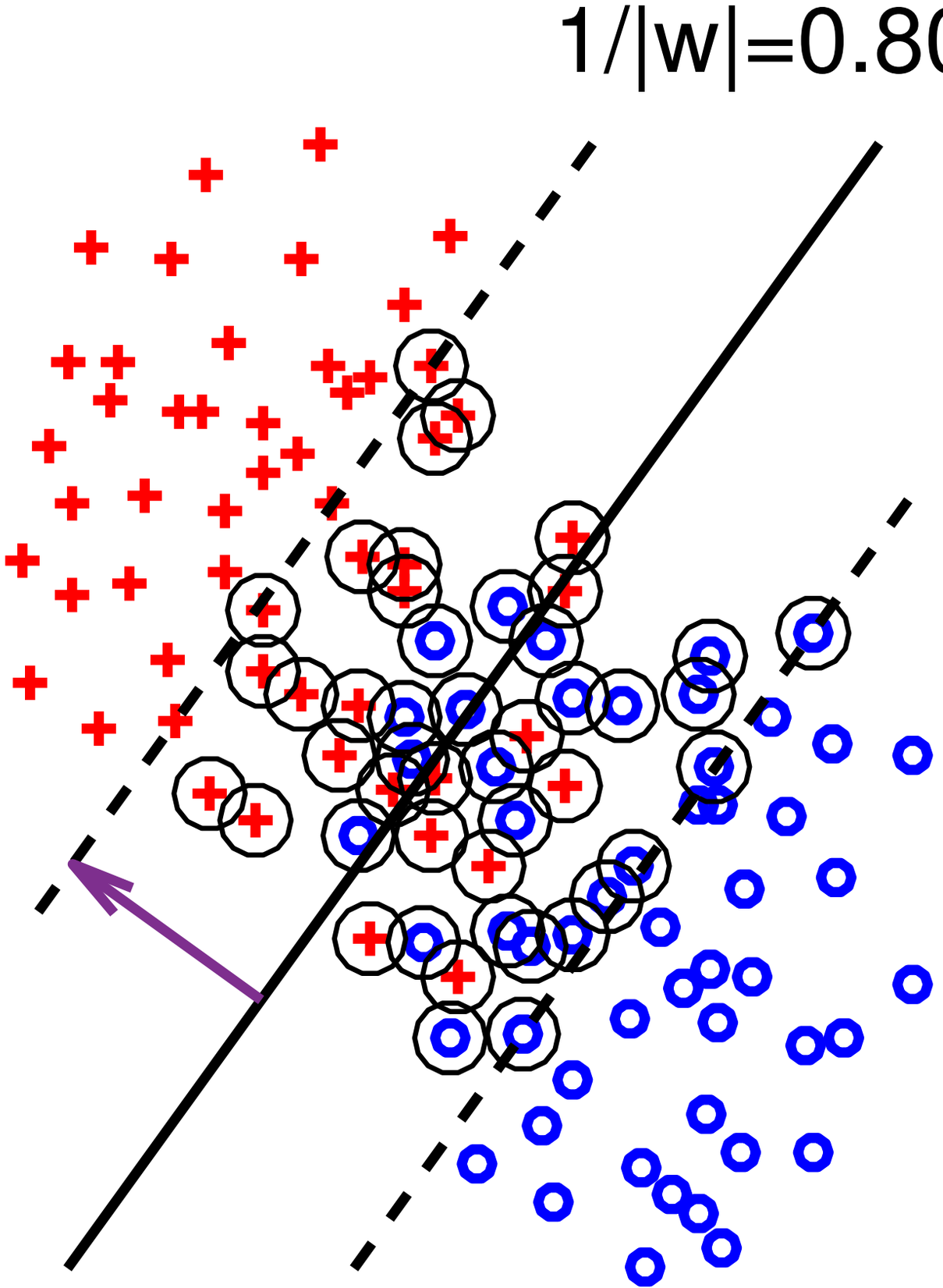
	$k(\boldsymbol{x}, \boldsymbol{x}')$	dim of $\lvert\boldsymbol{\Phi}(x)\rvert$
2nd polynomial	$\langle \boldsymbol{x}\,, \boldsymbol{x}' \rangle^2$	$n = \frac{d(d+1)}{2}$
RBF	$\exp\Big(-\gamma \, \lVert \boldsymbol{x} - \boldsymbol{x}' \rVert^2 \Big)$	$n = \infty$

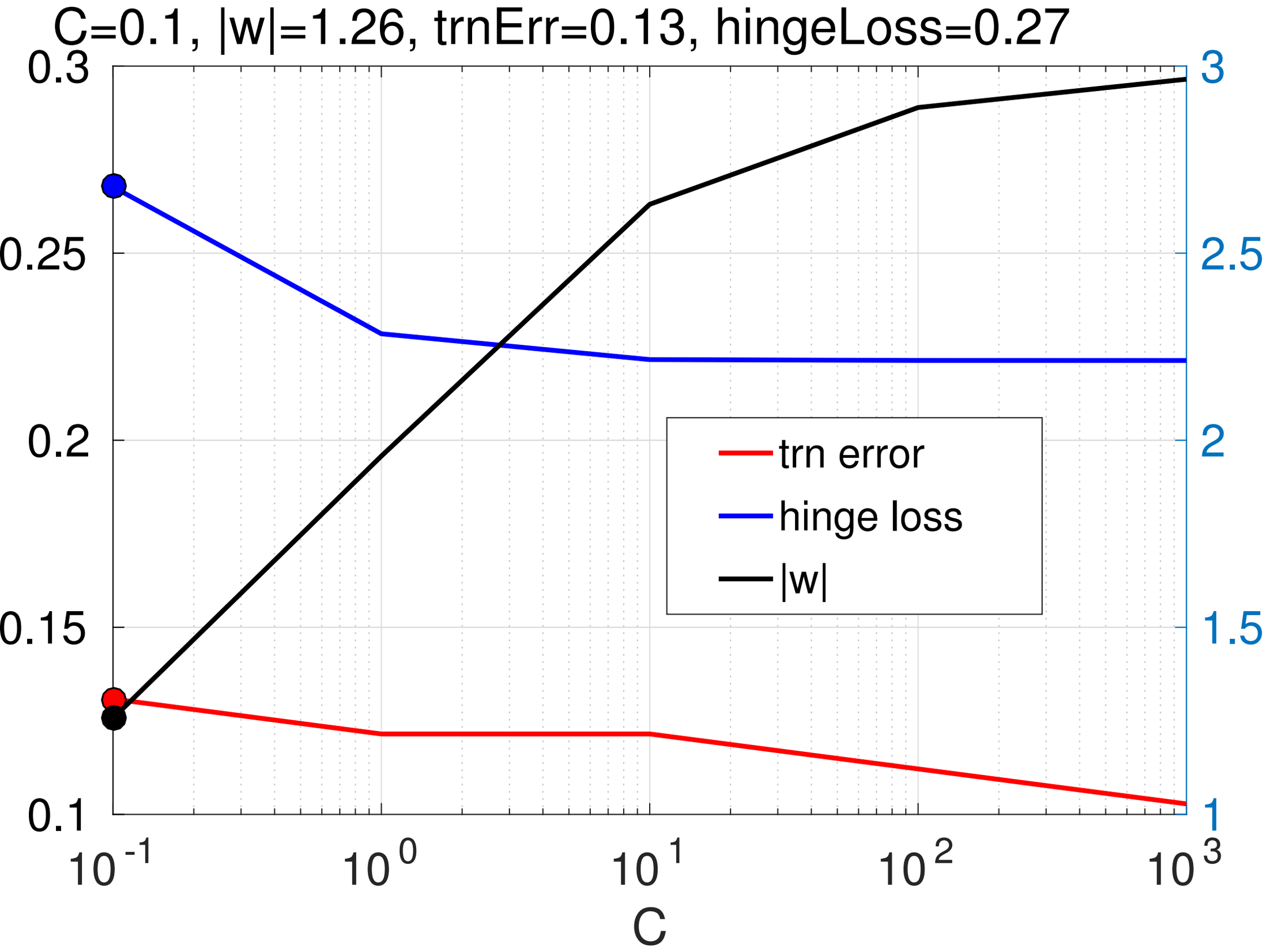
Summary

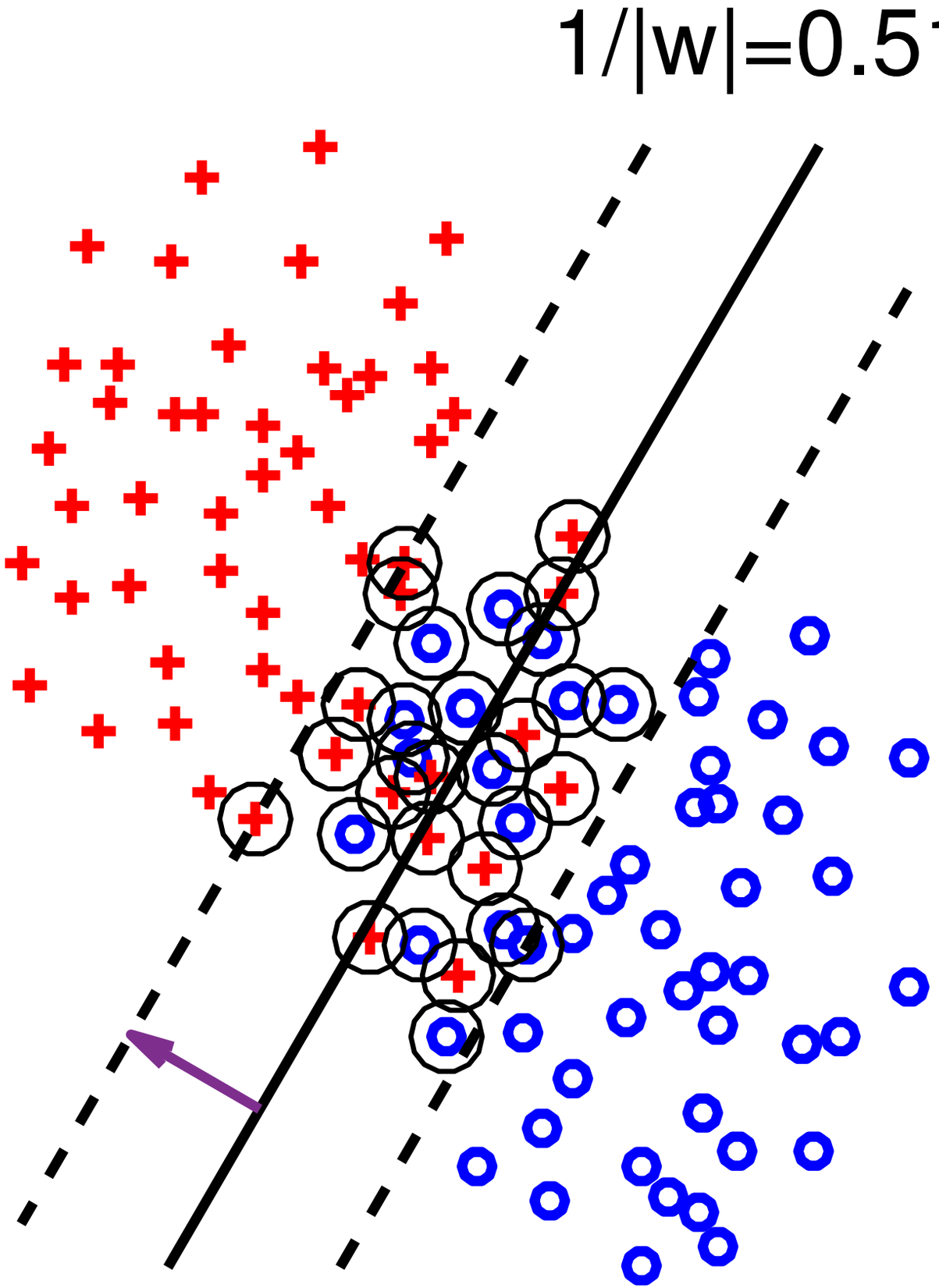
- SVM is a supervised algorithm for learning two-class linear classifier from examples with the goal to minimize the probability of misclassification on unseen examples.
- SVM converts learning into multi-critera optimization: i) minimize the training error and ii) prevent overfitting.
- SVM makes minimization of the training error tractable via using the hinge loss.
- SVM prevents overfitting via controlling the $L2$-norm of the classifier parameters.
- The experessive power of the SVM can be increased via lifting the data into a high dimensional feature space.
- Kernel trick makes learning in a high dimensional feature space tractable.

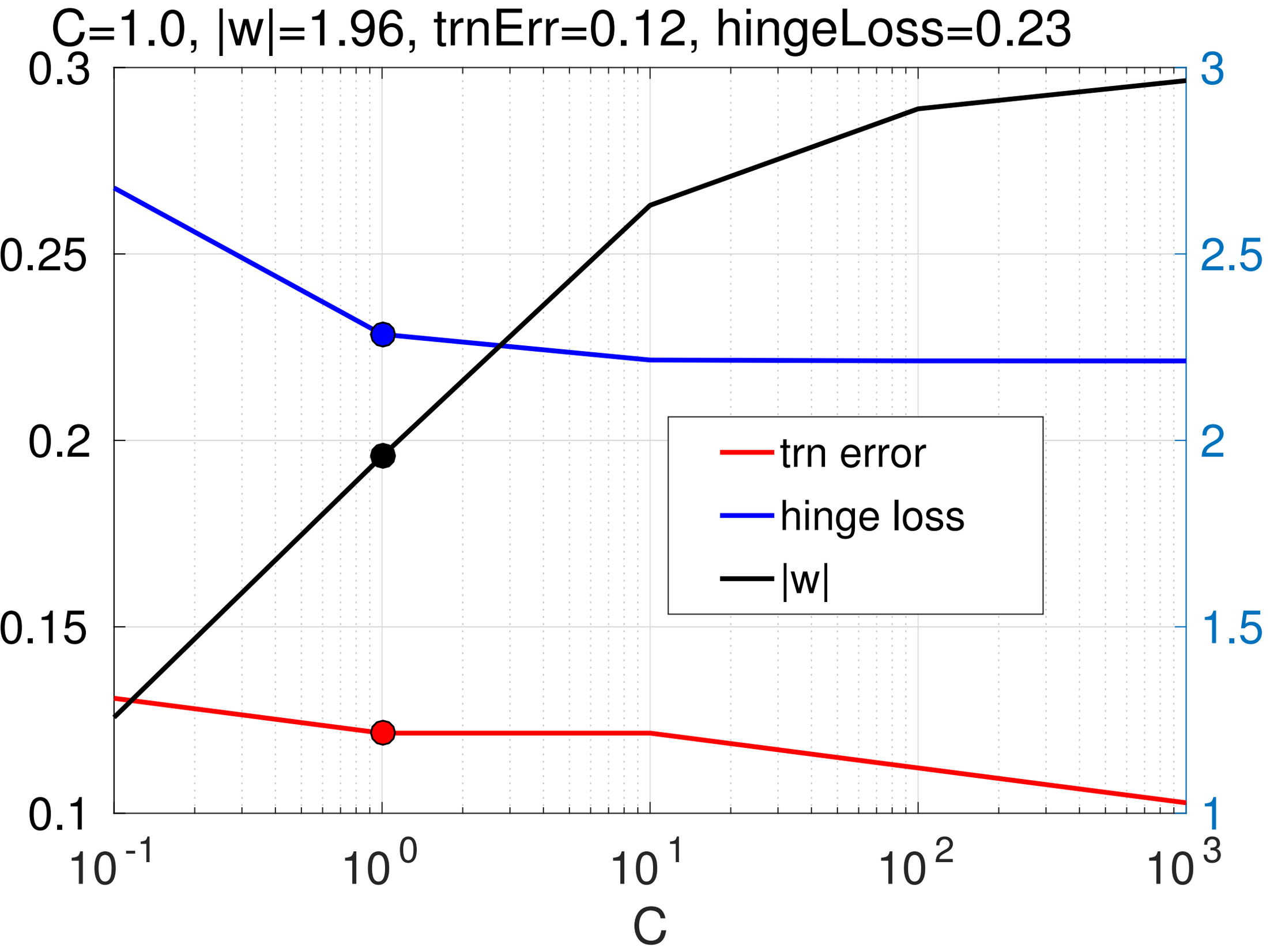

$\max(0, 1-t)$
$[\![t \leq 0]\!]$
1
0
1
t

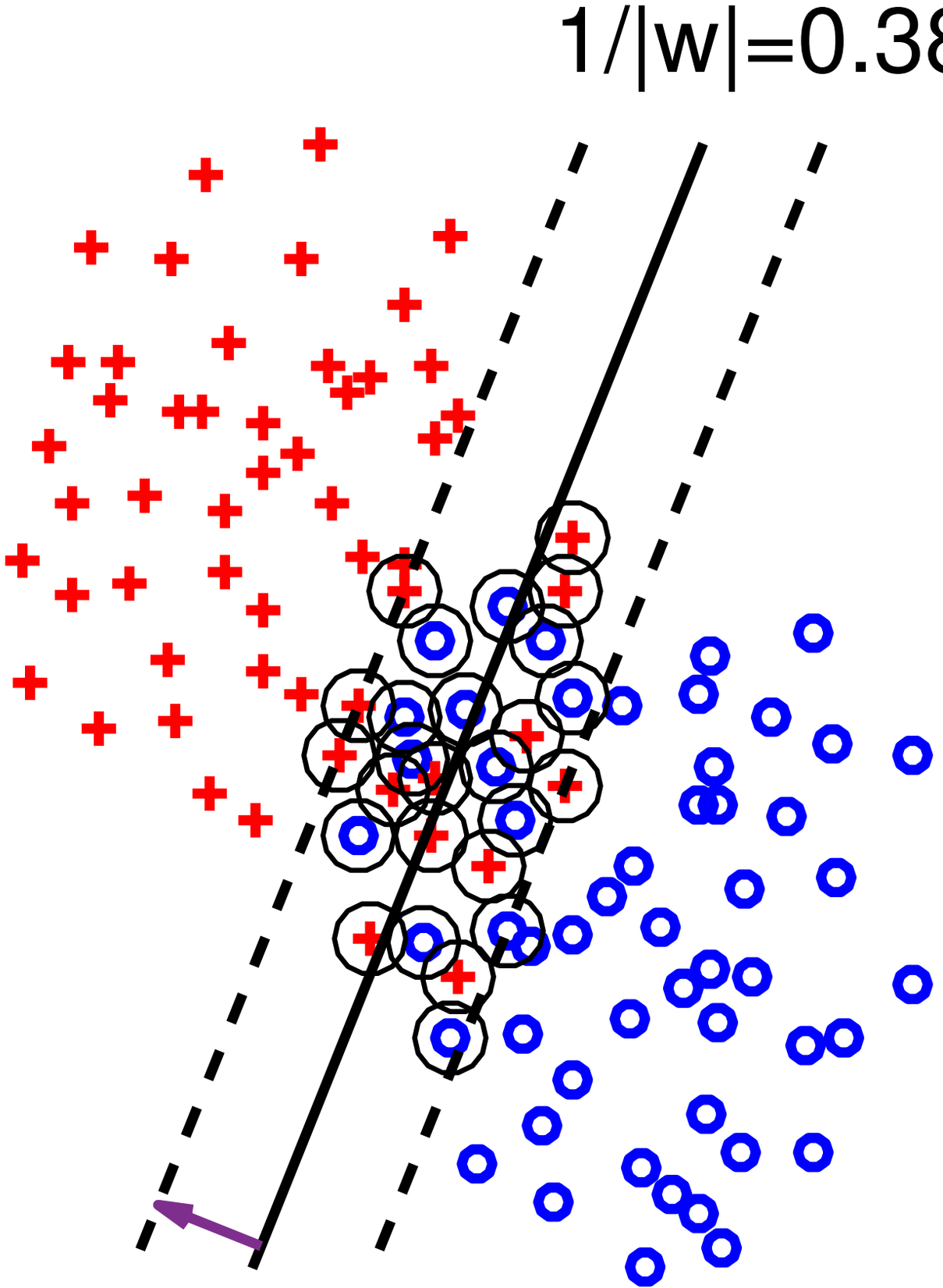


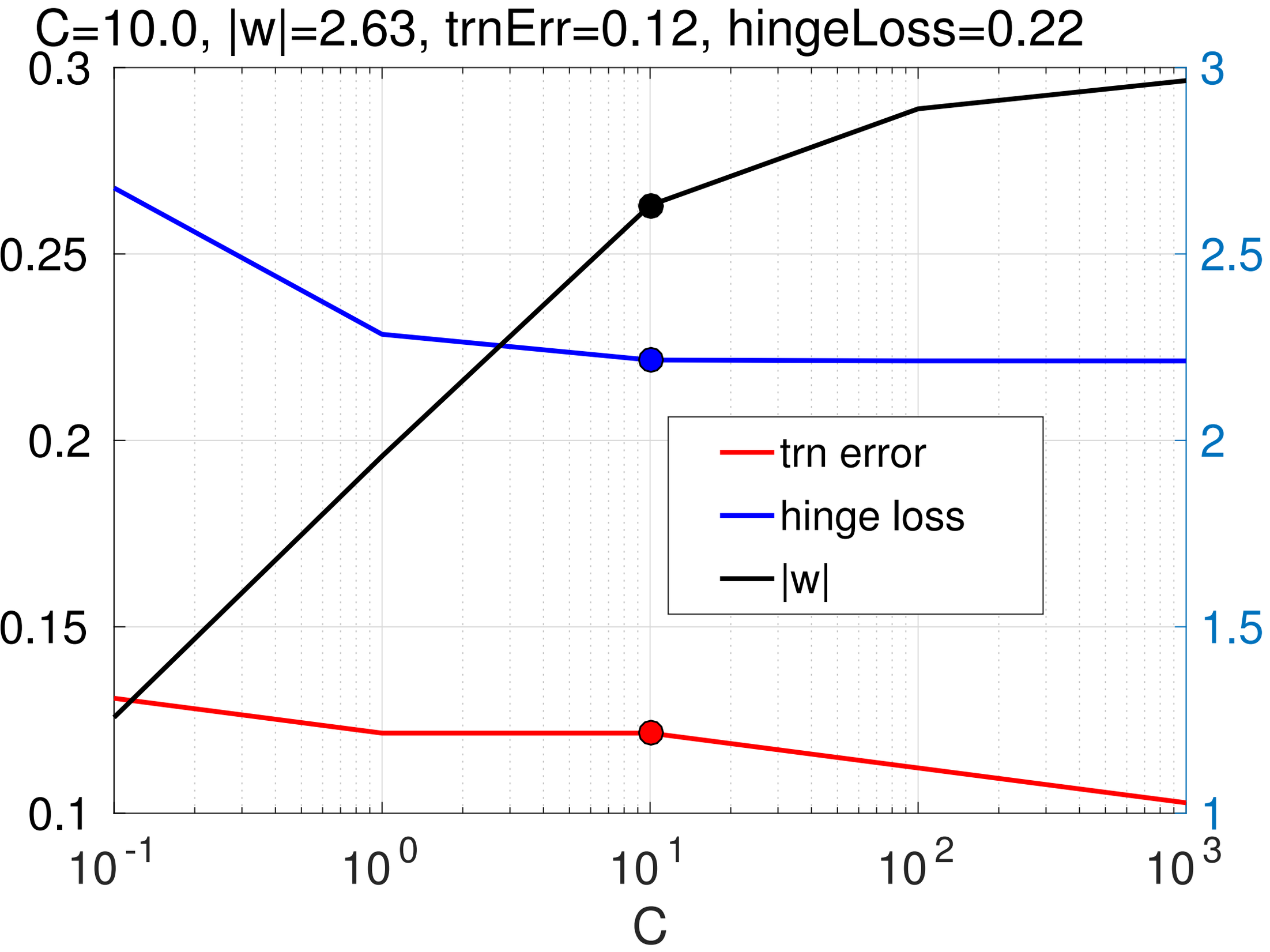

1/|w|=0.80

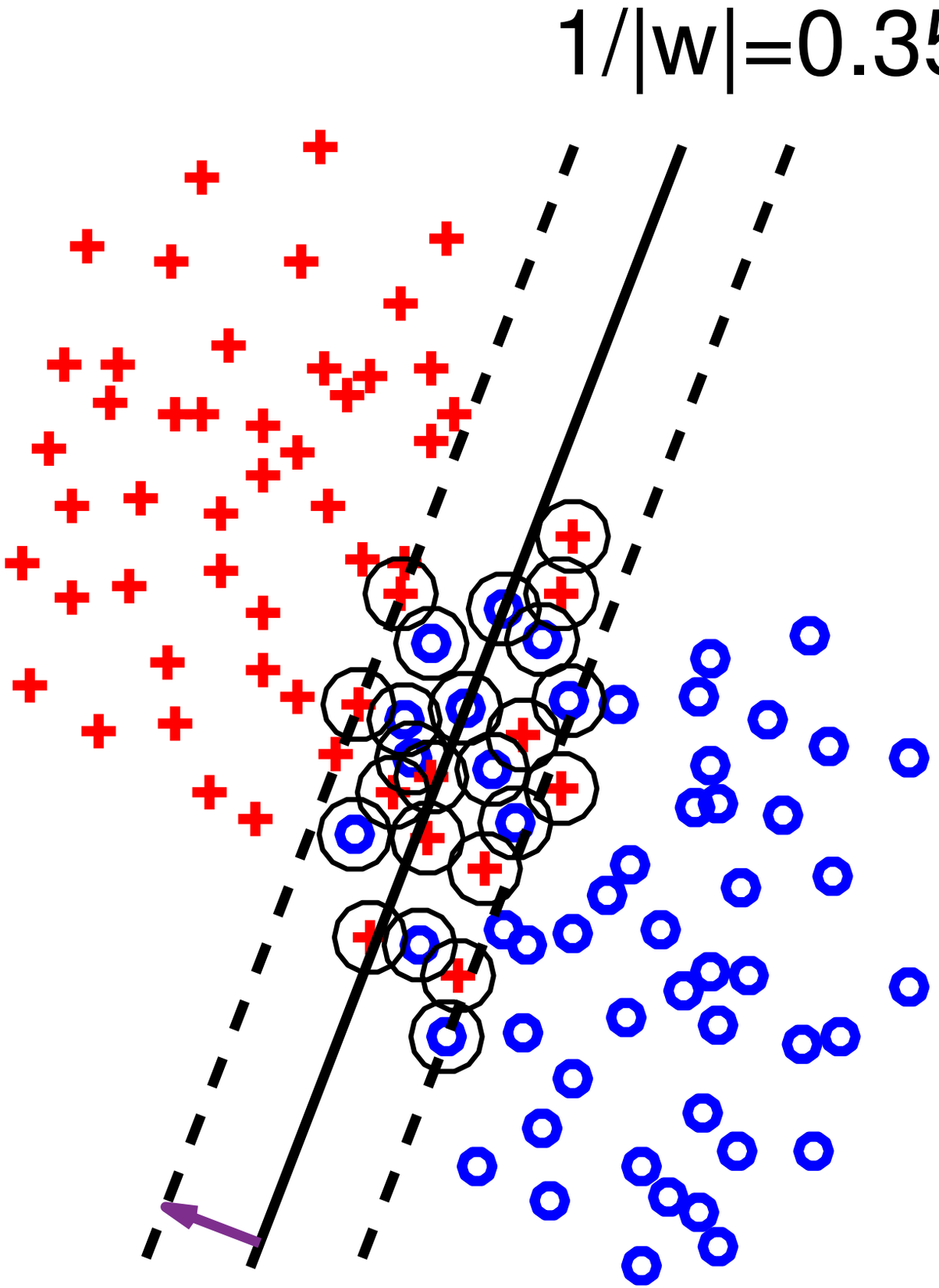

C=0.1, |w|=1.26, trnErr=0.13, hingeLoss=0.27
0.3
0.25
0.2
0.15
0.1
3
2.5
2
1.5
1
trn error
hinge loss
|w|
10 -1
10 0
10 1
10 2
10 3
C

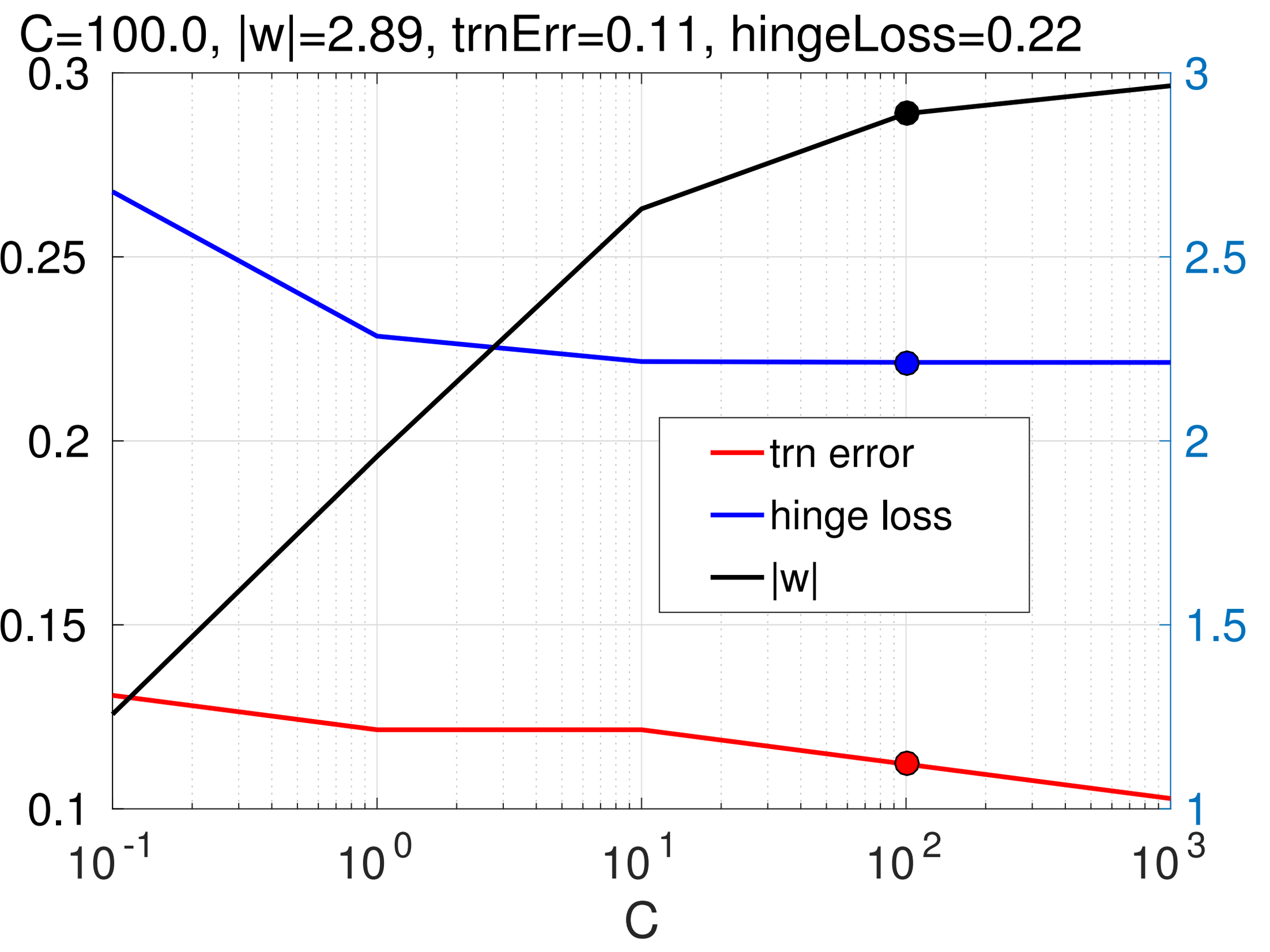

1/|w|=0.51

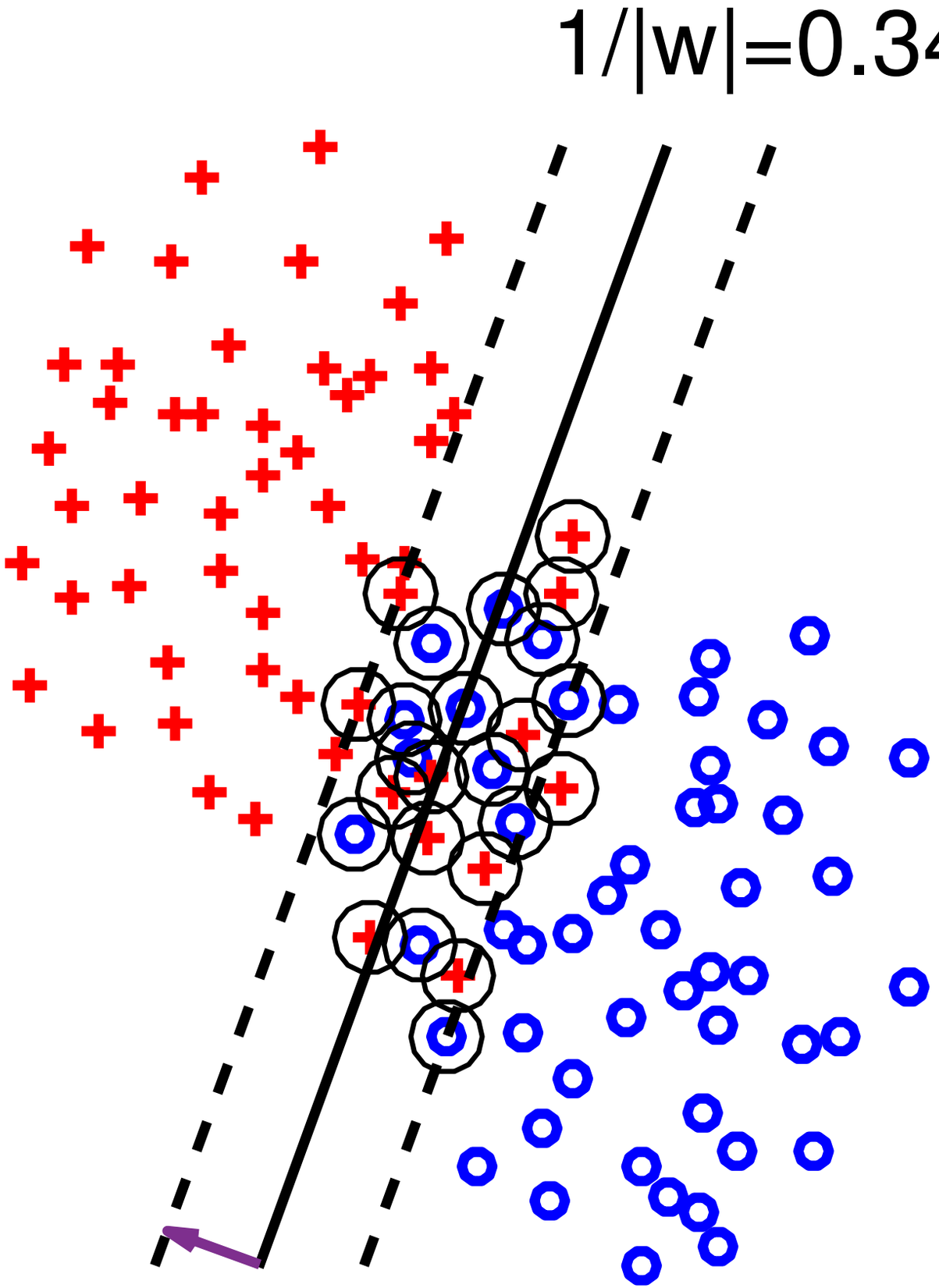

C=1.0, |w|=1.96, trnErr=0.12, hingeLoss=0.23
0.3
0.25
0.2
0.15
0.1
3
2.5
2
1.5
1
trn error
hinge loss
|w|
10 -1
10 0
10 1
10 2
10 3
C

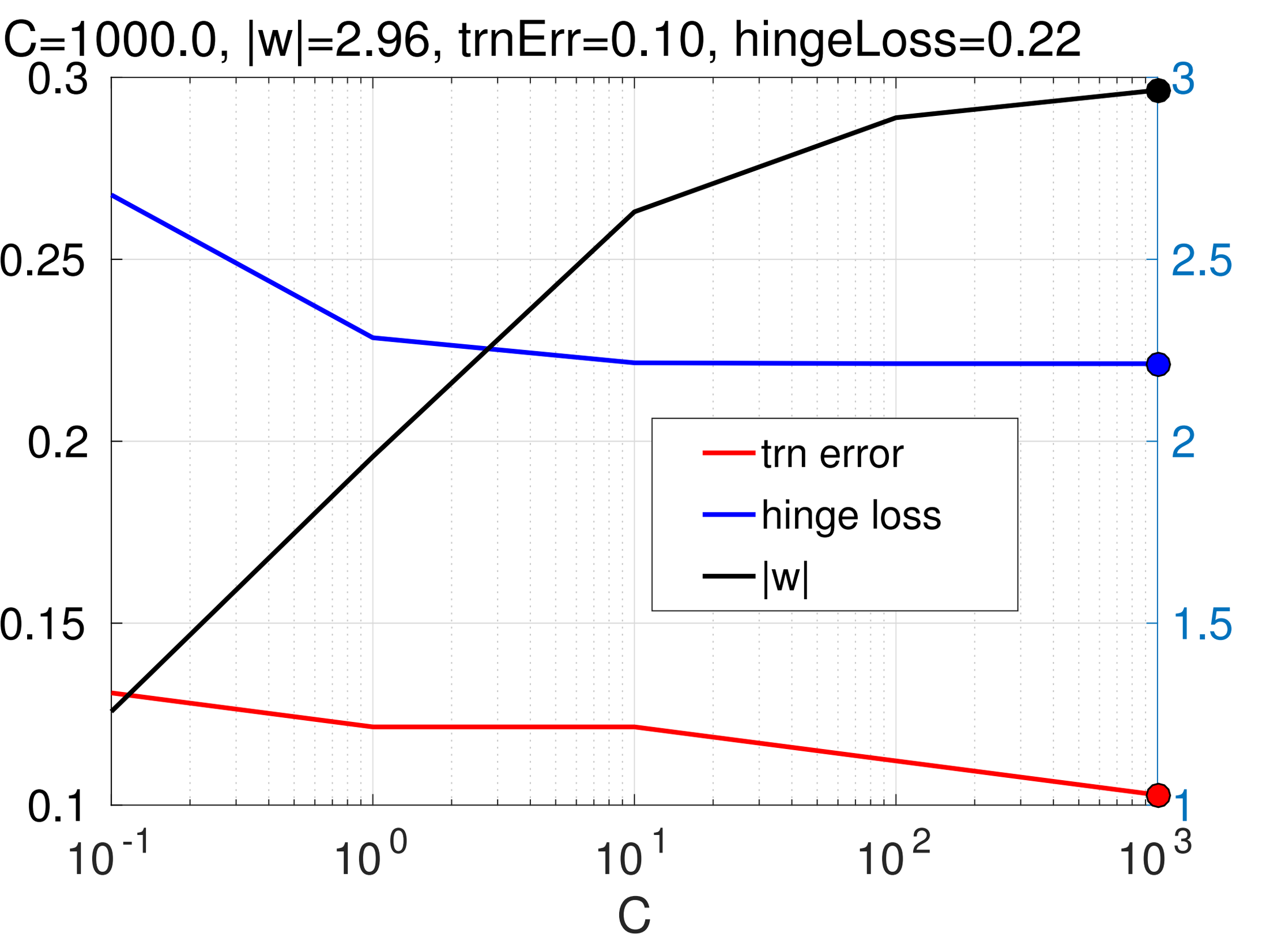

1/|w|=0.38

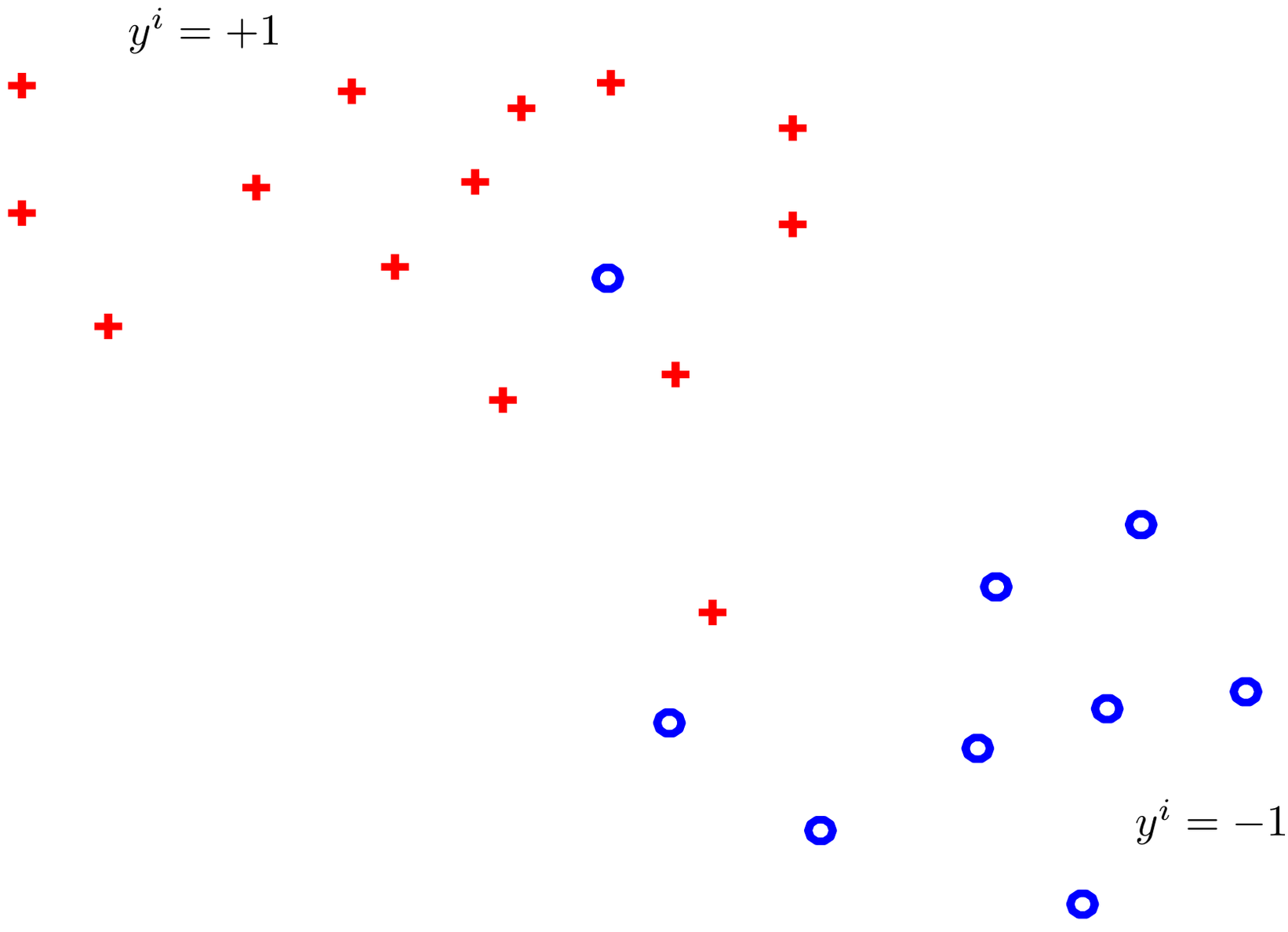

C=10.0, |w|=2.63, trnErr=0.12, hingeLoss=0.22
0.3
0.25
0.2
0.15
0.1
3
2.5
2
1.5
1
trn error
hinge loss
|w|
10 -1
10 0
10 1
10 2
10 3
C

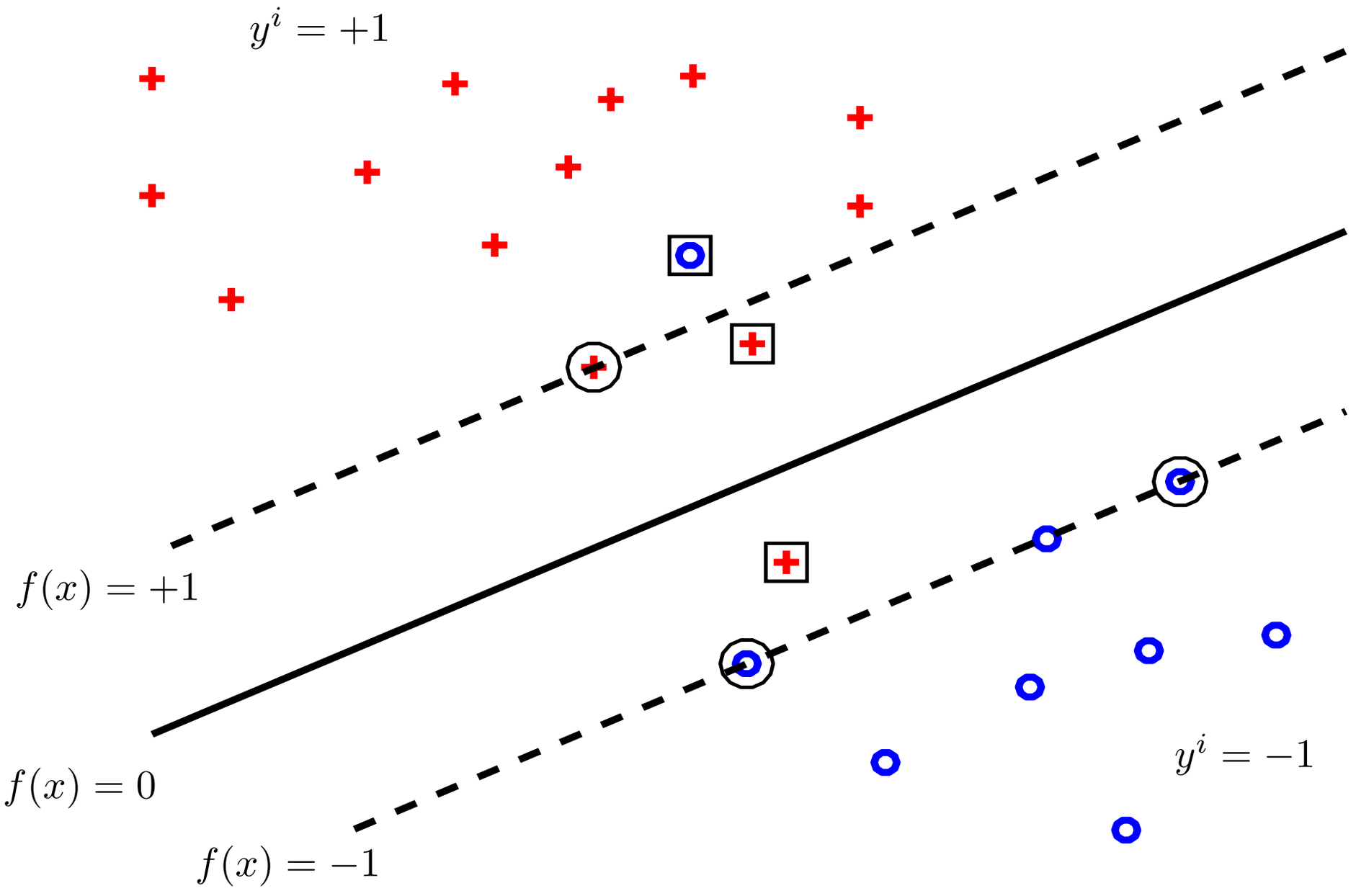

1/|w|=0.35

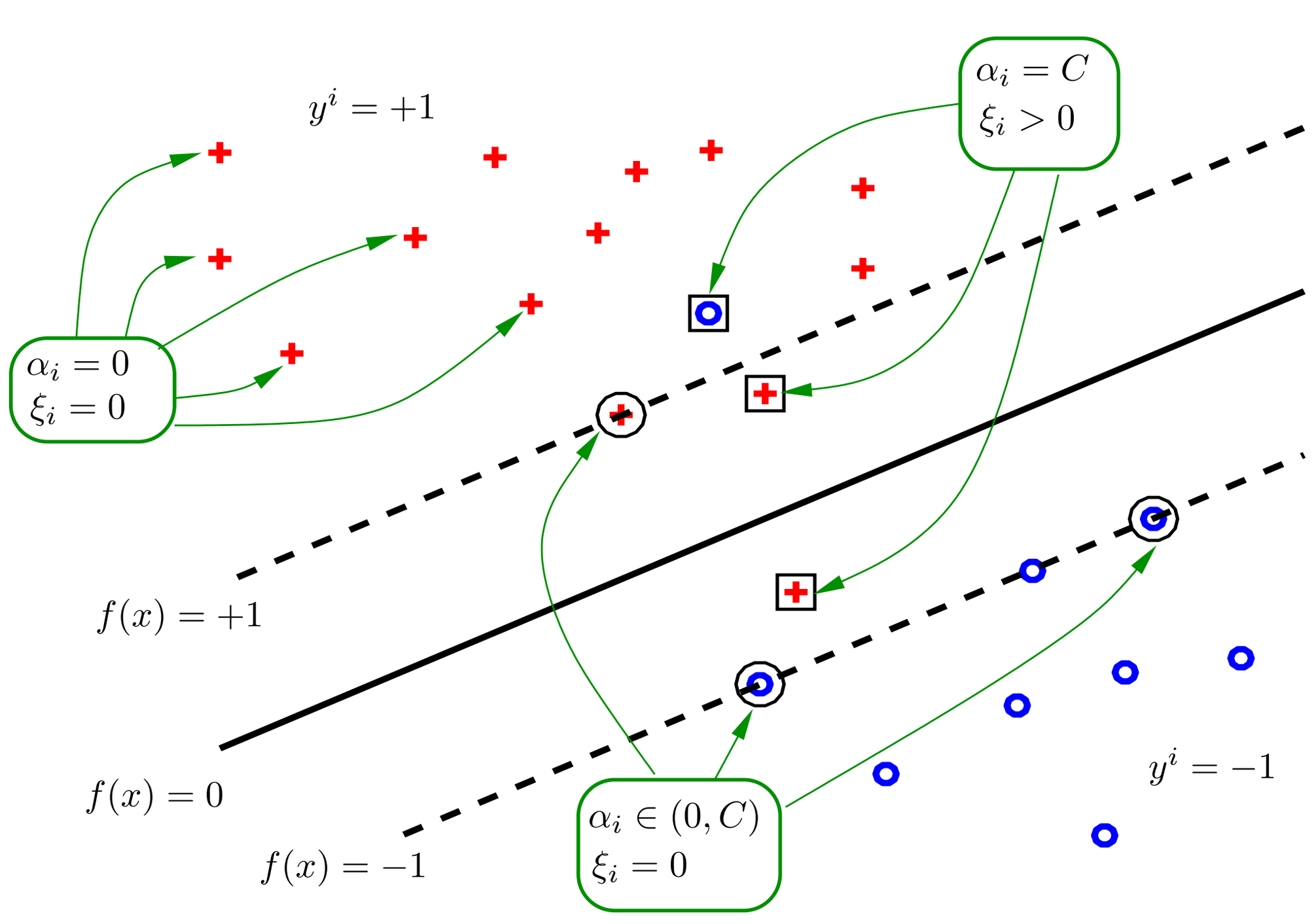

C=100.0, |w|=2.89, trnErr=0.11, hingeLoss=0.22
0.3
0.25
0.2
0.15
0.1
3
2.5
2
1.5
1
trn error
hinge loss
|w|
10 -1
10 0
10 1
10 2
10 3
C

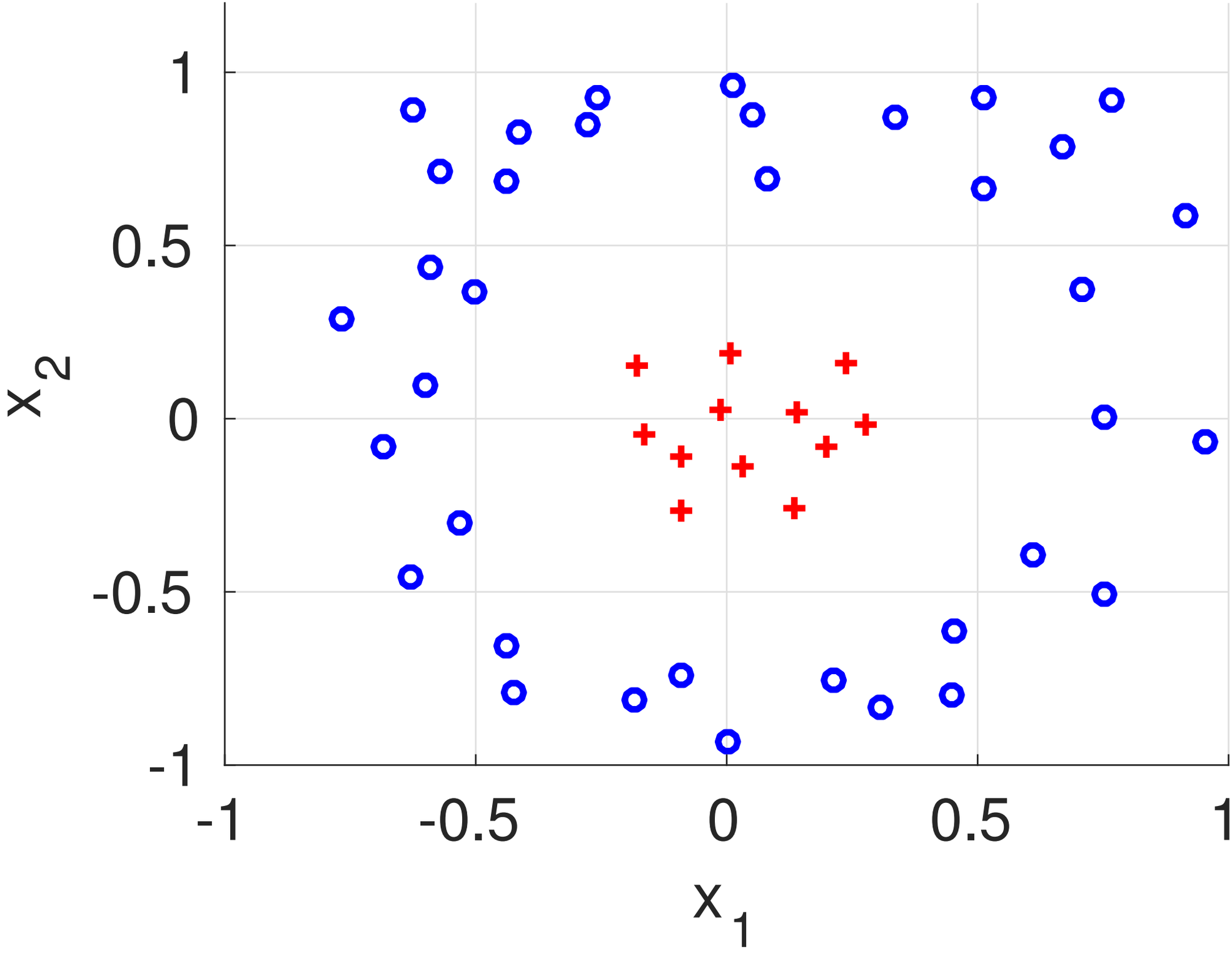

1/|w|=0.34

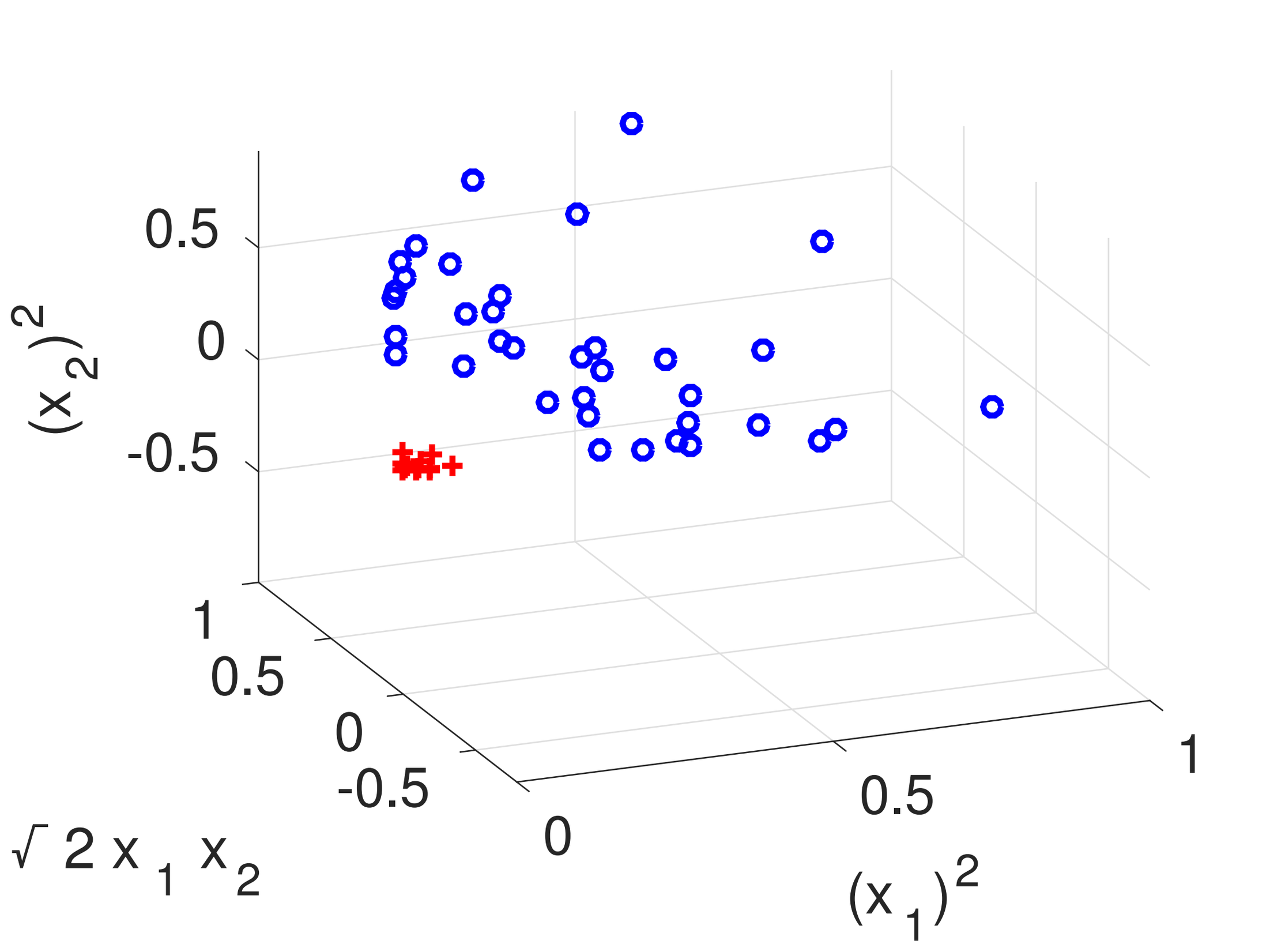

C=1000.0, |w|=2.96, trnErr=0.10, hingeLoss=0.22
0.3
0.25
0.2
0.15
0.1
3
2.5
2
1.5
1
trn error
hinge loss
|w|
10 -1
10 0
10 1
10 2
10 3
C

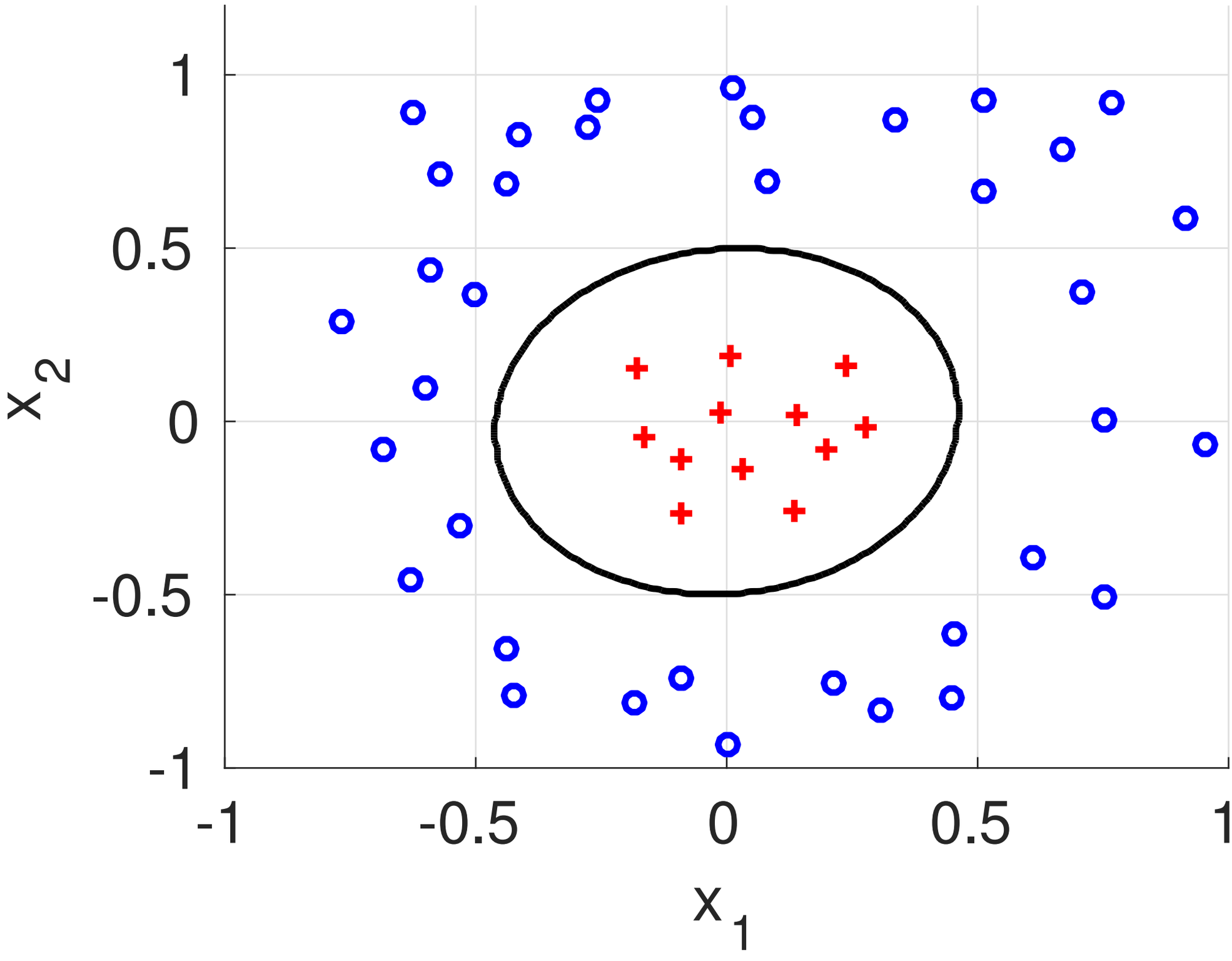

$y^i = +1$
$y^i = -1$

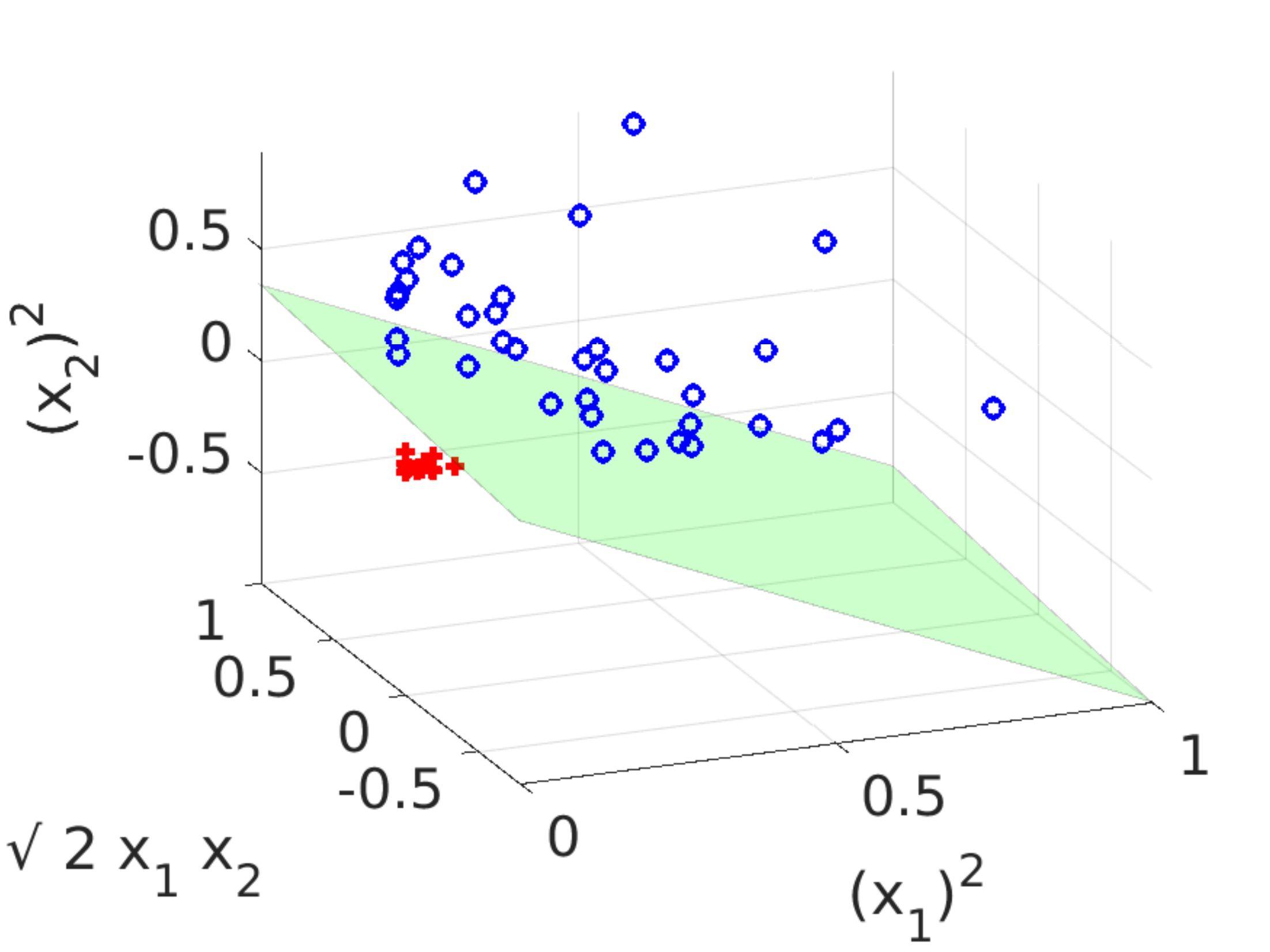

$y^i = +1$
$f(x) = +1$
$f(x) = 0$
$f(x) = -1$
$y^i = -1$

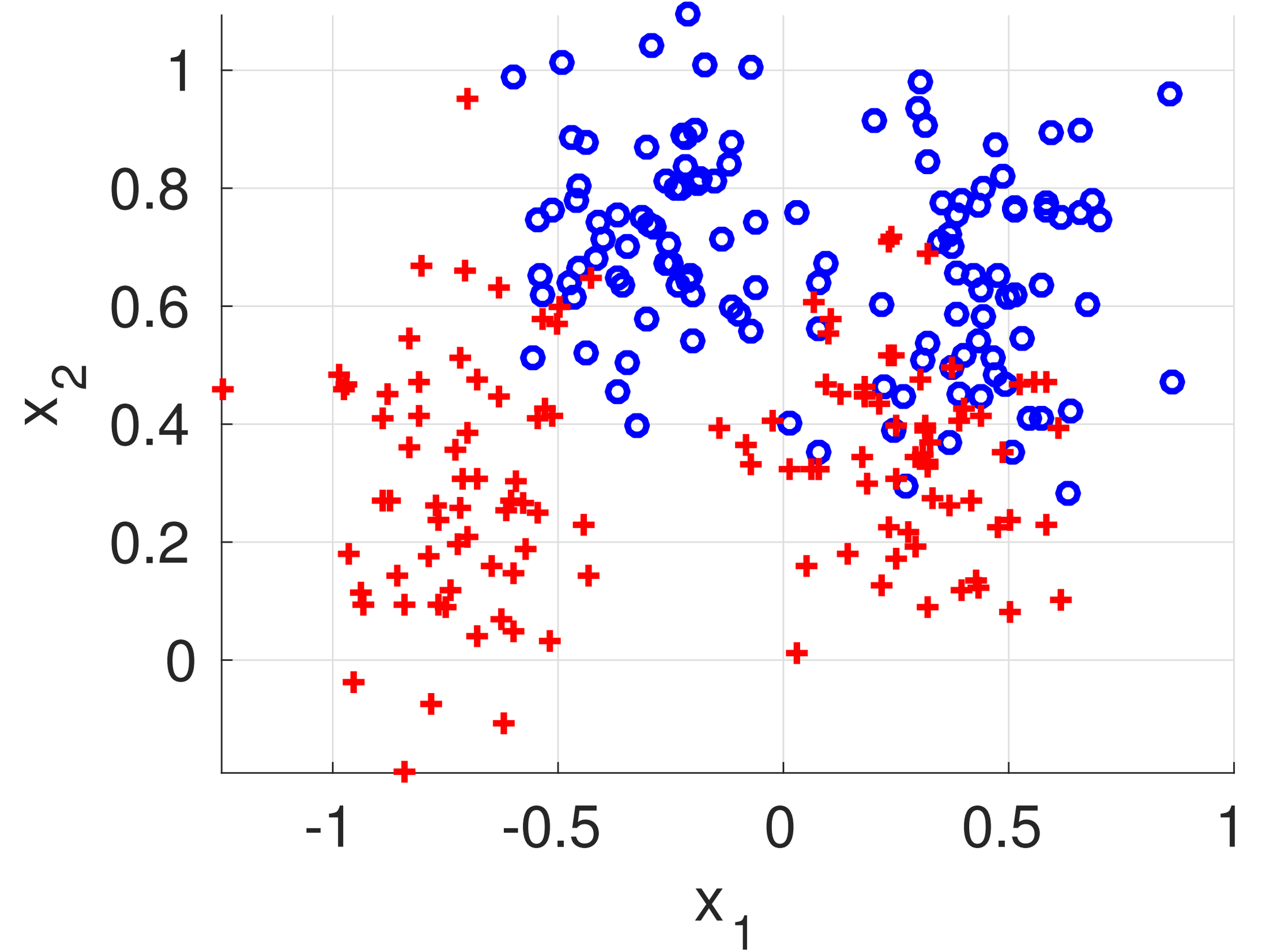

$y^i = +1$
$\alpha_i = C$
$\xi_i > 0$
$\alpha_i = 0$
$\xi_i = 0$
$f(x) = +1$
$f(x) = 0$
$f(x) = -1$
$\alpha_i \in (0, C)$
$\xi_i = 0$
$y^i = -1$

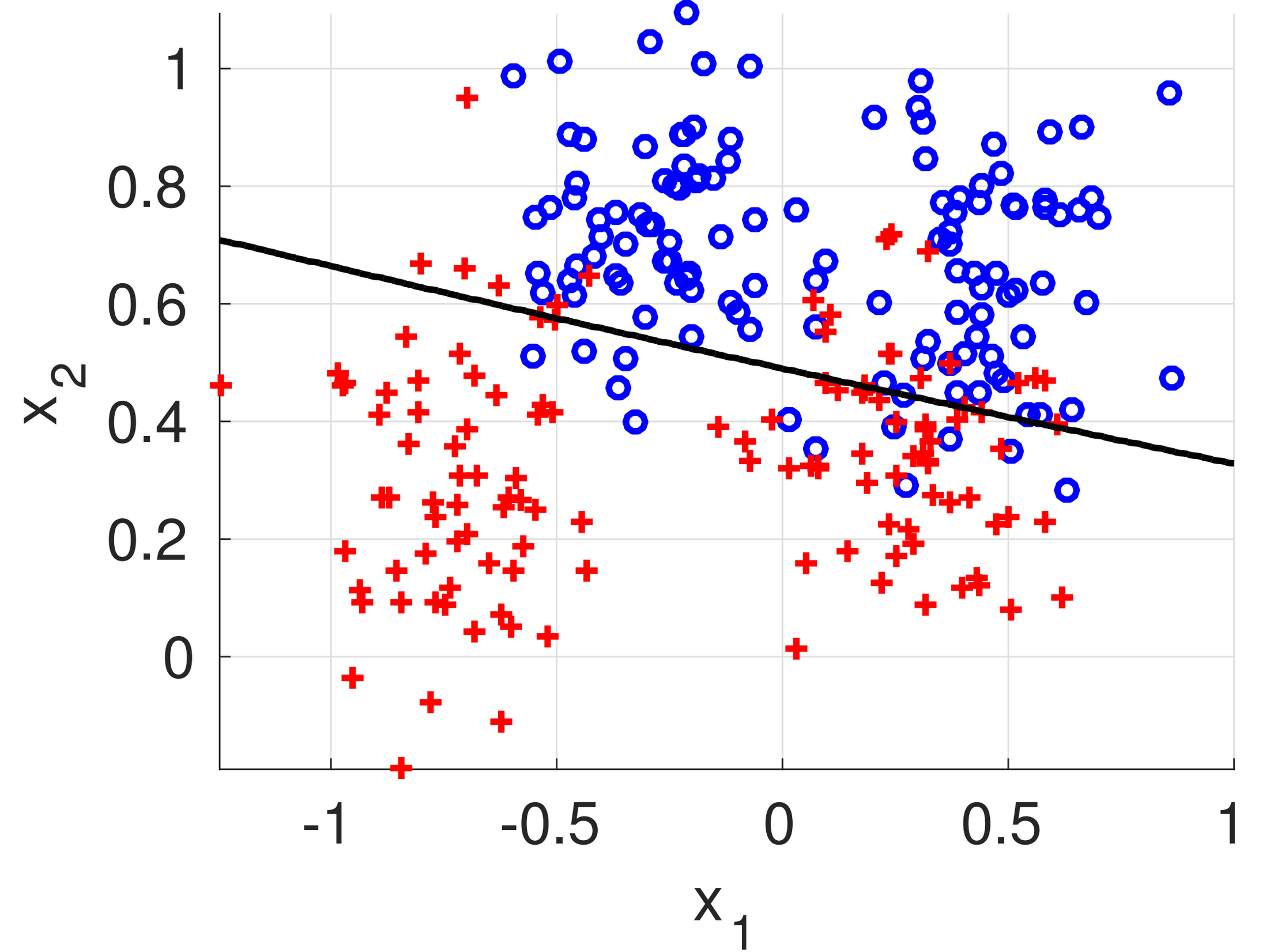

1
0.5
0
-0.5
-1
x_2
-1
-0.5
0
0.5
1
x_1

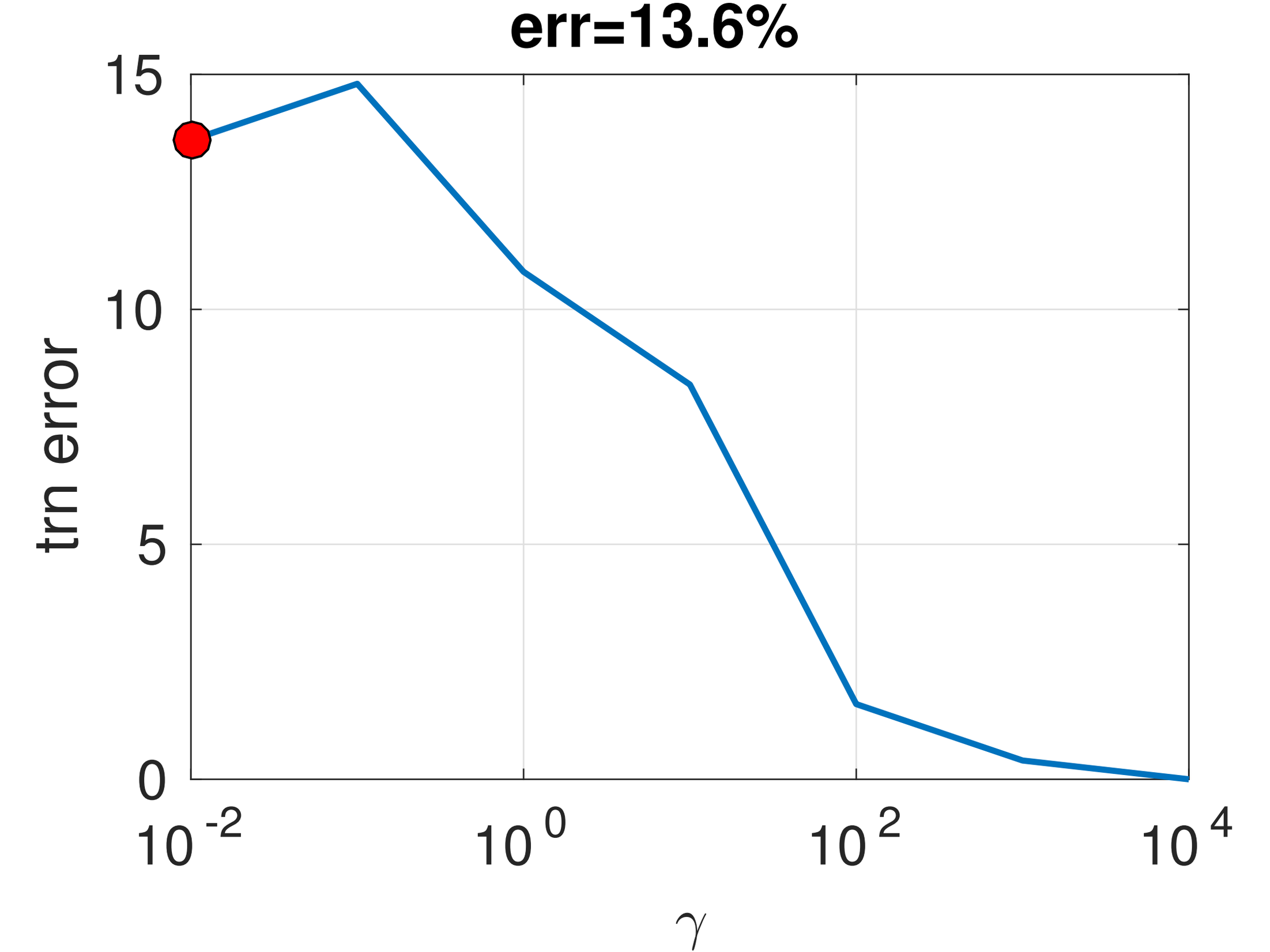

0.5
0
-0.5
$(x_2)^2$
1
0.5
0
-0.5
$\sqrt{2}\,x_1\,x_2$
0
0.5
1
$(x_1)^2$

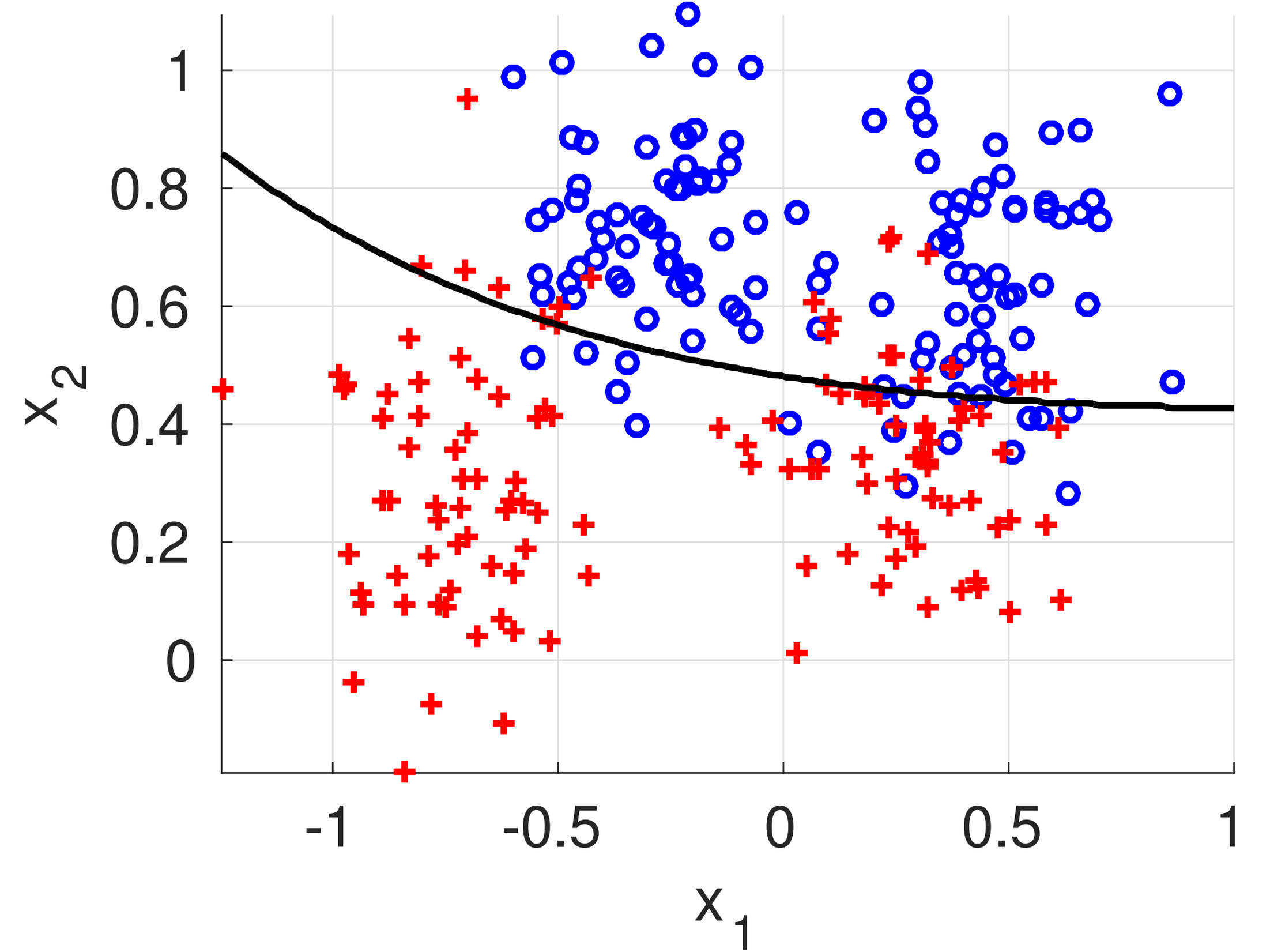

1
0.5
0
-0.5
-1
-1
-0.5
0
0.5
1
x2
x1

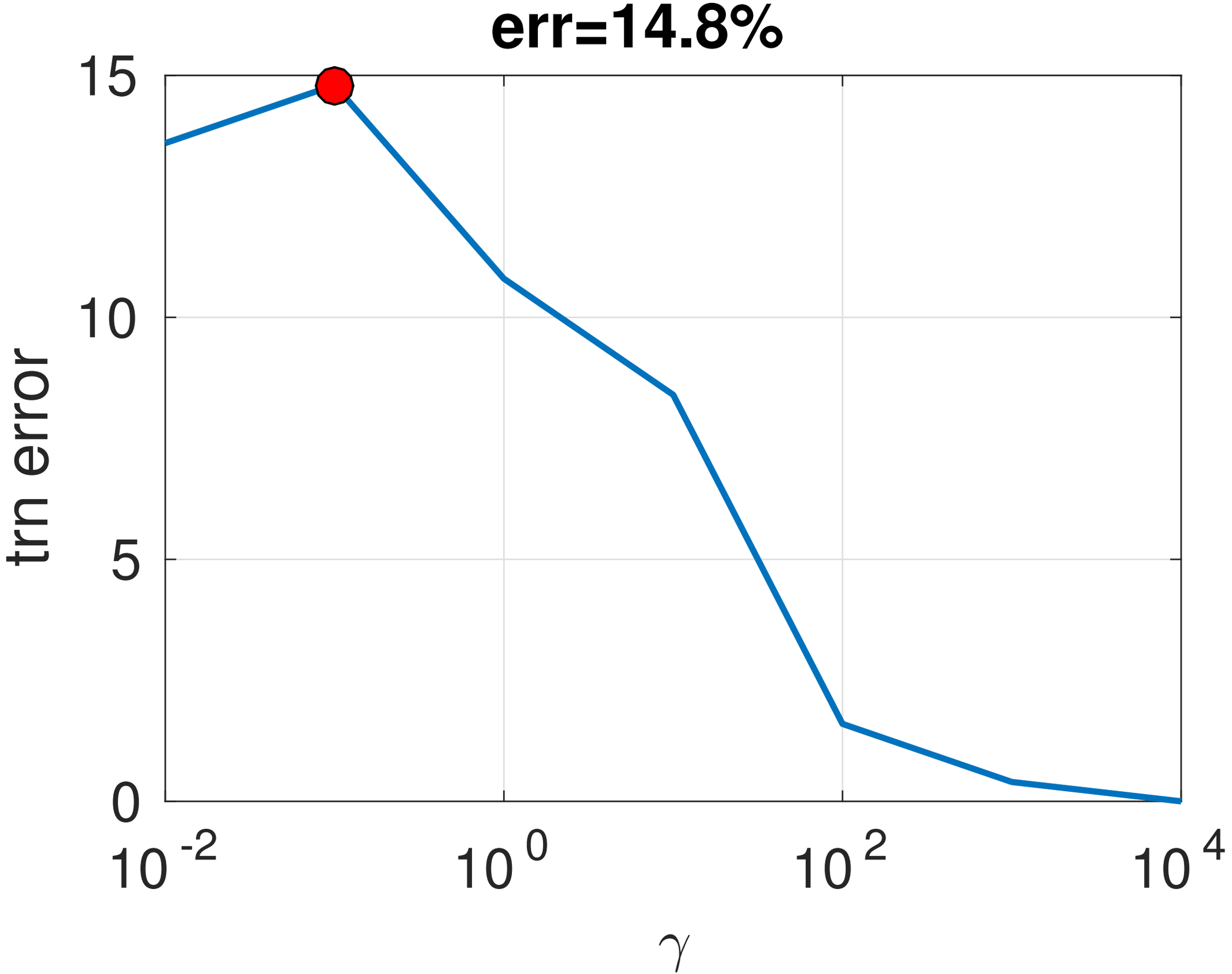

0.5
0
-0.5
$(x_2)^2$
1
0.5
0
-0.5
$\sqrt{2}\, x_1\, x_2$
0
0.5
1
$(x_1)^2$

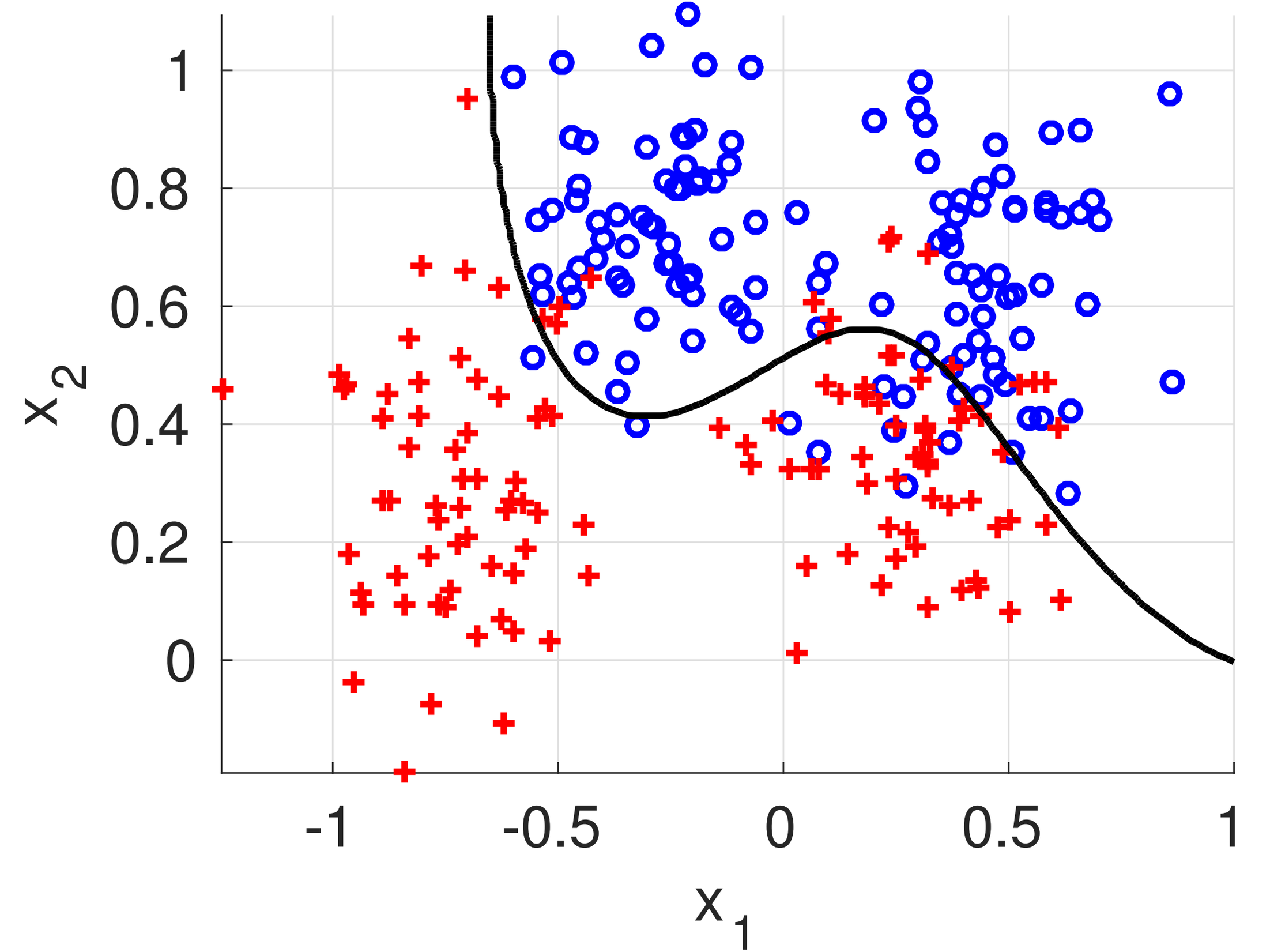

1
0.8
0.6
0.4
0.2
0
x_2
-1
-0.5
0
0.5
1
x_1

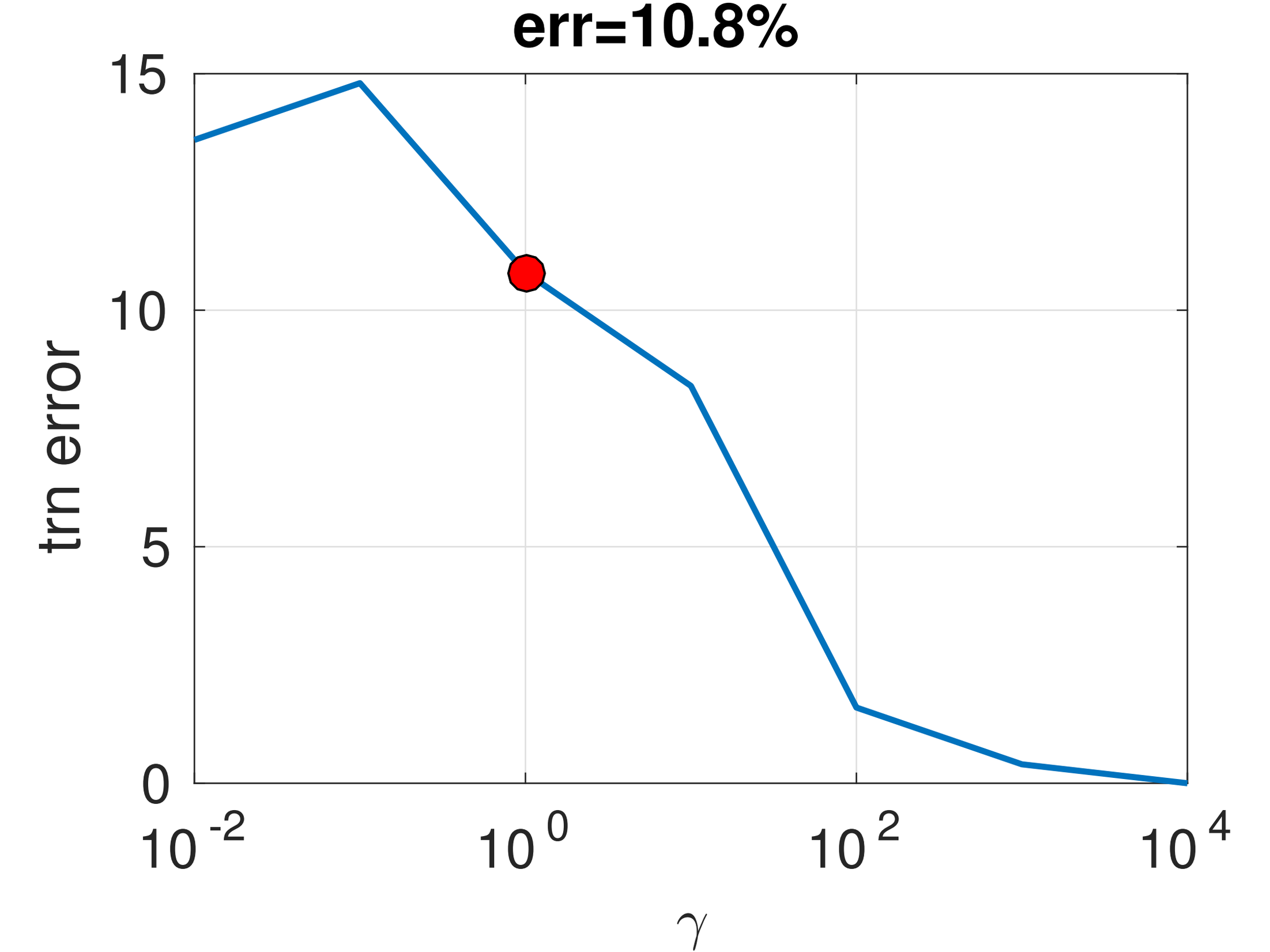

1
0.8
0.6
0.4
0.2
0
x_2
-1
-0.5
0
0.5
1
x_1

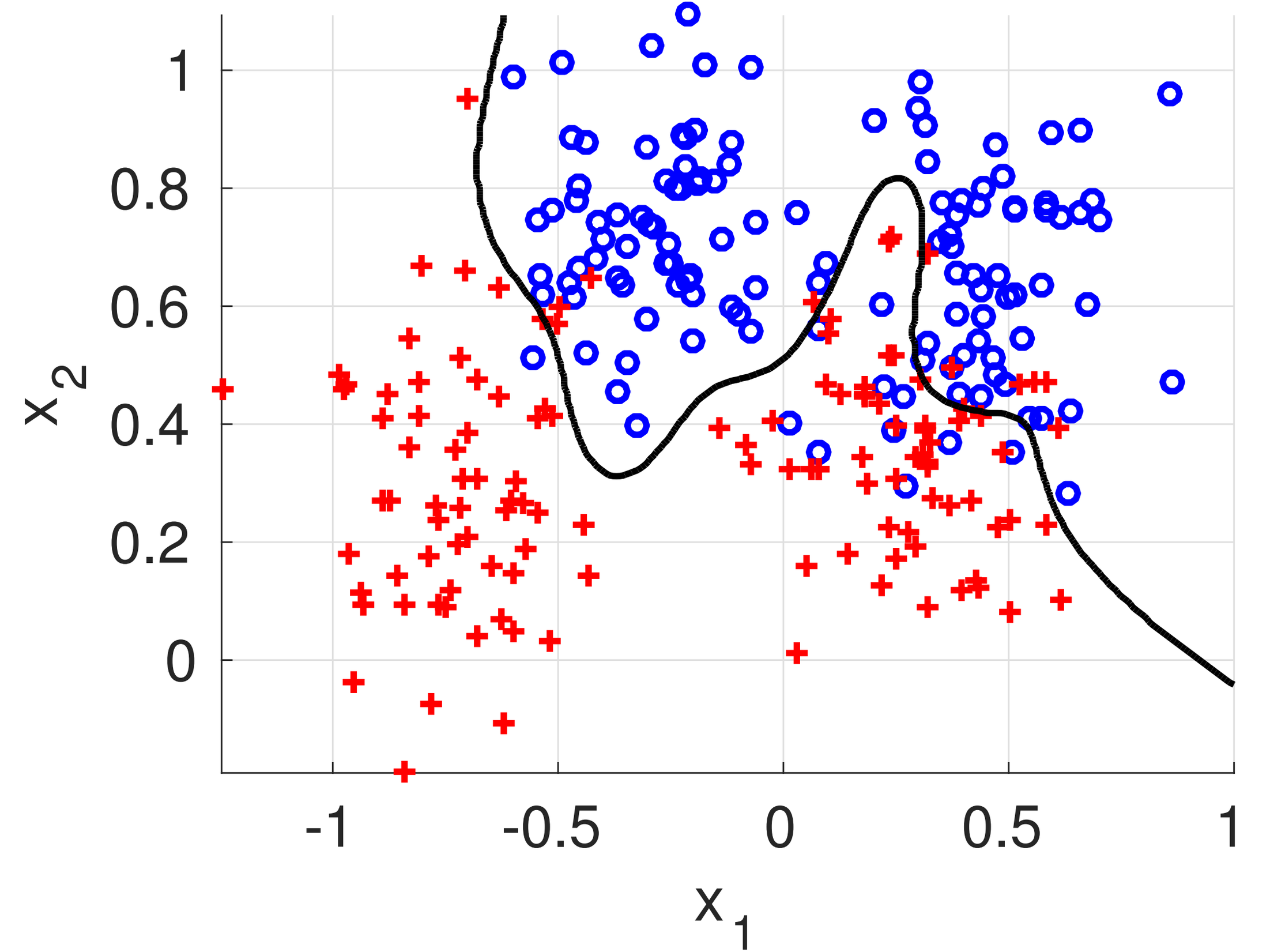

err=13.6%
15
10
5
0
trn error
10^{-2}
10^{0}
10^{2}
10^{4}
γ

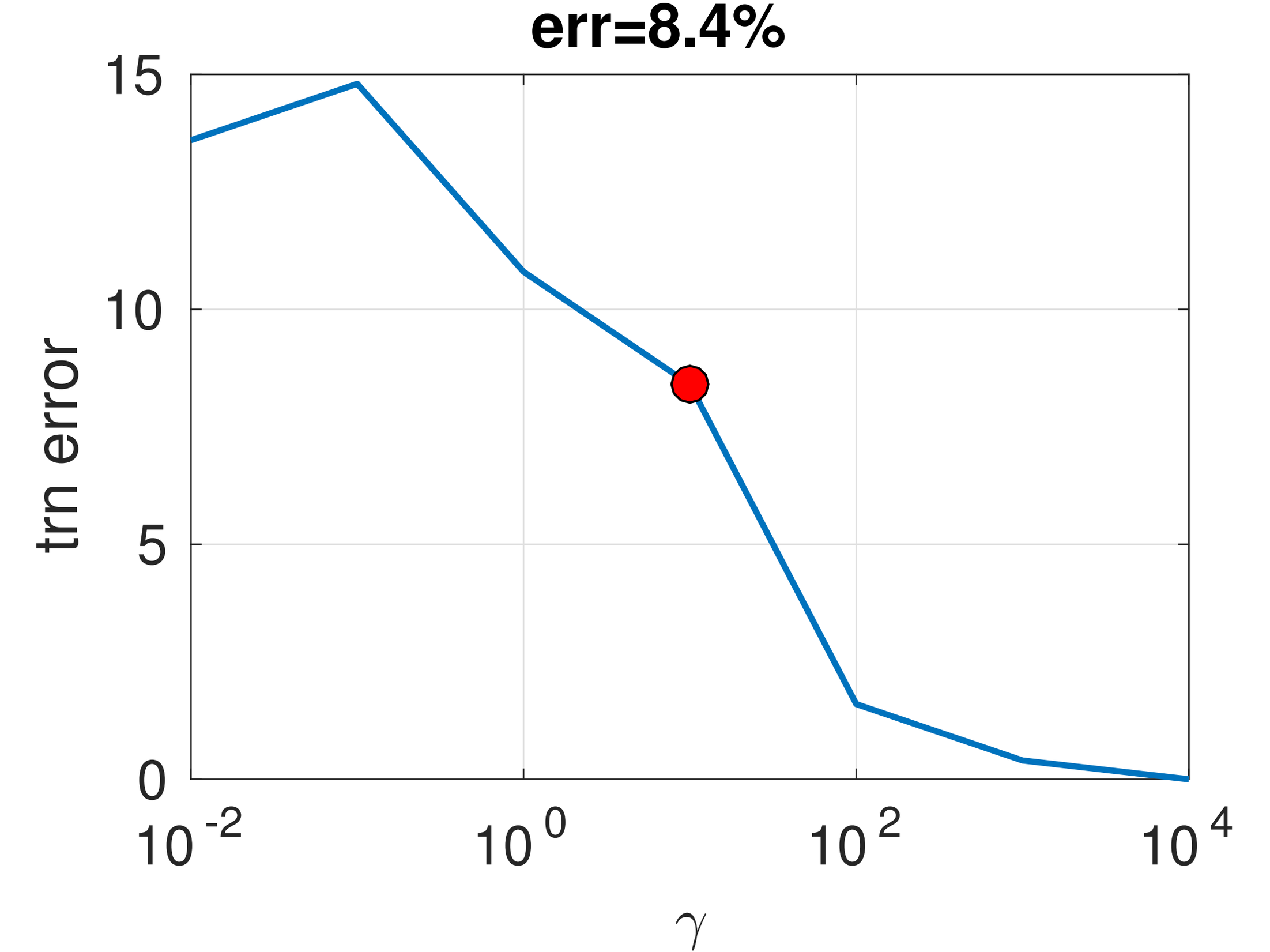

1
0.8
0.6
0.4
0.2
0
x_2
-1
-0.5
0
0.5
1
x_1

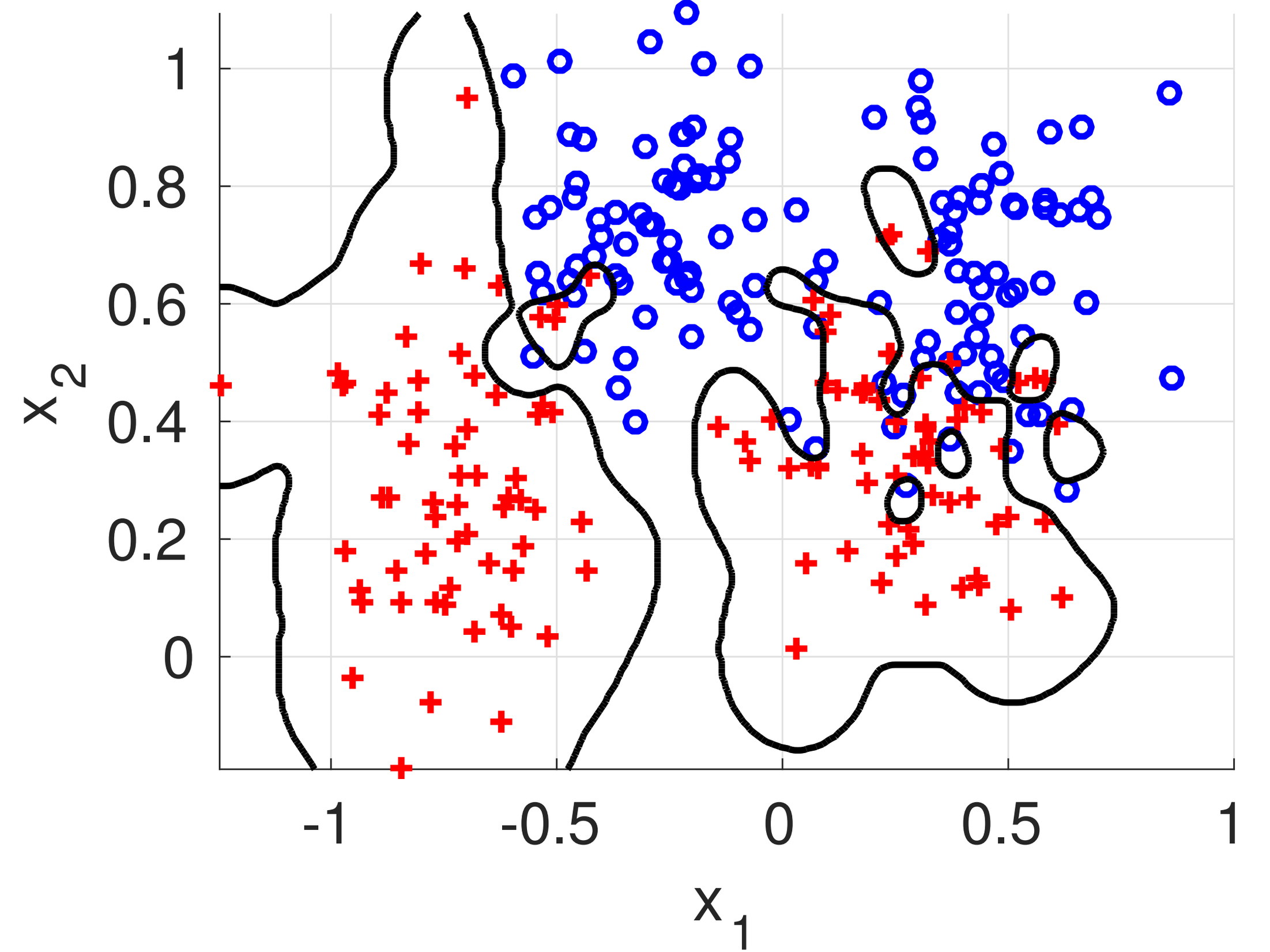

err=14.8%
15
10
5
0
trn error
10 -2
10 0
10 2
10 4
γ

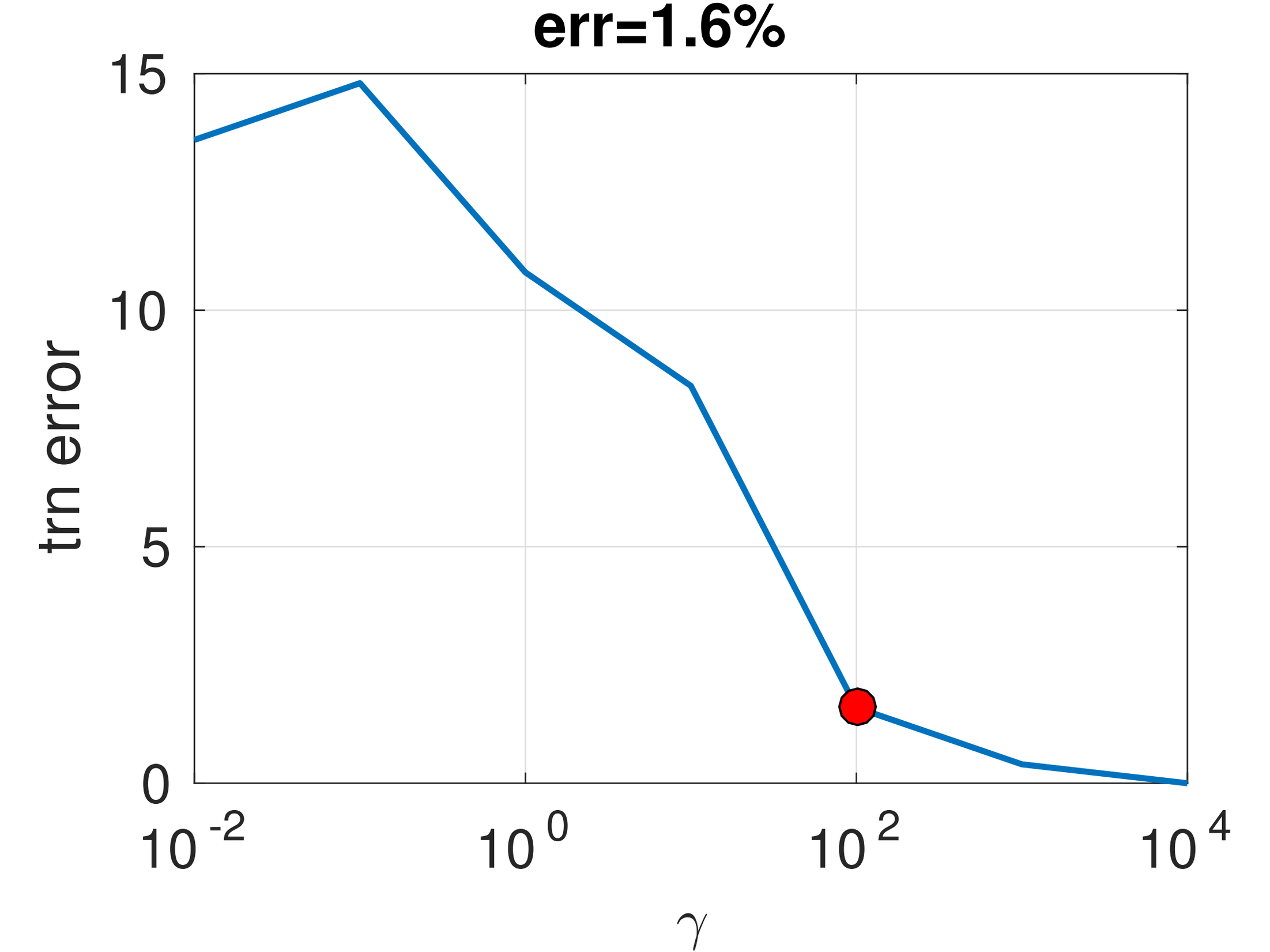

1
0.8
0.6
0.4
0.2
0
x_2
-1
-0.5
0
0.5
1
x_1

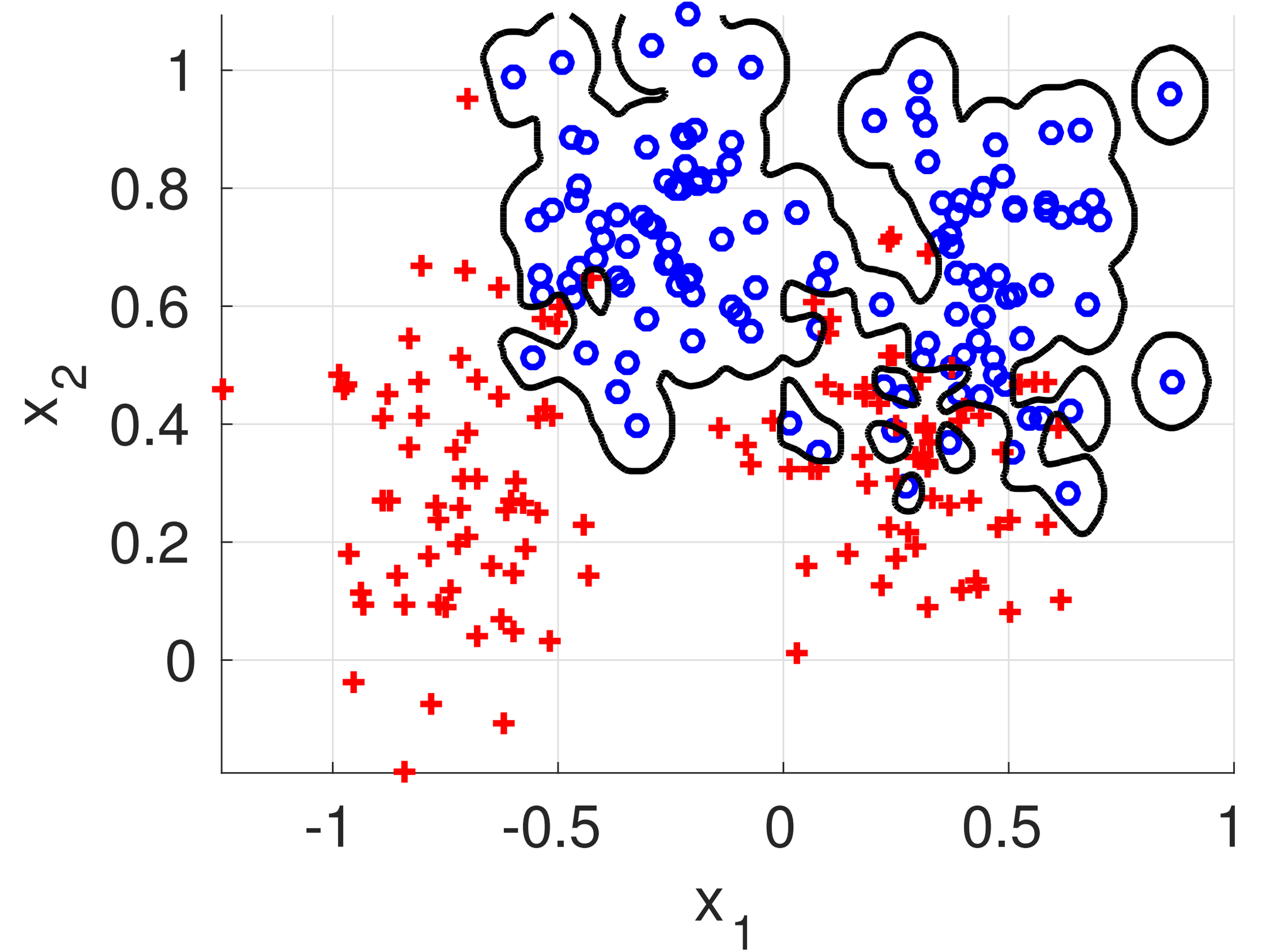

err=10.8%
15
10
5
0
trn error
10 -2
10 0
10 2
10 4
γ

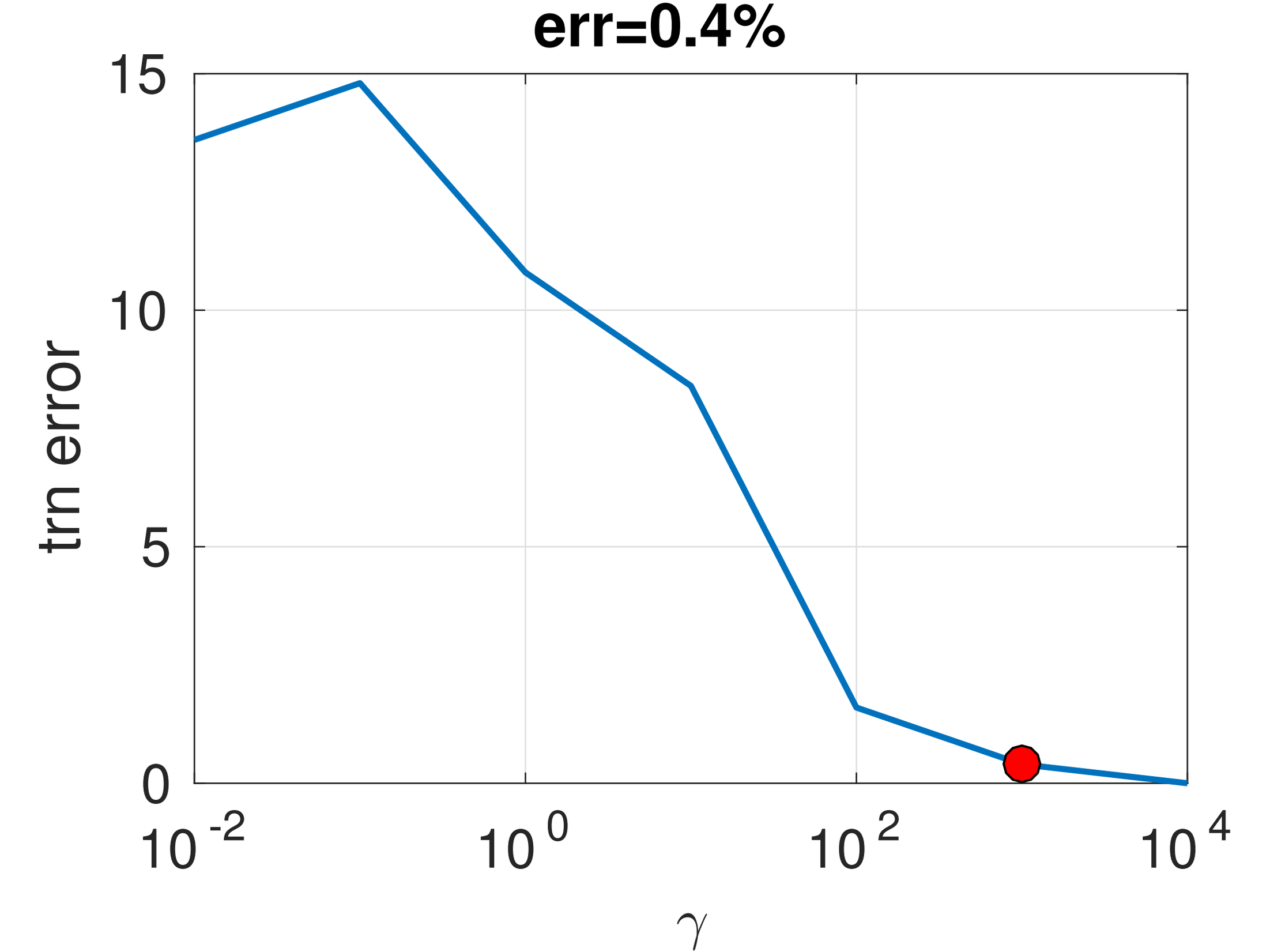

1
0.8
0.6
0.4
0.2
0
x_2
-1
-0.5
0
0.5
1
x_1

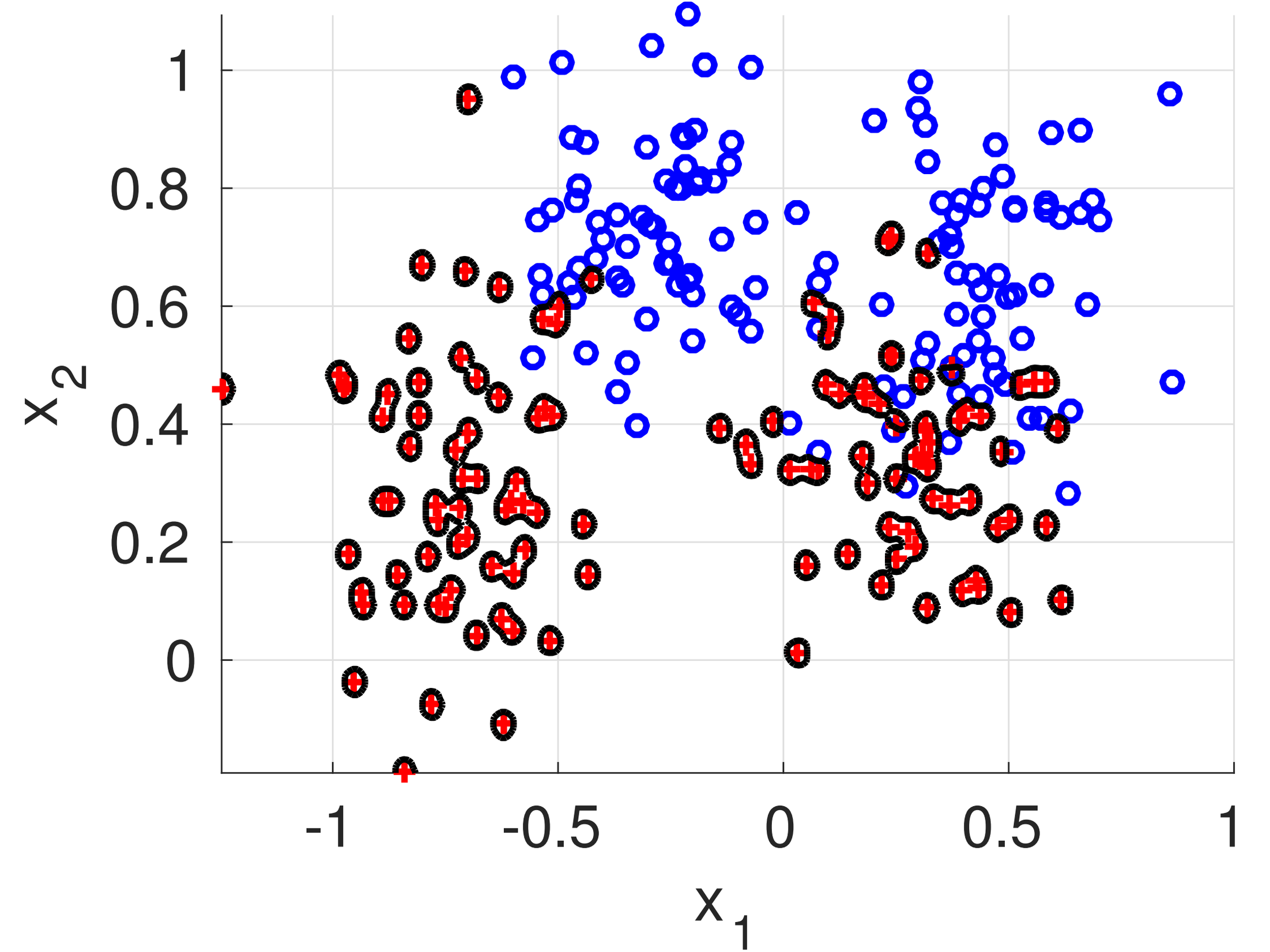

err=8.4%
15
10
5
0
trn error
10^{-2}
10^{0}
10^{2}
10^{4}
γ

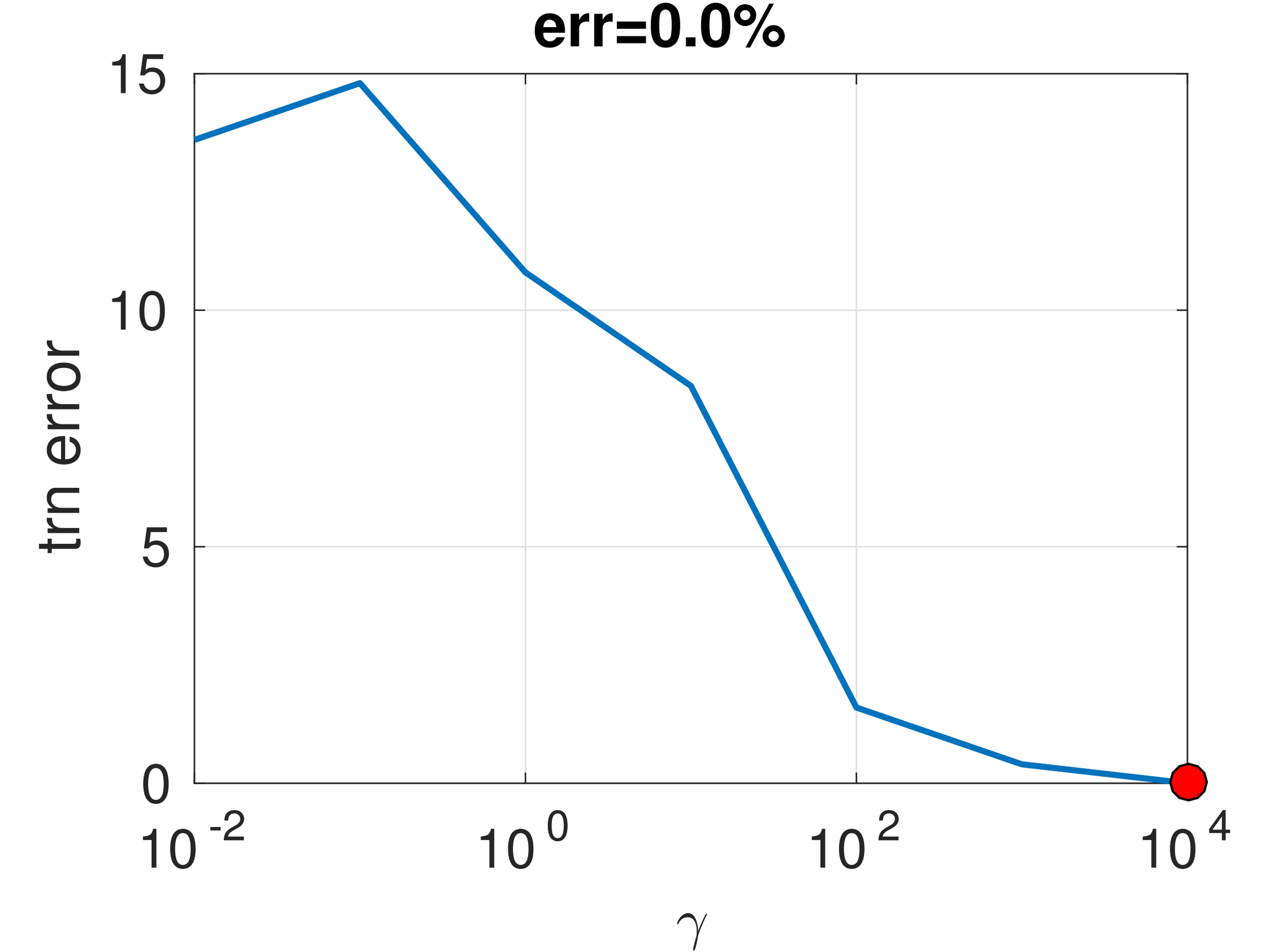

1
0.8
0.6
0.4
0.2
0
x_2
-1
-0.5
0
0.5
1
x_1

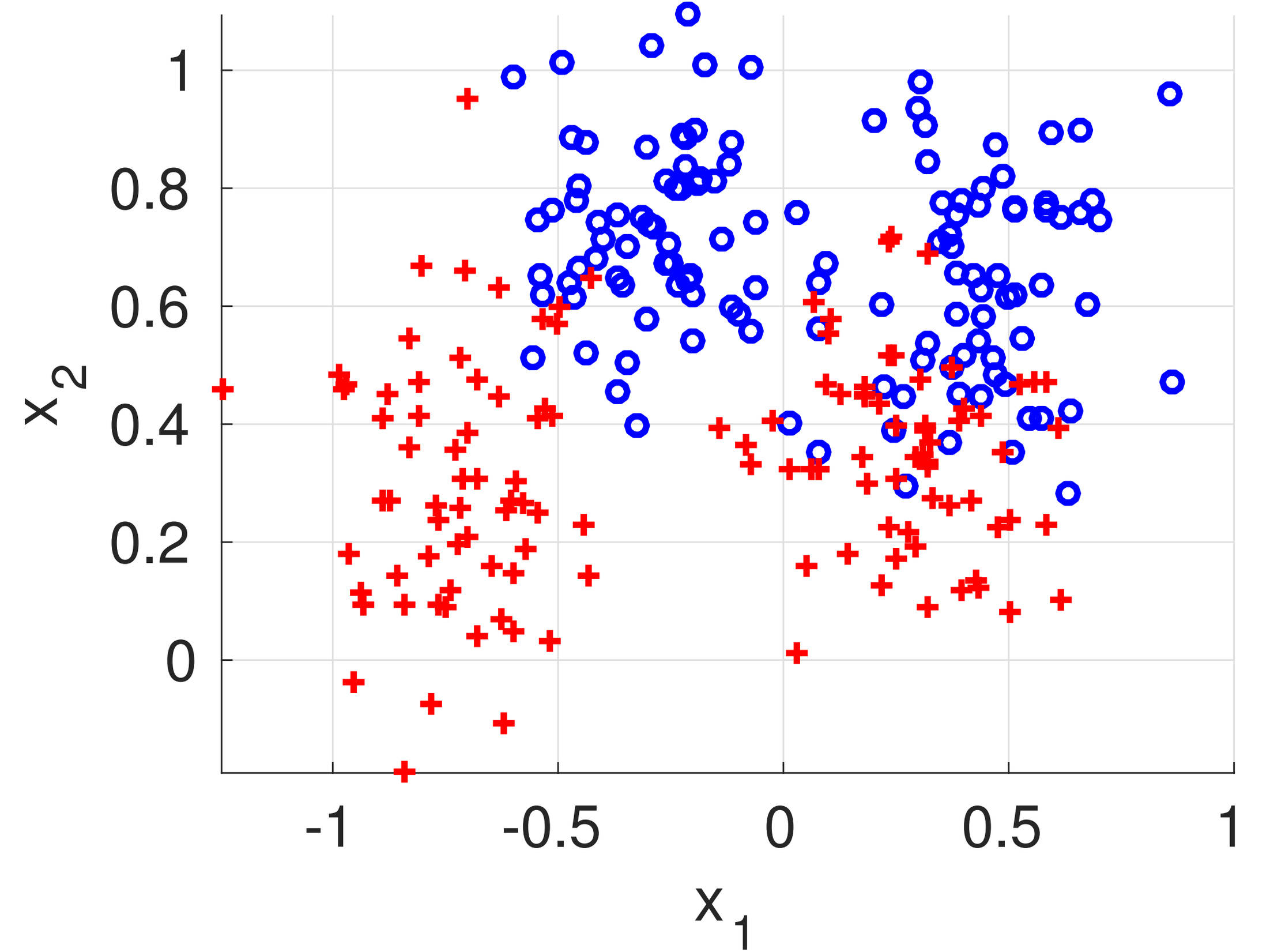

err=1.6%
15
10
5
0
trn error
10 -2
10 0
10 2
10 4
γ

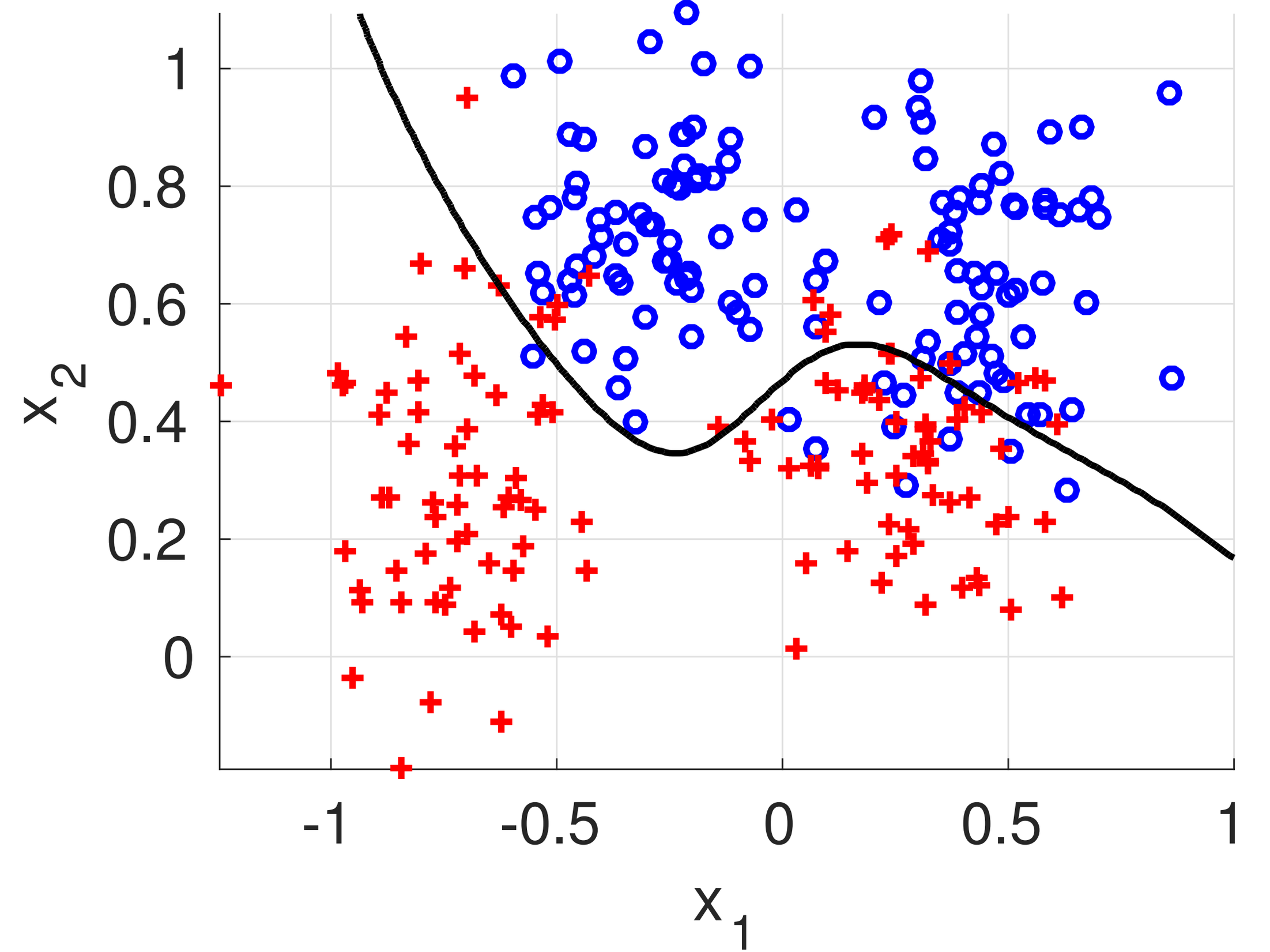

1
0.8
0.6
0.4
0.2
0
x_2
-1
-0.5
0
0.5
1
x_1

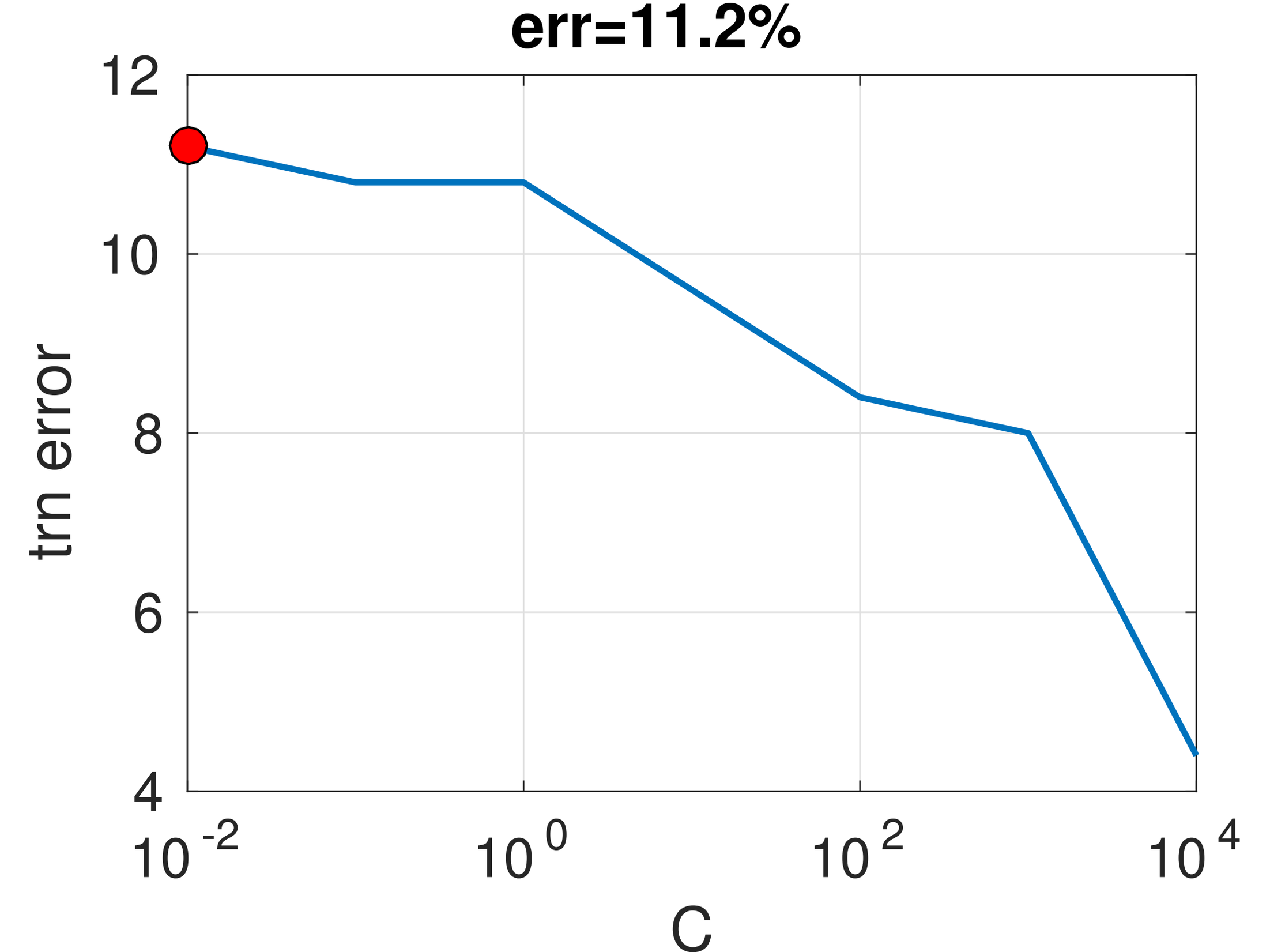

err=0.4%
15
10
5
0
trn error
10 -2
10 0
10 2
10 4
γ

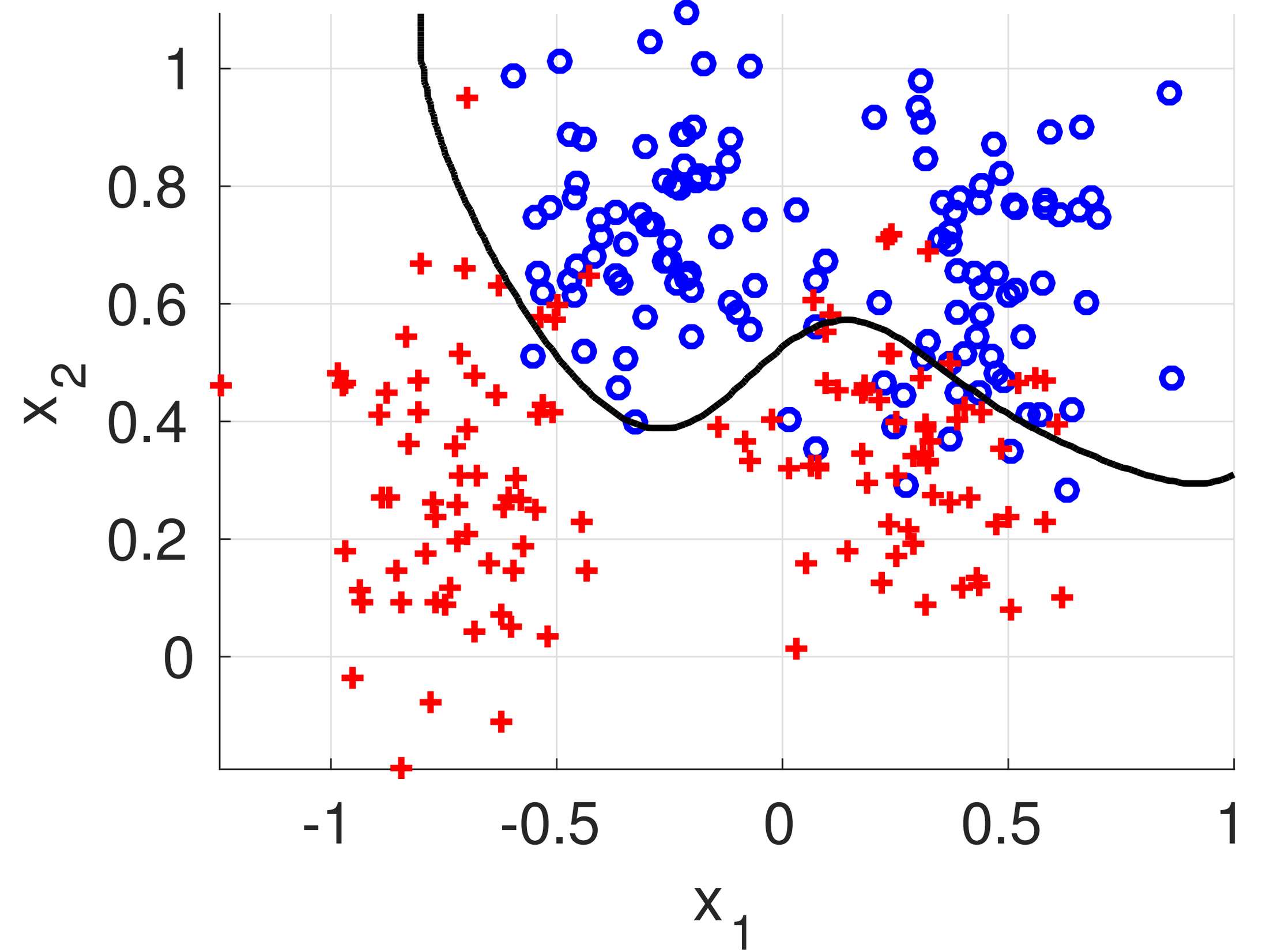

1
0.8
0.6
0.4
0.2
0
x_2
-1
-0.5
0
0.5
1
x_1

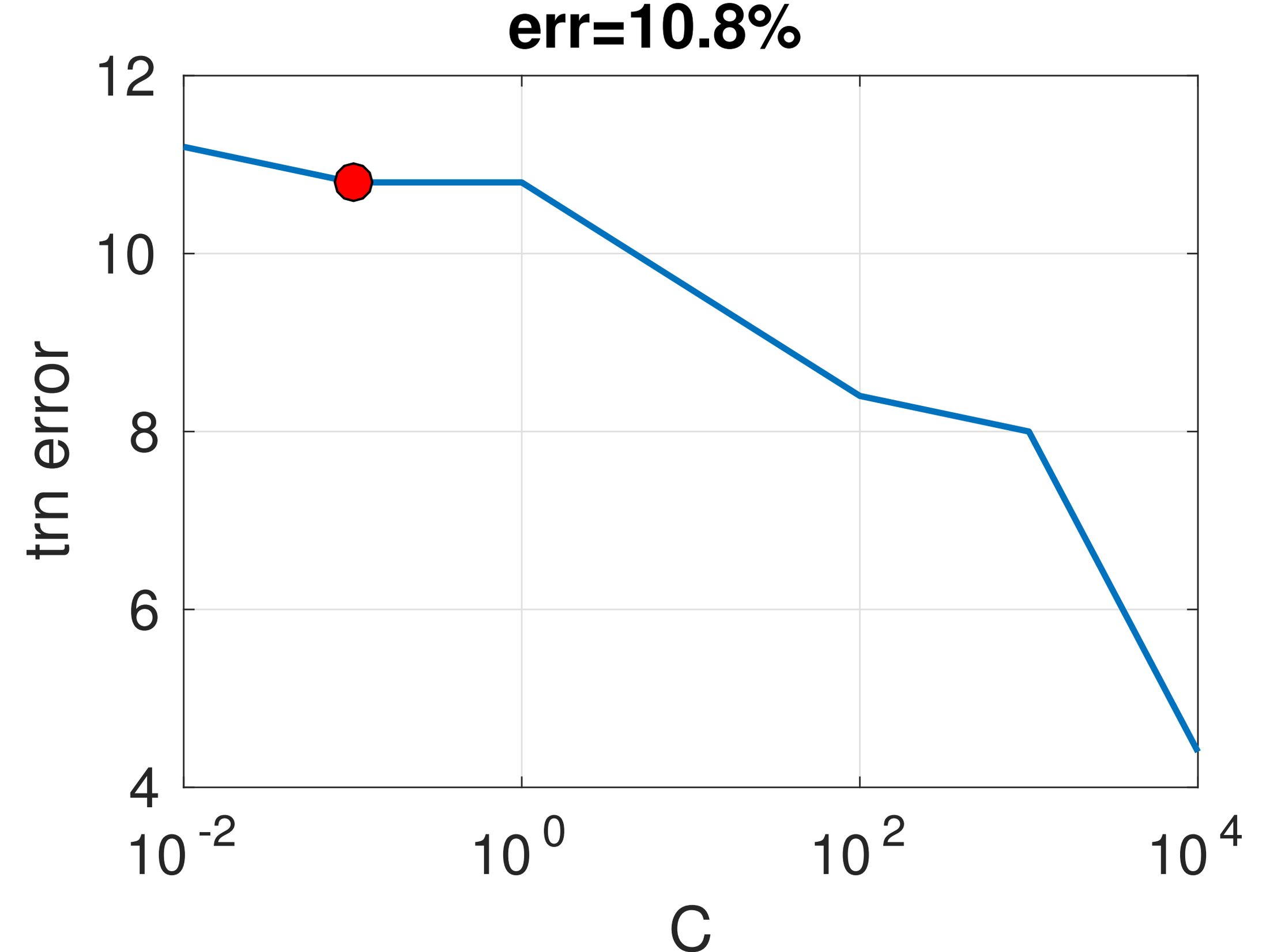

err=0.0%
15
10
5
0
trn error
10 -2
10 0
10 2
10 4
γ

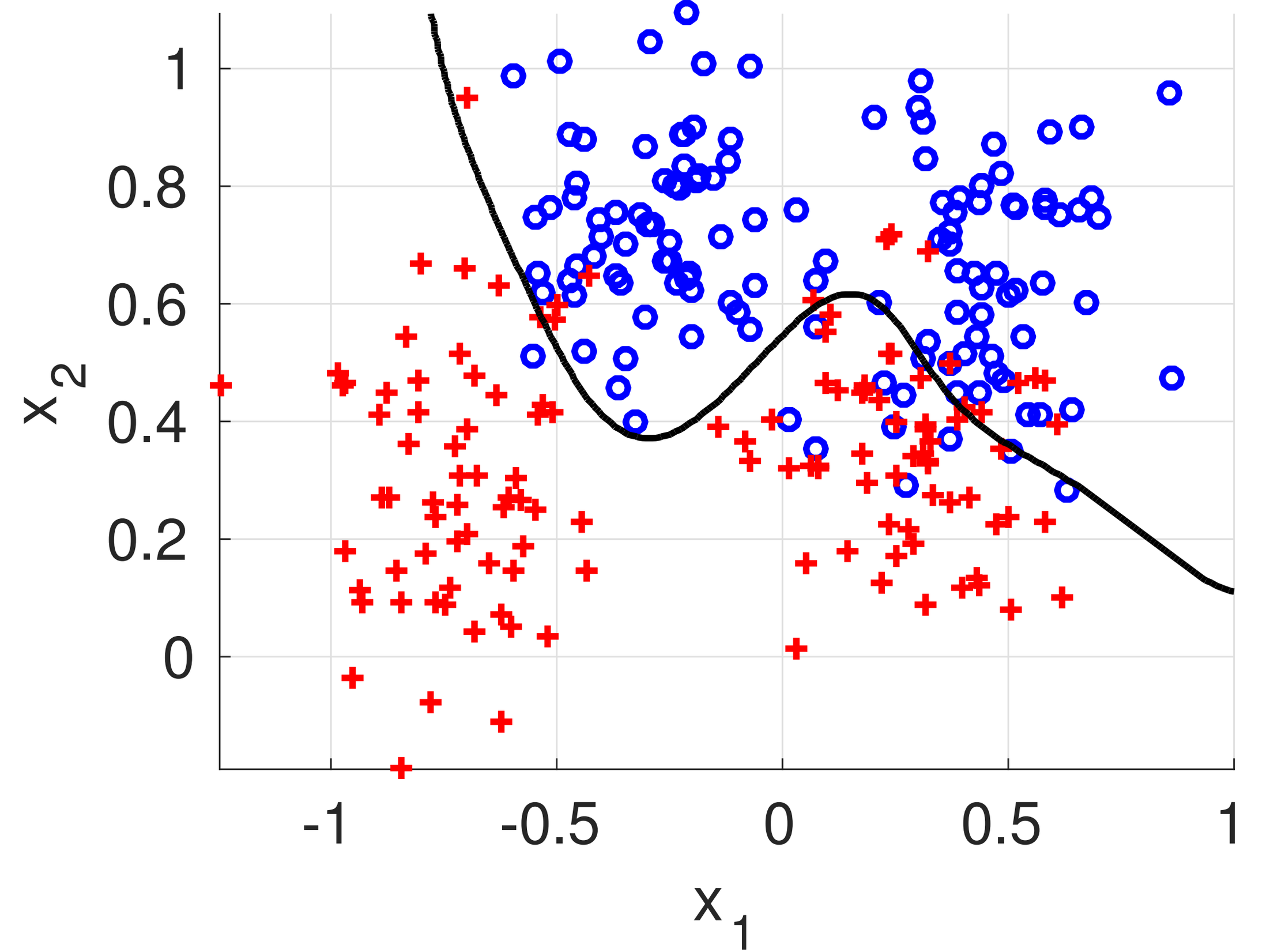

1
0.8
0.6
0.4
0.2
0
x_2
-1
-0.5
0
0.5
1
x_1

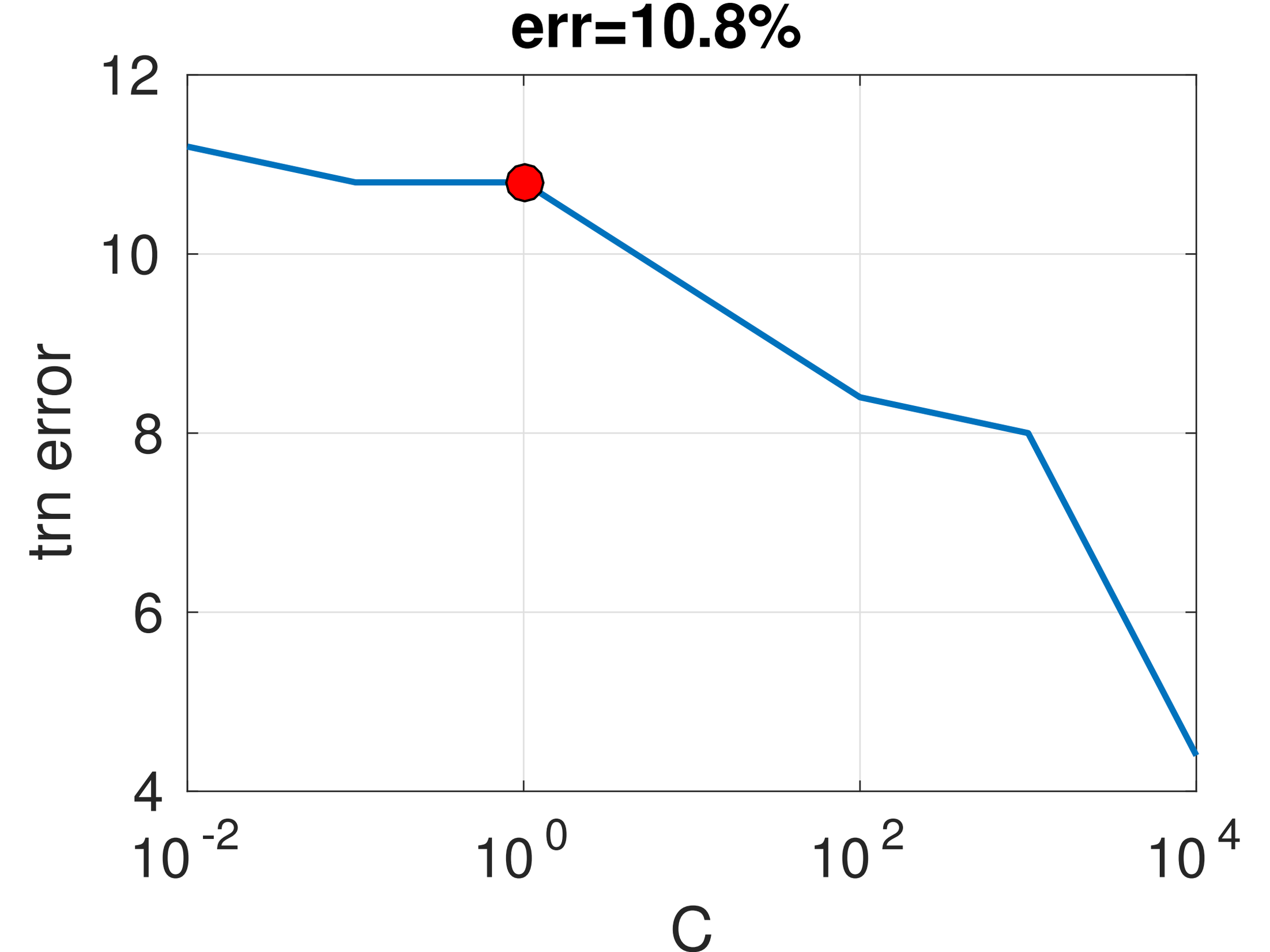

1
0.8
0.6
0.4
0.2
0
x_2
-1
-0.5
0
0.5
1
x_1

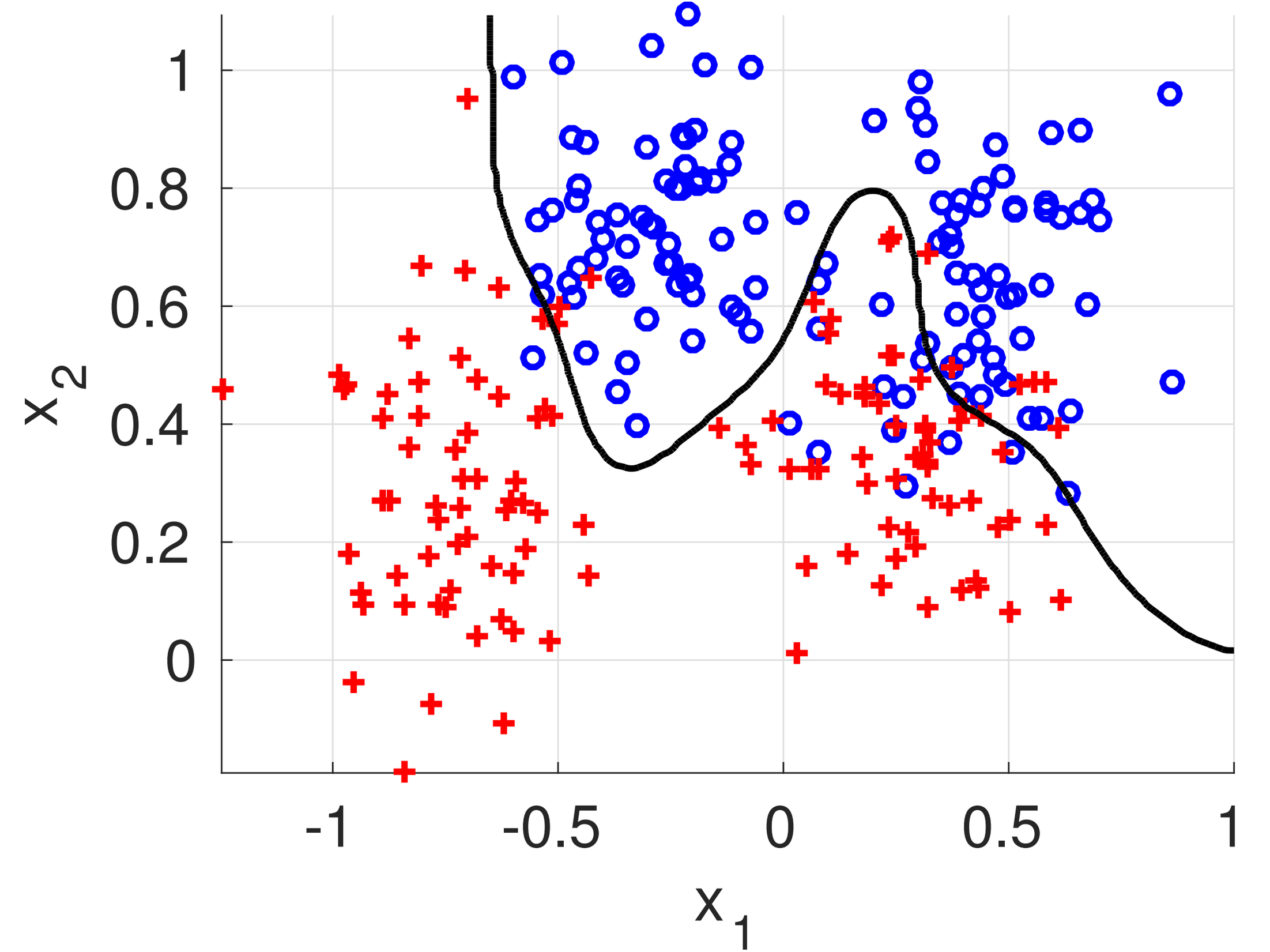

err=11.2%
12
10
8
6
4
trn error
10 -2
10 0
10 2
10 4
C

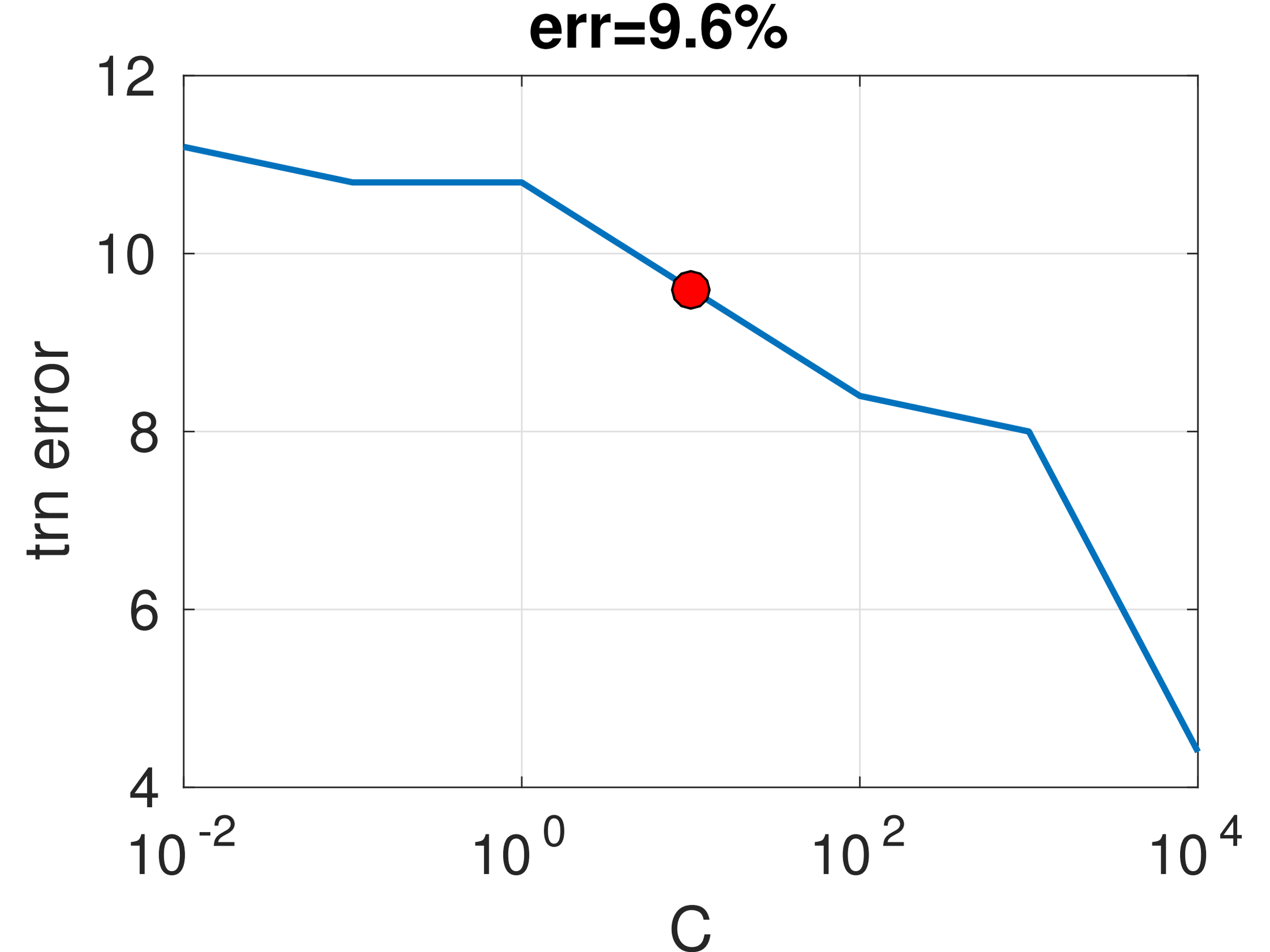

1
0.8
0.6
0.4
0.2
0
x_2
-1
-0.5
0
0.5
1
x_1

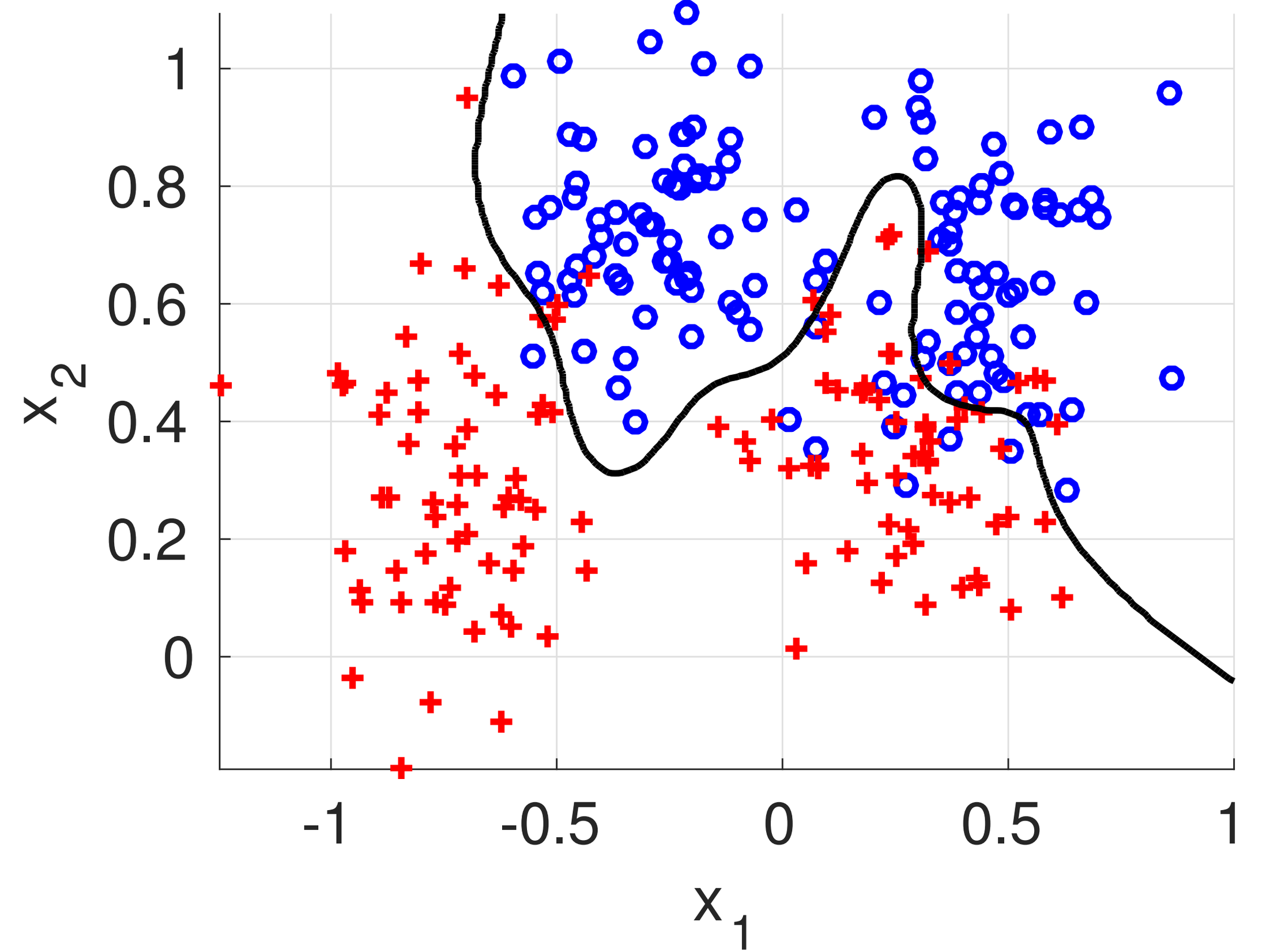

err=10.8%
12
10
8
6
4
trn error
10 -2
10 0
10 2
10 4
C

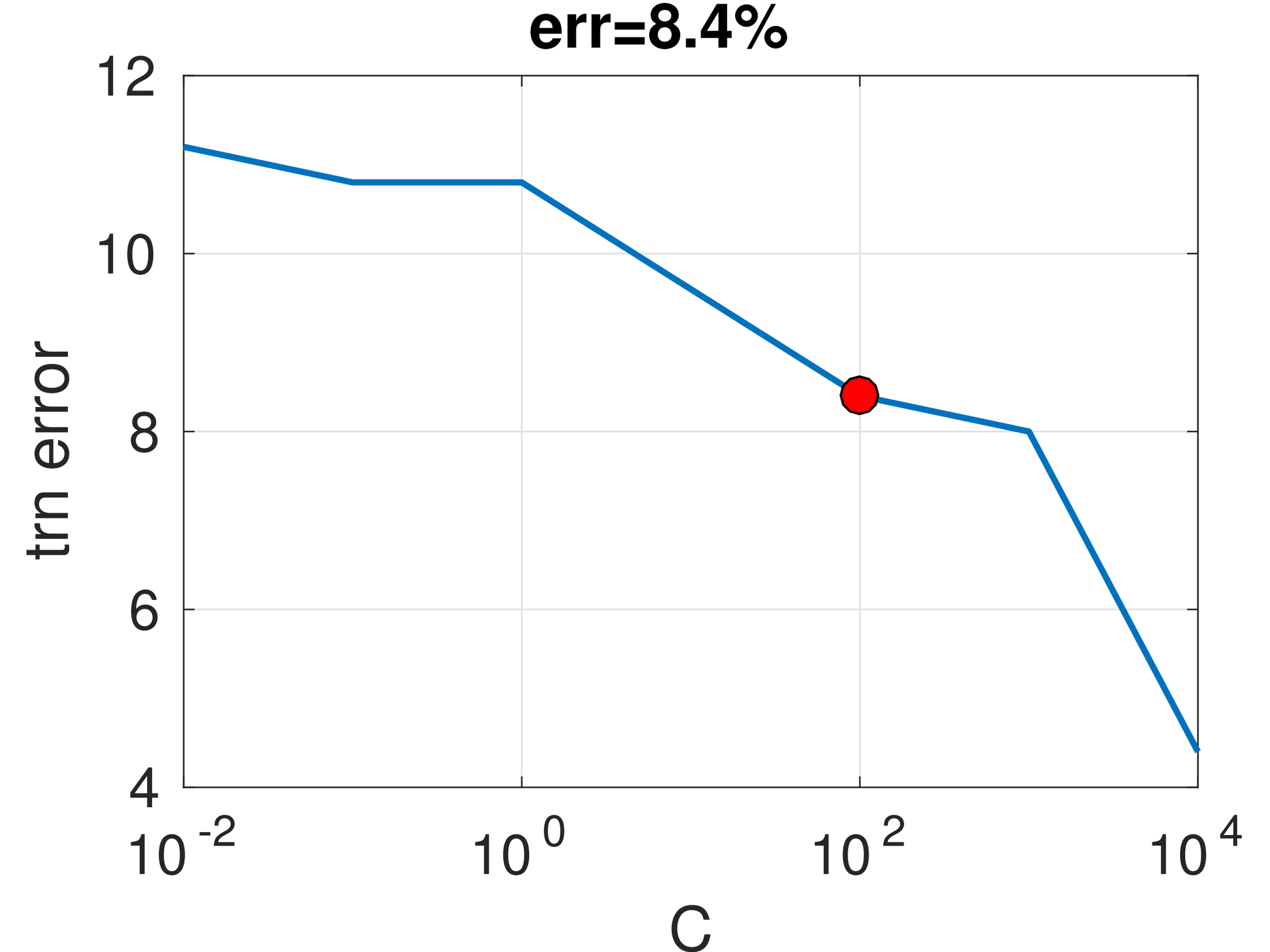

1
0.8
0.6
0.4
0.2
0
x_2
-1
-0.5
0
0.5
1
x_1

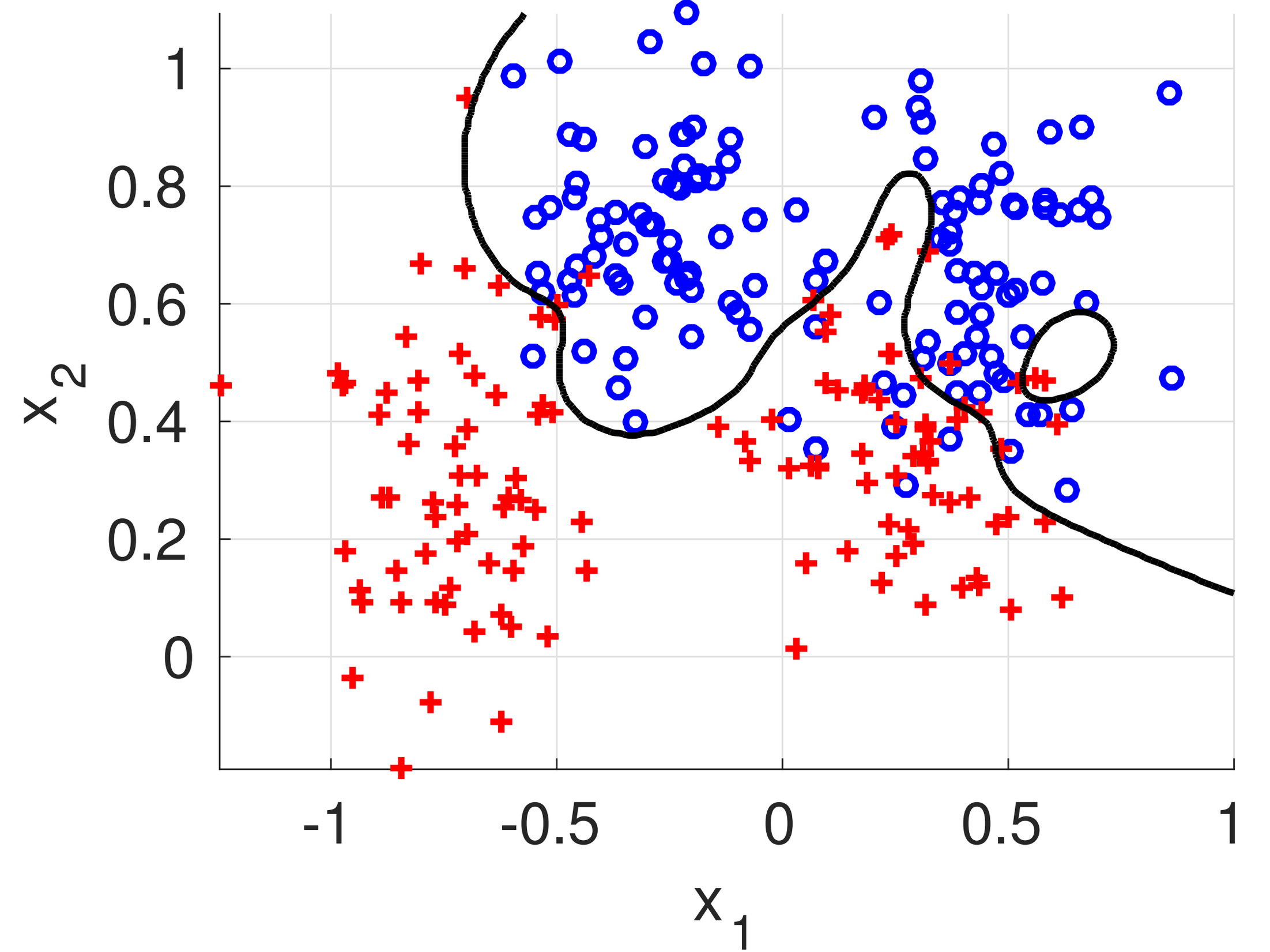

err=10.8%
12
10
8
6
4
trn error
10 -2
10 0
10 2
10 4
C

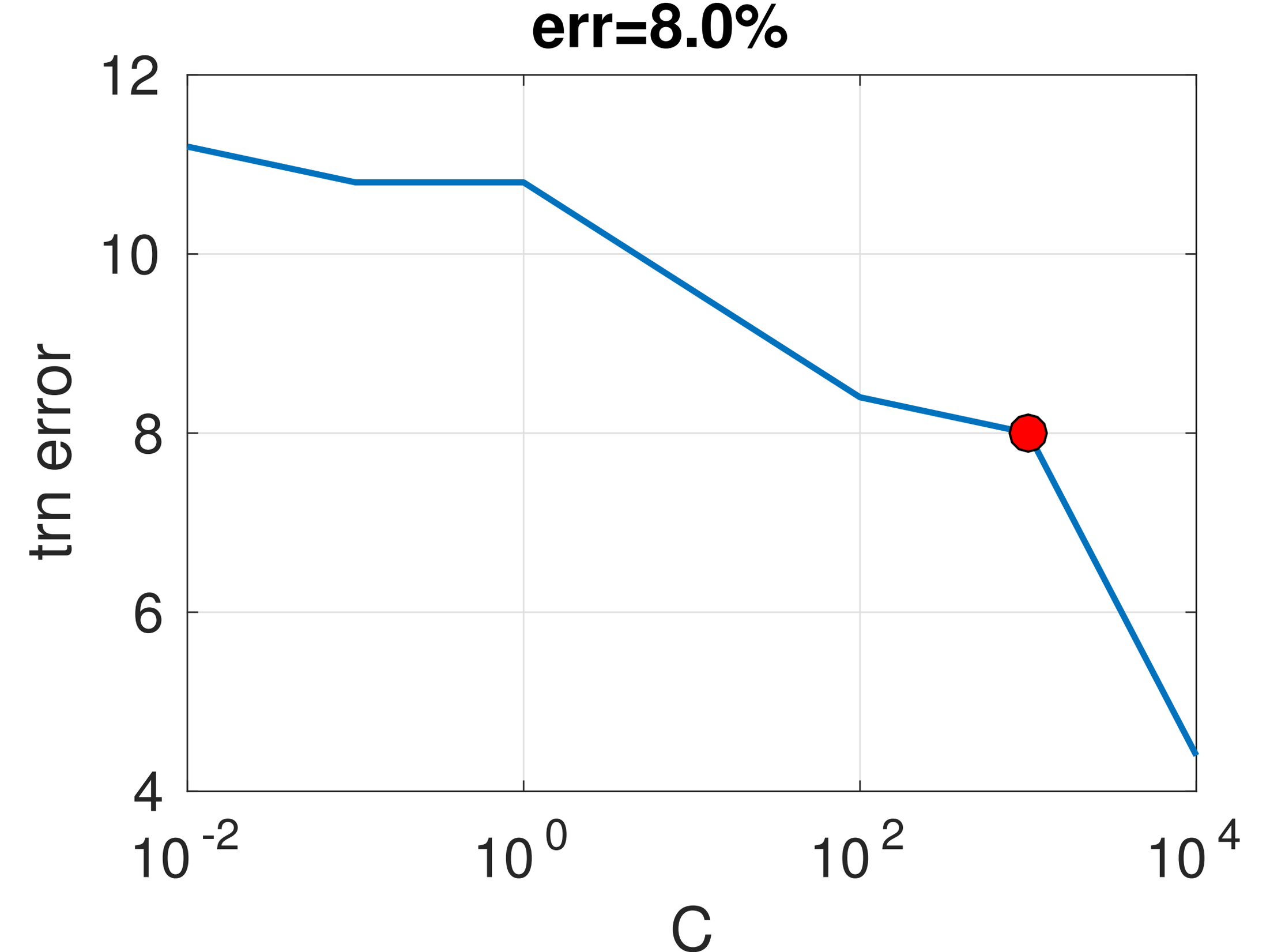

1
0.8
0.6
0.4
0.2
0
x_2
-1
-0.5
0
0.5
1
x_1

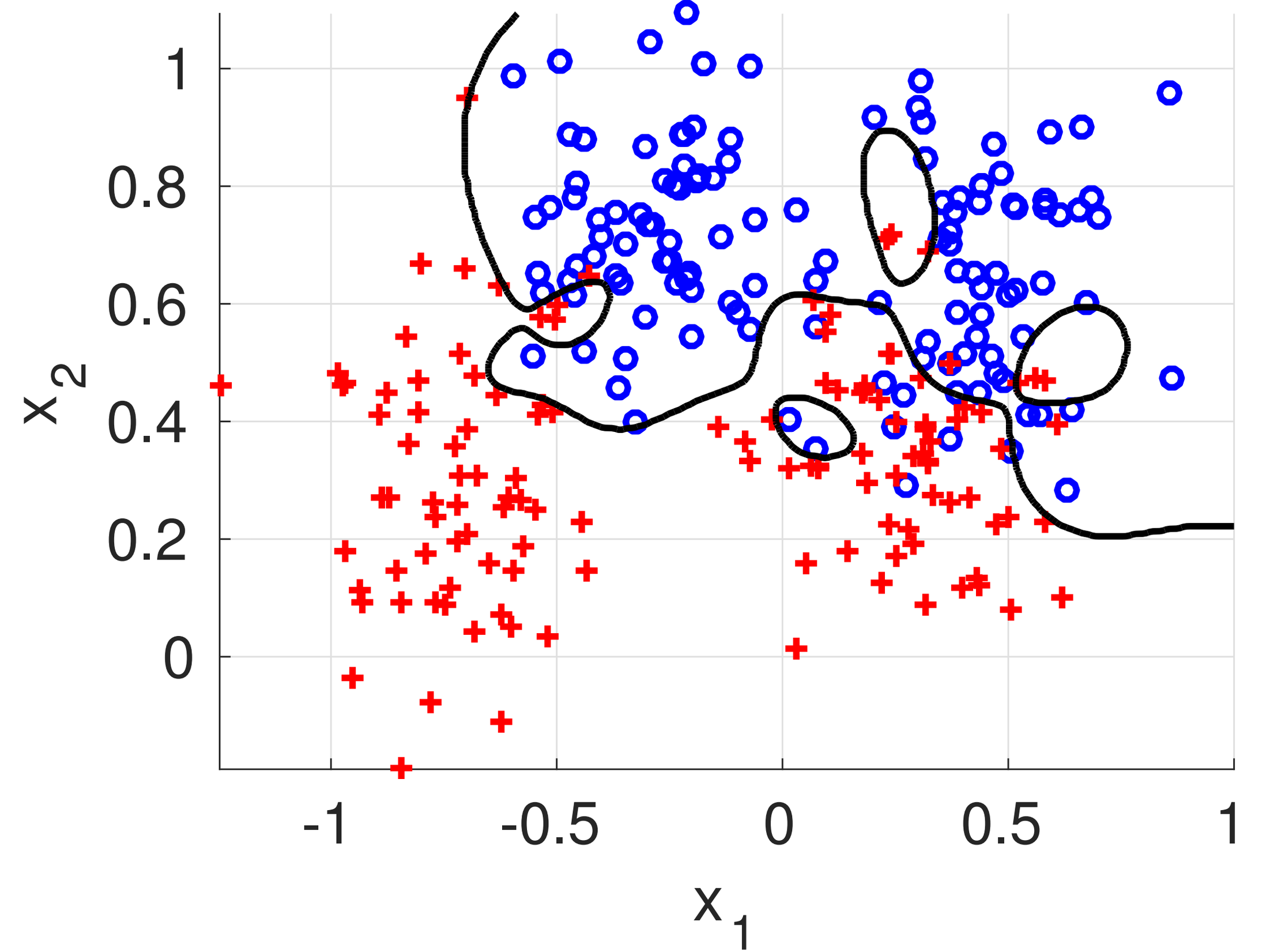

err=9.6%
12
10
8
6
4
trn error
10 -2
10 0
10 2
10 4
C

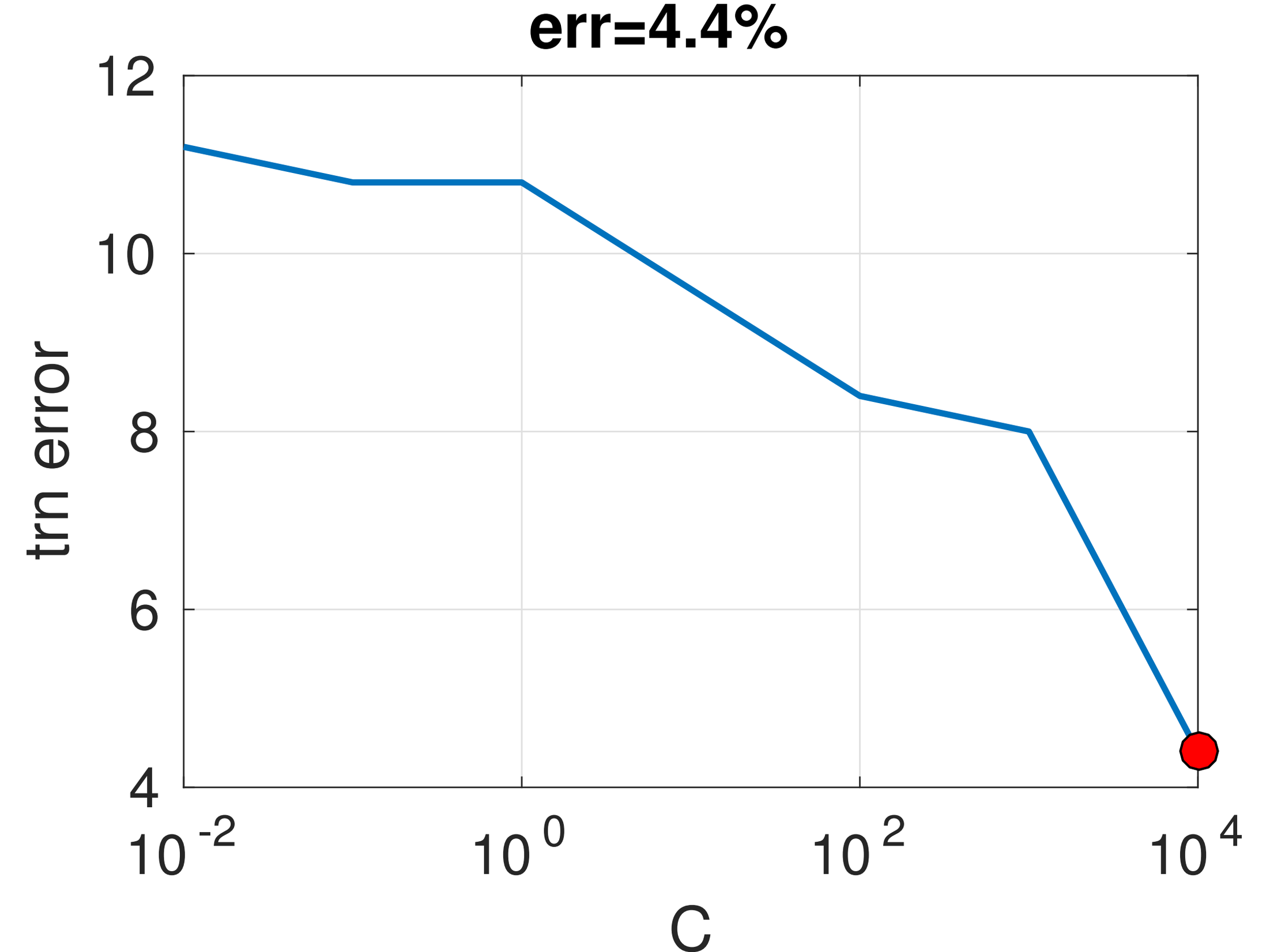

1
0.8
0.6
0.4
0.2
0
x_2
-1
-0.5
0
0.5
1
x_1


err=8.4%
12
10
8
6
4
trn error
10 -2
10 0
10 2
10 4
C


1
0.8
0.6
0.4
0.2
0
x_2
-1
-0.5
0
0.5
1
x_1


err=8.0%
12
10
8
6
4
trn error
10 -2
10 0
10 2
10 4
C


1
0.8
0.6
0.4
0.2
0
x_2
-1
-0.5
0
0.5
1
x_1


err=4.4%
12
10
8
6
4
trn error
10 -2
10 0
10 2
10 4
C